

VECTOR

APL2000 User Conference ...

- Thomson, reviewed by Reiter 31
- APL2000 Conference Reports 57
- Landau on Multiple Arguments 64
- McDonnell on Counterfeit Coins 93
- Cannon's Endlessly Rising Bach 104



*The Journal of the
British APL Association*

A Specialist Group of the British Computer Society

ISSN 0955-1433

www.vector.org.uk

Vol.18 No.3 January 2002

Contributions

All contributions to VECTOR may be sent to the Journal Editor at the address on the inside back cover. Letters and articles are welcome on any topic of interest to the APL community. These do not need to be limited to APL themes, nor must they be supportive of the language. Articles should be accompanied by as much visual material as possible (b/w or colour prints welcome). Unless otherwise specified, each item will be considered for publication as a personal statement by the author. The Editor accepts no responsibility for the contents of sustaining members' news, or advertising.

Please supply as much material as possible in machine-readable form, ideally as a simple ASCII text file on an IBM PC compatible diskette or via email. APL code can be accepted in workspaces from I-APL, APL+Win, IBM APL2/PC or Dyalog APL/W, or in documents from Windows Write (use the APL2741 TrueType font, available free from Vector Production), and MS Word (any version).

Except where indicated, items in VECTOR may be freely reprinted with appropriate acknowledgement. Please inform the Editor of your intention to re-use material from VECTOR.

Membership Rates 2001-2002

Category	Fee	Vectors	Passes
UK Private	£12	1	1
Overseas Private	£14	1	1
(Supplement for Airmail, not needed for Europe)	£4		
UK Corporate Membership	£100	10	5
Overseas Corporate	£135	10	
Sustaining	£430	10	5
Non-voting Member (Student, OAP, unemployed)	£6	1	1

The membership year normally runs from 1st May to 30th April. Applications for membership should be made to the Administrator using the form on the inside back page of VECTOR. Passes are required for entry to some association events, and for voting at the Annual General Meeting. Applications for student membership will be accepted on a recommendation from the course supervisor. Overseas membership rates cover VECTOR surface mail, and may be paid in sterling, or by Visa, Mastercard or JCB, at the prevailing exchange rate.

Corporate membership is offered to organisations where APL is in professional use. Corporate members receive 10 copies of VECTOR, and are offered group attendance at association meetings. A contact person must be identified for all communications.

Sustaining membership is offered to companies trading in APL products; this is seen as a method of promoting the growth of APL interest and activity. As well as receiving public acknowledgement for their sponsorship, sustaining members receive bulk copies of VECTOR, and are offered news listings in each issue.

Advertising

Advertisements in VECTOR should be submitted in typeset camera-ready format (A4 or A5) with a 20mm blank border after reduction. Illustrations should be photographs (b/w or colour prints) or line drawings. Rates (excl VAT) are £250 per full page, £125 for half-page or less (there is a £75 surcharge per page if spot colour is required).

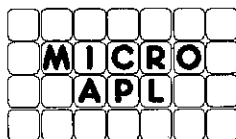
Deadlines for bookings and copy are given under the Quick Reference Diary. Advertisements should be booked with, and sent to Gill Smith, Vector Production, Brook House, Gilling East, YORK YO62 4JJ. Tel: 01439-788385.

Email: apl385@compuserve.com.

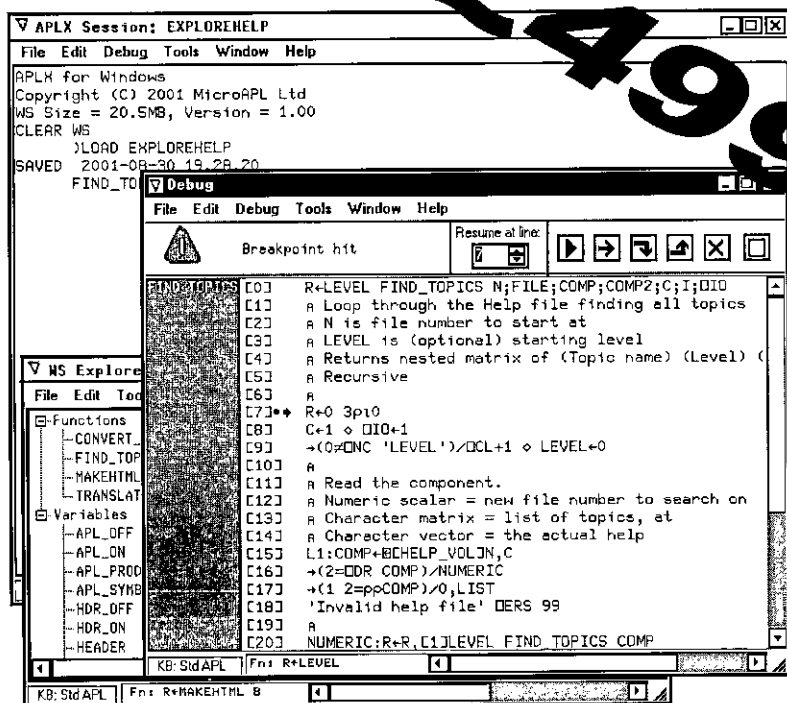
Contents

		Page
Editorial	Stefano Lanzavecchia	3
APL NEWS		
Quick Reference Diary		5
British APL Association News		
Notice of AGM and call for new committee		7
Update on A+ for Windows		8
News from Sustaining Members		9
APL Product Guide	Gill Smith	15
Review of "J: the Natural Language for Analytic Computing"	Cliff Reiter	31
The Education Vector		
Zark Newsletter Extracts	edited by Jon Sandles	38
Crossword		46
J-ottings 31: Symphony in J minor, op.31	Norman Thomson	52
RECENT MEETINGS – APL2000 at Naples		
Conference Report	Ray Cannon	57
Strategies for Dealing with Multiple Arguments: the Case for Names	Eric Landau	64
GENERAL ARTICLES		
What, in Computing, is Worth Learning?	Anthony Camacho	71
The Image2 Addon	Cliff and Zachary Reiter	80
TECHNICAL SECTION		
Hacker's Corner: Ghost Variables	Ray Cannon	90
At Play with J: The Counterfeit Coin Problem	Gene McDonnell	93
Bach's Endlessly Rising Canon	Ray Cannon	104
Extending the Domains	Phil Chastney	115
Index to Advertisers		127

APLX



for Mac OS 9 Windows 95/98/ME
Mac OS X Windows NT/2000/XP



The new, cross-platform, full-featured, user-friendly, GUI-enabled, cost-effective APL.
Check it out at <http://www.microapl.co.uk/apl>

APLX and MicroAPL are trademarks of MicroAPL Ltd. Mac is a trademark of Apple Computer Inc. Windows, Windows NT ME & XP are trademarks of Microsoft Inc. Price shown excludes VAT.

Editorial

by Stefano Lanzavecchia

"Bert, what utter nonsense! Why do you always complicate things that are really quite simple?"

"Kindly do not attempt to cloud the issue with facts"

"Supercalifragilisticexbealidocious"

Mary Poppins, the movie

XML (eXtensible Mark-up Language): did we really need yet another TLA (Three Letters Acronym)? I doubt I'd be able to give a crash course on XML, since despite its immense popularity, like many of you, I am still trying to understand what it is all about. First of all, a few facts: it is a language, that is, a verbal description of something formalised by a grammar; it has a simple, unambiguous syntax, that, admittedly, should be easily read by humans and parsed in a relatively easy way by computer programs; it's used to describe tree-structured documents in which the nodes can be given a kind and be decorated with named attributes; its natural form is a flat text string.

When seen from the distance, an XML document looks like the source of an HTML page, with all those angular brackets, those double quotes and those slashes. Not surprisingly so, since they are both children of SGML. In fact, I dare say that the success of XML owes much to the success of the World Wide Web and its original Hyper-Text Mark-up Language. While HTML has a well-defined niche of application, to me it is still unclear what kind of data really benefits from being represented in XML.

A quick survey on the Web shows that people are trying to put XML clothes on just about any form of data. One simple use of XML is to give structure to configuration files. Another popular application is the encoding of messages between applications: from the simple XML-RPC to the all-inclusive newborn SOAP (Simple Object Application Protocol) which permits two peers to send to each other function calls with the associated data and receive structured results in a way that is general, flexible, cross-platform and, if needed, firewall-friendly when using HTTP as the transport layer. Text documents, spreadsheets, slide-presentations can be saved in smart XML format by most of the modern Office program suites. Entire relational databases are exchanged in XML shape. In fact the flexibility of XML is such that the limit seems to be only in the fantasy of the developers involved.

But there are technical limitations. The one I am particularly wary of is that an XML document is flat text: after having been parsed by a program, it can be kept in memory as a tree of pointers, so that document traversal as well updates in place can be as efficient as they need to be. But, by definition, when serialised for exchange the document must be represented by a simple string. This means that even to modify a small leaf, it's necessary to read, parse, validate the entire document, and then rewrite it all once the changes have been applied. In a context where XML is used to exchange messages, or encode a small set of configuration options this is clearly not an issue. On the other hand, XML cannot be used to persist in an efficient way a multi megabyte relational database. To make the situation a bit worse from this same point of view, there's the fact that XML is quite verbose. Not only binary data must be converted to text (for instance using Base64 encoding) before it can be inserted in an XML stream, but even simple text and numbers seem to drown in a swamp of tags, somehow defeating the original intent of human readability of the document.

Another technical limitation is that parsing XML, despite the deceiving look, is not trivial, especially in languages such as APL, where character-by-character analysis of a large string is a performance monster. Simple subsets of XML can be parsed efficiently by vector algorithms, as shown by Arthur Whitney's ultra-concise parser coded in K, but the handling of all the subtleties of the XML specs is better left to experts and to fast scalar-oriented languages. Once again, modern APL programmers, instead of re-inventing the usual squared wheel, can resort to tools written in other languages. Microsoft's XML parser, to mention one, is a COM component and it's completely free: Jonathan Barman showed in a recent Vector issue (17.4) how simple it is to use it from Dyalog APL. In the Unix world, many Java parsers exist and Sharp APL's Java can probably interface to them via its Java front-end.

Many grand unifying theories have been proposed since the dawn of modern computing, in the field of cross-platform, inter-process, inter-application exchange of information. Every time the developers were promised that it was only a matter of years before the new paradigm conquered the whole world and washed away all the previous troubles, finally concluding the chaotic middle-age of custom solutions. If you are preparing a presentation to the board with the future plans of your development team, don't forget to spice it up with a bit of XML and some extra HTTP, XSD, XSLT, SOAP, WSDL, UDDI, SAX, XPath, XInclude (such wonderful acronyms!), WebServices and all the other related technologies. They are being advertised as the brand new ultimate magic tool to solve all the problems in computing: they play a fundamental role in Microsoft's .NET platform, in Sun's JavaOne, and other big players such as IBM are also funding research in the field. I will suspend the verdict until I know more and I have

experienced pros and cons, but, nevertheless, I will continue to spend time investigating, experimenting, understanding, reporting on, and, whenever possible, having fun with XML.

Quick Reference Diary

Date	Venue	Event
6-9 Feb 2002	Helsinki, Finland	Forests in Finland VI
6-9 April 2002	Toronto, Canada	Toronto SIG with APL2000 joint meeting
24 May 2002	RSS, London	AGM and Vendor Presentations
22-25 July, 2002	Madrid, Spain	APL2002 Conference

The Toronto meeting is provisionally scheduled with a tutorial day on the Sunday and two full days of presentations to follow. Please check the Toronto SIG website or email Richard Proctor for more details.

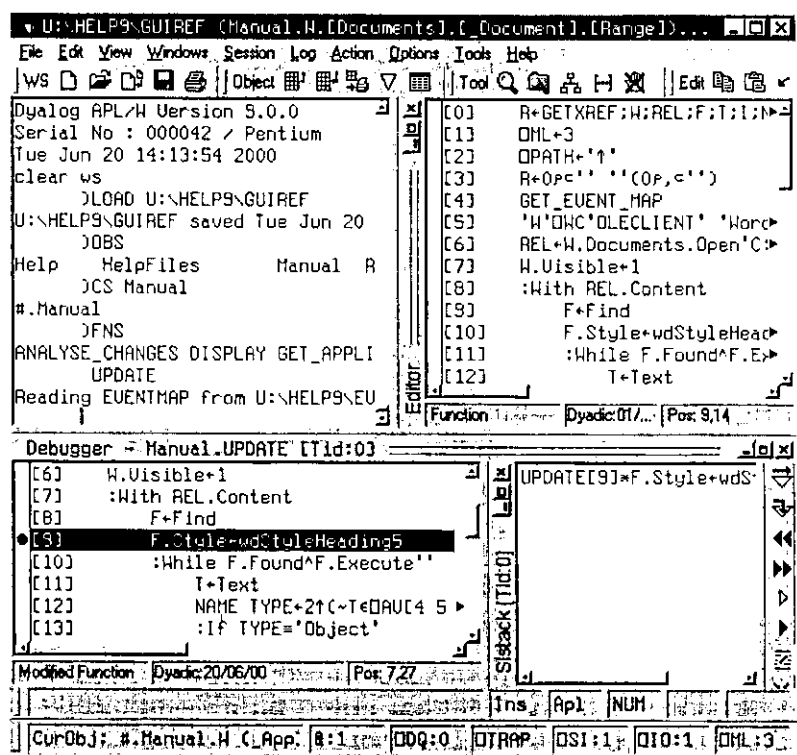
The BAA AGM will (as usual) be followed by several Vendor presentations. MicroAPL and Dyadic Systems will definitely be there, and there will be a progress report on the A+ interpreter for Windows.

Dates for Future Issues of VECTOR

	Vol.18	Vol.19	Vol.19
	No.4	No.1	No.2
Copy date	8th March	31st May	6th Sept
Ad booking	15th March	7th June	13th Sept
Ad Copy	22nd March	14th June	20th Sept
Distribution	April 2002	July 2002 (Madrid)	October 2002

dyalog APL

The Definitive APL for Windows™



Version 9 - an even better IDE(A)

<http://www.dyadic.com>

Dyalog Systems Limited, Riverside View, Basing Road, Old Basing, Basingstoke,
 Hants. RG24 7AL, United Kingdom.

Tel:+44 1256 811125 Fax:+44 1256 811130 Email: sales@dyadic.com

Microsoft is a registered trademark and Windows and the Windows Logo are trademarks of Microsoft Corporation

British APL Association Annual General Meeting

*2.00 p.m. Friday 24th May 2002
Royal Statistical Society, Errol Street, London EC1Y 8LX*

Agenda

- 1 Minutes of Annual General Meeting 2001
- 2 Chairman's Report
- 3 Subscriptions
- 4 Audited Statement of Accounts – Treasurer's report
- 5 Questions to Committee
- 6 Appointment of Auditor
- 7 Other business

Other business cannot be discussed without giving notice and may require that a notice is sent to members. If you wish to raise a matter of other business, please let the secretary know by 31st March 2002 at the latest.

Anthony Camacho, Hon Sec, January 2002

Vendor Presentations

There will be a full programme of Vendor presentations immediately following the AGM, with a teabreak around 3:00. MicroAPL and Dyadic Systems will definitely be among those presenting the latest features of their APL systems.

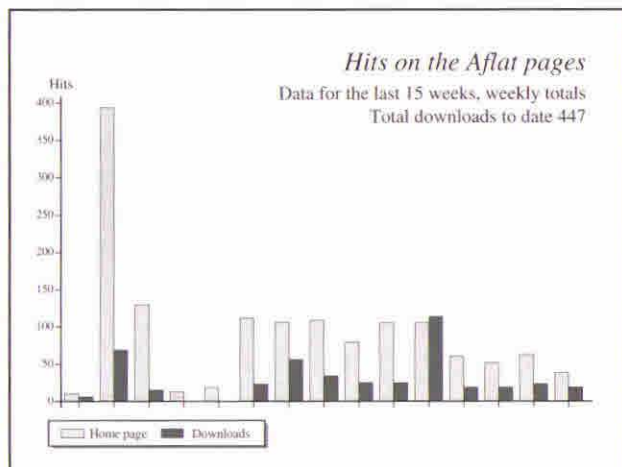
Committee Nominations for 2002-2003

Nominations are invited for next year's committee, to hold office from 1st May 2002 to 30th April 2003. Please send the name of the nominee (who must have consented to stand), the proposer and the seconder (all must be members of the Association) to the secretary, Anthony Camacho, 11 Auburn Road, Redland, Bristol BS6 6LS or by email to acam@tesco.net. Nominations must be in the secretary's hands before the 31st March 2002. If there is more than one nomination for any post there will be a postal election.

The A+ Project – Progress Report

Current Status

The first thing to say is that there has been a steady trickle of interest in the website at www.vector.org.uk/aflat/:



There was a major peak when Vector 18.2 was mailed, followed by a rather flat patch, followed by a steady download rate at about 20 per week.

The online manual has been getting a similar smattering of hits, spread quite evenly across all the pages.

Several people have mailed in substantial

pieces of A+ code, and Sasha Skomorokhov has nearly completed the 'Weigang' translator which will turn ASCII-mode A+ to APL mode and back.

Of course, nearly everyone would like a 'real' APL session; only Sasha is completely happy with Emacs and the console we already provide!

Next Moves

At the Vector working group on January 19th, Stephen Taylor volunteered to manage the next phase of the A+ project, with the object of having APL in use in ten British schools at the beginning of the 2003-4 school year. We will shortly be asking the BAA for the funds to complete the port of the interpreter (which must now lose the last vestiges of its Unix heritage) and to begin the design work on the kind of environment we believe will be necessary to achieve Stephen's goal.

Sustaining Members' News

Dyadic Systems Ltd

Dyalog.Net

Dyalog.Net is now in the final Beta Test stage and will be ready soon after the availability of the Microsoft .NET Framework, which is imminent.

Dyalog APL for the Microsoft .NET Framework is the result of a collaborative effort between Dyadic and Microsoft that began in December 1999. Working alongside Microsoft as the .NET Framework code has been developed, Dyadic has integrated Dyalog APL with the .NET product so that all of the features and capabilities of this new computing platform will be available to Dyalog APL programmers the moment .NET is shipped by Microsoft.

The Microsoft .NET Framework is a highly object-oriented technology whose functionality is implemented in terms of .NET Classes. It comes with a huge number of built-in classes for performing all manner of tasks, including an object-oriented implementation of the Windows GUI and a set of classes for developing Internet applications. A third-party .NET class industry is already emerging.

Building upon the namespace technology and syntax that is central to Dyalog APL, Dyalog.Net provides syntax for using .NET classes from APL that closely follows the syntax used by other programming language. Dyalog.Net makes it as easy to use .NET classes from APL as from C# or VB. Furthermore, sample C# and VB code published in help files and manuals can readily be transposed into the APL environment.

.Net Classes

Not only is it easy to use .NET classes in an APL application, it is also very simple to write new classes in Dyalog.Net which can then be exploited by other programming languages and development environments. For example, you could write a class that exposed a whole set of actuarial formulae and calculations (easily and efficiently expressed in APL) which can then be called directly by an application written in Visual Basic, or from an Excel spreadsheet.

APLScript, DLLs and .EXEs

Dyalog.Net includes APLScript, a scripting version of Dyalog APL that supports the use of APL in character based ASP.NET Web Pages and Web Service script files. APLScript may be used with or without traditional APL workspaces, giving you a variety of options for implementation.

.NET Classes written in Dyalog APL are saved for you as DLL files and are indistinguishable from classes written in other languages. You deploy a Dyalog.Net class by simply supplying the DLL file together with DYALOG10.DLL, the Dyalog.Net engine.

Also new in Dyalog.Net is the ability to package a workspace as a .EXE that can be distributed as a stand-alone program depending only upon DYALOG10.DLL. This facility applies whether or not your application makes use of the .NET Framework.

Internet Applications

The .NET Framework is not just about classes; it includes some major new technologies for implementing Internet and Intranet applications and Web Services. Through Dyalog.Net, all these new technologies are immediately available to the APL programmer.

ASP.NET is a new Microsoft technology for publishing web pages. ASP.NET supports server-side scripts, including APL scripts, that create web pages on the fly. Now you can very easily harness the power of APL to the Internet and generate pages that are not just simple database lookups, but contain the results of really complex calculations.

Web Services are classes that expose their functionality over the Internet. Typically, a Web Service contains methods that take arguments and return results. You can think of a Web Service as a set of remote procedure calls, but calls that are processed over the Internet. Dyalog.Net allows you to develop, debug, test and implement Web Services in APL. This means that, subject to security controls, you can quickly and easily make your APL functions directly accessible to any .NET program running on any computer that is connected to the Internet.

Whether implemented as Web Pages or Web Services, Dyalog.Net APL code is controlled and run by the ASP.NET engine inside Microsoft IIS. Dyalog.Net web applications are managed in the same way as other Internet applications and require no specialized knowledge for deployment.

Dyadic has surely removed the final barriers for the successful exploitation of APL on the Web; the rest is up to you!

MicroAPL Ltd

Our new cross-platform APLX range of interpreters for Windows, MacOS, Linux and AIX – announced in Vector 18.2 – has received an enthusiastic response from APLers around the world. (Our favourite comment came from a user who

described it as 'my best Christmas gift'.) We thought we would focus in this issue of Vector on two areas which we are often asked about.

Portability of APL code

The *APLX* interpreter itself is 100% upwards-compatible with our previous *APL.68000* and *APL Level II* products. This means that the language specification is very similar to IBM's *APL2*, with the addition of a number of commercially-oriented extensions (but without complex number support). IBM *APL2* workspaces can be imported using *)IN*. Users have also reported that converting code from *APL+Win* is generally straightforward. See our web page <http://www.microapl.co.uk/apl/spec.html> for a summary of the language features.

Of course, the real compatibility issues arise not so much in the core *APL* language, but in the operating-system and GUI support. User-interface programming in *APLX* is based on the *□WI* system function, and we have tried very hard to make this as compatible as possible across *MacOS* and *Windows*. This means that *APLX* applications can easily be written to be platform-independent, adapting automatically to the look-and-feel of *Windows*, *MacOS 9* or *Mac OS X*.

We have also ensured that old *APL.68000* and *APL Level II* *Mac* workspaces can be loaded and run with very little change, although in some cases it will be necessary to *)COPY* new versions of some of the utility workspaces supplied with the product. The *□WI* implementation of *APLX* is upwards compatible with our previous *Mac* products, with a few minor exceptions.

Existing *APL+Win* GUI code, written using the *□WI* facility provided by *APL2000, Inc*, will require some modifications to run under *APLX*. However, we have tried to retain as much compatibility as possible, so that in most cases it is reasonably straightforward to take an existing *APL+Win* GUI application and convert it to run under *APLX* on *Mac OS 9* or *OS X* (or *Windows*, of course). *Dyalog APL* applications are likely to require bigger changes.

Runtime licences

MicroAPL pioneered the concept of freely-distributable runtime versions of *APL*, and many users have asked whether this facility will be available for *APLX*. The answer is that we will shortly be announcing an *APLX Packager* which takes your *APLX* workspace, and converts it to a standalone *Mac* or *Windows* application, which you can distribute without paying royalties to *MicroAPL*.

If you would like to be kept informed of this and other developments, please fill in the feedback form on our web site <http://www.microapl.co.uk/apl>.

APL2000 Inc

APL+Win Version 4.0

APL2000 Inc. is pleased to announce the availability of APL+Win version 4.0 for Windows 95/98/NT/ME and 2000 in the United States and Canada. The International Version will be available shortly.

The most remarkable enhancement to this version of the software is the addition of the APL.Grid object to the APL+Win class library. This ActiveX control provides for the display of APL derived data in a familiar spreadsheet format. An APL.Grid workbook can contain multiple pages with the customary tabbed interface, complete with row and column headings, fixed panes, cell decoration, and other features familiar to users of commercial spreadsheet programs.

While the grid object does not contain a calculation engine, direct transfer of array-oriented APL data into and out of the grid allows the interpreter to easily handle all computational needs. Customizable formatting for text, numbers, dates, times and currency data types is provided. Localization to native standards for dates, times, currency, and numerics is supported, as is specialized formatting based on value (e.g., negative currency amounts).

Most all aspects of the grid are accessible under program control, but a standard user interface is provided as well. Cell content is easily edited by a user and verifiable by the program. Cell selection is accomplished with the usual actions, and multiple cell ranges are supported. The potential for a truly unique user interface is enhanced by several special cell types. Rather than data, a cell can contain a button, a combo box, or a check box. Tooltips can be provided to explain the content or function of specific cells, and overlay images can be set into cells to provide visual cues.

APL.Grid, as an ActiveX control in APL+Win, also benefits from those enhancements made to almost all controls in the class library. Chief among these are new properties which make the provision of truly context-sensitive help easier. Version 4.0 also enables use of the question mark button on form borders to allow for the implementation of "What's This?" help. All visible controls now have a `noredraw` property which can help control screen flicker when updating complex forms, and the new `imagelist` operation of the `Draw` method allows easy visual customization of all controls.

Other areas of the system have been enhanced in this latest release. Interaction with printers has been augmented by better connection with the standard Windows print dialog boxes. Printer capabilities such as paper bins and collation are now easily available to your functions. You can easily find or set the default printer, and can now detect and use printer resources connected to your application's station by a network. Specific controls have had their capabilities enhanced as well. The Combo box has been extended to support images and indents; the RichEdit control supports bullets, indents, and paragraph numbering; and the Selector (tab control) offers new options for tab appearance and behaviour. More controls now benefit from enhanced tooltip support, and overlay images have been added to more controls' display.

Enhancements for those in special circumstances round out the list of features in this release. Large applications may benefit from a greatly increased maximum file size for both component and native files, and a doubling of the file tie quota. Applications using ActiveX objects can exploit an expanded calling syntax which can chain method calls, name parameters explicitly, and reference defined constants. Your applications have better control of their INI files, both in accessing them and creating them, and can initialize themselves without displaying the APL+Win banner.

And last but not least, APL+Win now comes with a full complement of printed documentation. The three-volume perfect bound set contains a newly revised User Manual, Systems Functions Manual, and a Windows Reference manual. The text of the printed documentation is still distributed on the APL+Win CD, along with a comprehensive help system, practical utilities, and sample workspaces.

For the latest on this release, or to learn about our other products, please visit us at <http://www.apl2000.com>

New APL2000 Distributor for the UK

APL Systems IDC SL of Madrid, Spain have been asked to provide sales and support services for APL2000, Inc., a COGNOS, Inc. company, in the United Kingdom. The Bloomsbury Software Company Limited, the former APL2000 distributor, have been merged with WebGain Ltd. Fred Honea, the contact at APL Systems, says he is looking forward to working with the APL users in the UK. Fred adds that APL will provide maintenance and support in the UK in addition to providing made to order courses of instruction. If anyone needs any information or has a question, please call or email Fred at +34 60 680 5949 or fhonea@apl2000.net.

The Bloomsbury Software Company Ltd. have long been a strong supporter of APL in the United Kingdom. In response to changing market conditions, Bloomsbury have chosen to concentrate on the webcentric aspects of their business. This was accomplished by an alliance with WebGain Ltd. and a cessation of APL activities in the UK earlier this year. We are all fortunate that APL Systems IDC SL have been able to step into the breach and offer both sales and support to our APL users who have formerly depended on Bloomsbury.

Trial Version of APL+Win

APL2000, Inc. are currently studying the possibility of releasing a trial version of APL+WIN. It would be based on Version 3.6 and would not include a Runtime version, adf files, nor hardcopy manuals. The price would be about one fourth that of the full version and, of course, the upgrade to the full version 4.0 would carry an attractive price. It could be downloaded or packaged and mailed as a CDROM.

Causeway Graphical Systems Ltd

Perhaps the Internet really is changing the way some of us do business. The received wisdom has always been that you cannot "make a million dollars with ProductX" unless you first burn a huge amount of someone else's money promoting it. Causeway is taking the "better mousetrap" approach which is a complete opposite – build your product, write about it on the internet, wait for the world to find you. It worked for Google, and it seems to be working for us. GraPL.NET really is just *RainPro* in a box, so it is first and foremost a programmer's tool. Programmers just go trawling the internet (unlike managers who read the expensive promotions in glossy journals) and if they like what they see, they grab the demo, try it, then tell the manager to go and buy it.

This is good news for all you budding millionaires out there. APL is a way that a small company can build a better mousetrap and get rich selling it while staying small. APL lets you keep control of your codebase and your vision, and allows you to move quickly to respond to customer needs. What's more you can test your product in a very demanding (but very tolerant) market segment; the APLers of the world will not only road-test it for you – they will most likely fix it when it breaks and mail you the fix!

This way, we get all the benefits of Open Source development (in a strictly limited domain) and at the end a product that knocks the socks off all those 'real' programmers out there who have never heard of APL. Thanks guys!

The Vector Product Guide

compiled by Gill Smith

VECTOR's exclusive Product Guide aims to provide readers with useful information about sources of APL hardware, software and services. We welcome any comments readers may have on its usefulness and any suggestions for improvements.

We reserve the right to edit material supplied for reasons of space or to ensure a fair market coverage. The listings are not restricted to UK companies and international suppliers are welcome to take advantage of these pages.

For convenience to readers, the product list has been divided into the following groups ('poa' indicates 'price on application'):

- Complete Systems (Hardware & Software)
- APL and J Interpreters
- APL-based Packages
- Consultancy
- Other Products
- Overseas Associations
- Vendor Addresses
- World Wide Web and FTP Sites

Every effort has been made to avoid errors in these listings but no responsibility can be taken by the working group for mistakes or omissions.

We also welcome information on APL clubs and groups throughout the world.

Your listing here is absolutely free, will be updated on request, and is also carried on the Vector web site, with a hotlink to your own site. It is the most complete and most used APL address book in the world.

Please help us keep it up to date!

All contributions and updates to the Vector Product Guide should be sent to: Gill Smith, Brook House, Gilling East, York, YO62 4JJ. Tel: 01439-788385, Email: apl385@compuserve.com

COMPLETE APL SYSTEMS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Dyadic	IBM RS/6000 MD320	11,736	APL POWERstation (Greyscale) 27.5 MIPS, 7.4 Mflops RISC Processor 8Mb RAM, 120Mb Disk 19" 1280x1024 Greyscale Graph Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD320	13,817	APL POWERstation (Colour) 27.5 MIPS, 7.4 Mflops RISC Processor 8Mb RAM, 120Mb Disk 16" 1280x1024 Colour Graphics Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD320	22,656	Advanced APL POWERstation 27.5 MIPS, 7.4 Mflops RISC Processor 16Mb RAM, 320Mb Disk, 150Mb Tape 16" 1280x1024 Colour Graphics Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD520	37,114	APL POWERsystem (8-users) 27.5 MIPS, 7.4 Mflops RISC Processor 16Mb RAM, 320Mb Disk, 150Mb Tape CD-ROM Drive, 16 Ports AIX, Dyalog APL (2-8 user licence)
	IBM RS/6000 MD530	72,054	APL POWERsystem (16-users) 34.5 MIPS, 10.9 Mflops RISC Processor 32Mb RAM, 1.34Gb Disk, 2.3Gb Tape CD-ROM Drive, 16 Ports AIX, Dyalog APL (8+ user licence)
	IBM RS/6000 MD540	122,842	APL POWERsystem (32-users) 41 MIPS, 13 Mflops RISC Processor 64Mb RAM, 1.7Gb Disk, 2.3Gb Tape CD-ROM Drive, 32 Ports AIX, Dyalog APL (8+ user licence)
Optima	IBM Compatible	poa	Complete networked or stand-alone solutions including configuration installation, maintenance and commissioning.

APL INTERPRETERS

COMPANY	PRODUCT	PRICES(£)	DETAILS
APL Borealis Inc.	Dyalog APL	poa	Distributor of Dyalog APL products from Dyadic
	APL2000	poa	Distributor of APL2000 products
APL Systems IDC SL	APL+Win v3.6 Full version	1895	A 32-bit Windows-hosted interpreter that runs under Windows 95/98/ME/NT/XP
	Upgrade to v3.6 from v3.x	474	
	to v3.6 from APL+Win v2.x	1034	
	to v3.6 from APL+Plus III v1.x	1249	
	Migration to v3.6 from APL+Plus II or APL+Plus PC	1464	
	APL+Unix		(Please contact us for details.)
Beautiful Systems	Dyalog APL/W for Windows	poa	US Distributor of Dyalog APL products from Dyadic.
	Dyalog APL for Unix	poa	See Dyadic listing for product details.
Dinosoft Oy	Dyalog APL/W for Windows	poa	Finnish distributor of Dyalog APL products.
	Dyalog APL for Unix	poa	See Dyadic's listing for product details.
Dittrich & Partner	APL+Win	poa	Cognos/APL2000 Inc products
	Dyalog APL	poa	Dyalog Systems Ltd. products
	IBM APL products	poa	
Dyadic	Dyalog APL for DOS/386	995	Second generation APL for DOS. Runs in 32-bit mode, supports very large workspaces. Unique "window-based" APL Development Environment and Screen Manager. Requires 386/486 based PC or PS/2, at least 2Mb RAM, EGA or VGA, DOS 3.3 or later.
	Dyalog APL/W for Windows	995	As above, plus object-based GUI development tools. Requires Windows 3.0 or later.

	Dyalog APL for Unix	995-12,000	Second generation APL for Unix systems. Available for Aitos, Apollo, Bull, Dec, HP, IBM 6150, IBM RS/6000, Masscomp, Pyramid, NCR, Sun and Unisys machines, and for PCs and PC/2s running Xenix or AIX. Oracle interface available for IBM, Sun and Xenix versions.
DynArray	DICE for Windows	poa	Software development kit which includes an APL interpreter as a DLL and the ability to run and link existing and new APL code to non APL code such as VB, C/C++, Java and integration with various Windows software applications and database packages such as MS Office.
I-APL Ltd	I-APL/PC or clones	8	ISO conforming interpreter. Supplied only with manual (see 'Other Products' for accompanying books).
	I-APL/BBC Master	8	As above
	I-APL/Archimedes	8	As above
IBM APL Products	TryAPL2	free	APL2 for educational or demonstration use. Write, fax or Email to APL Products; specify disk size desired.
	APL2 PC (US Version)	\$630	Product No. 5799-PGG. PRPQ Number RJ0411. Order from 1-800-IBM-CALL
	APL2 PC (European Version)	£348	Product No. 5604-260. Part number 38F1753. From all IBM dealers, including MicroAPL.
	APL2 for OS/2 Entry Edition	\$185	Part No 89G1556.
	APL2 for OS/2 Advanced Edition	\$650	Part No 89G1897. Contains all facilities of the Entry Edition plus: DB2 interface; co-operative processing TCP/IP interface; tools for writing APs; TIME facility
	APL2 for Windows version 1.0	\$1500	Product No. 5639-d46, part number 4229558.
	APL2 Runtime environment	\$250	Part No. 39L8419 - Packager and runtime CDs. One additional runtime install \$200, 5 additional \$900, 10 \$1,500.
	APL2 for Sun Solaris	\$1500	Product No. 5648-065.
	APL2 for AIX 6000	poa	Product No. 5765-012.
	APL2 Version 2	poa	Product No. 5688-228. Full APL2 system for S/370 and S/390
	APL2 Application Envt Vn2	poa	Product No. 5688-229. Runtime environment for APL2 packages
	Cognos/APL2000 Inc	poa	Leading distributor of APL2000 products in Denmark
Insight Systems	Dyadic Systems Ltd.	poa	Leading distributor of Dyalog APL products in Denmark
	IBM	poa	Leading distributor of IBM APL & GraphX products in Denmark
	J	poa	Distributor for Austria and Switzerland
J Austria	Dyalog APL	poa	Distributor
	Causeway Products	poa	Distributor
	Structural Analysis Software	poa	Complete package by IG Zenkner&Handel to perform structural analysis/engineering calculations. Also suitable for dynamic problems, e.g. earthquake simulation.
JSoftware Inc.	J on the Web online registration ...		
	J Professional (online reg.)	\$895	includes manual set and one year of updates
	J Standard (online reg.)	Free	Free for download only
	Books and accessories		
	J Dictionary	\$50	
	J Phrases	\$50	
	J Primer	\$50	
	Fractals, Visualization and J	\$80	
	Concrete Math	\$40	
	Exploring Math	\$50	

Lescasse Consulting	APL+PC	poa	Lescasse Consulting is the exclusive APL2000 distributor in France and also distributes in Switzerland and Belgium. Call for price quotes.
	APL+Unix	poa	
	APL+DOS	poa	
	APL+Win	poa	
	Dyalog APLW	poa	French distributor for Dyalog
Master/Work Software	Manugistics Products and ISI	poa	New Zealand distributor
MicroAPL	APLX for Windows/MacOS	499	Cross-platform APL development environment with GUI programming facilities. Interpreter modelled on APL2. Available for Windows 95/98/ME/NT/2000/XP, Mac OS 9 and Mac OS X.
	APLX Server Edition	poa	For running large multi-user APL applications on x86 Linux, RS/6000 AIX, and Windows NT/2000.
Oasis	Dyalog APL	poa	Dyadic Systems
	APL*PLUS	poa	Manugistics
	APL.68000	poa	MicroAPL Ltd
	APL2	poa	IBM
Omega	Zero	poa	A "small simple and fast" alternative to APL
Optima	Dyalog APLW	995	Fully fledged Windows development environment.
RE Time Tracker Oy	APL+PC (APL*PLUS/PC)	poa	Complete APL+ and Statgraphics product range and links to various 3rd party products.
	APL+DOS (APL*PLUS II)		
	APL+Win (APL*PLUS III), APL+Link		
	APL+UNIX		
	APL*PLUS Sharefile		
Soliton Associates	SHARP APL for OS/390	poa	for IBM OS/390 mainframes
	SHARP APL for UNIX	poa	for SunOS and IBM AIX
	SHARP APL for Linux	poa	for Intel Linux
Strand Software	Canada		
	All APL*PLUS products	poa	All APL*PLUS products including upgrades and educational.
	Dyadic and JSoftware products	poa	
	USA		
	Dyadic and JSoftware products	poa	

APL PACKAGES

COMPANY	PRODUCT	PRICES(£)	DETAILS
ADAPTA Software	MPS	poa	Master Production Scheduling
	FBS	poa	Forecasting and Budgeting System
	DRP	poa	Distribution Requirements Planning
Adaptable Systems	FLAIR	poa	Finite loader and interactive rescheduler. Customisable full-function scheduling system. (Available outside Australia by special arrangement only.)
Adaytum	Adaytum e.Planning	poa	Adaytum e.Planning offers a Web-based solution that combines planning, forecasting, budgeting, modelling and reporting in a single, integrated application.

APL Software/Services	APL Utilities	poa	Software: mostly .AWS for DOS, utilities for most APL interpreters. Public domain APL*Plus v10 with on-screen documentation and interactive tutorials. APL Conference Software, Books: APL user manuals for STSC, IBM, and Sharp. Request email catalog from dick.holt@juno.com.
APL Systems (DC SL	APL+Linkpro 3.0 New System	646	Database Access SQL Client for OBDC
	Upgrade to v3.0		
	from APL+Linkpro-32	280	
	from APL-Link or APL+Linkpro	474	
	GraphX (Includes ChartX)	599	
	GraphX Lite (No distribution)	340	
	SPREAD (An APL grid control)	86	
	QPLOT (x,y plotting in APL)	47	
	QWIN	215	(Runs APL+DOS functions in APL+Win)
Beautiful Systems	ASF_FILE	\$399	Dyalog APLW auxiliary processor for access to APL*PLUS/PC APL component files (*.ASF).
	NAT_FILE	\$299	Dyalog APLW auxiliary processor which emulates the APL*PLUS/PC quad-N native file subsystem for access to the DOS file system.
	DBF_FILE	\$299	Dyalog APLW auxiliary processor for efficient block mode access to dBASE format files. Designed to get large amounts of data in and out of dBASE. Not suited for random access to small amounts of data (it does not handle keys).
	SF_READ	poa	Dyalog APLW functions to read APL*PLUS data objects of any type or structure from *.SF style component files created by APL*PLUS II or III.
Causeway	CausewayPro for DyalogW	400	Causeway application development platform for Dyalog APLW.
	RainPro Business Graphics	250	The ultimate graphics toolkit for the APL developer. Adds 3D charting capability, Web publishing and clipboard support to the shareware product. Charts can be included in <i>NewLeaf</i> reports. Functionally compatible across DyalogW and APL+Win.
	NewLeaf for Dyalog and +Win	400	Frame-based reporting tool with comprehensive table-generation and text-flow support. Offers multiple master-page capability, bitmap wrap-around and on-screen preview with pan and zoom. Fully supported on DyalogW and APL+Win
Cinerea AB	ORCHART	250	Organization chart package for IBM APL2/PC. Full & heavily commented source code included - free integration into other applications. NB: ASCII output with line-drawing (semi-graphic) characters for boxes.
CODEWORK	3-way TANGRAM	poa	3-way TANGRAM is a DSS-OLAP product, basically a powerful and versatile handler of multi-way tables (also known as hypercubes). It entails 46 analysis modules, including computations, cube merging, sensitivity analysis, time intelligence, free format queries, HTML and LaTeX outputs. Current version is 7.0 At the time of writing, the product is available on Dyalog APL 8.3.1 for Windows 95/98/NT. Future plans include an APL+WIN version and later a LINUX version.

CoSy	K.CoSy	\$30 % yr sub	K.CoSy is a general purpose computing and programming environment constructed, all in open code, in Arthur Whitney's very high level, yet structurally transparent Array Programming Language, K, and its tightly coupled GUI. K.CoSy is an extremely productive environment in one of the most powerful and fastest of APL's progeny, and therefore, likewise, of all languages. K.CoSy provides a workspace-like interactive development environment previously impossible in K. Because of its unique open construction within the language itself, this environment is clearly competitive in a large domain with the APLs from the other vendors, for just a token cost. K.CoSy notepad nature, interactivity, and open K code vocabulary make learning Arthur Whitney's K far less daunting and far more productive than its raw console, or any external scripting method. K.CoSy being young and developing open code, lends itself to offer by subscription. Charter subscriptions are just \$30 (half that for students). (Requires separate download of K from Kx.com.)
DynArray	DynaWeb Server	poa	A web server providing web based access to applications running on the DICE interpreter from DynArray, or on an IBM mainframe running APL2.
	DynaHarry	poa	A DSS system which offers the next generation capabilities for current APLDI, IC/E and IC/I users. It comes with ROLAP capabilities, multisystem access to a wide variety of databases and data warehouses.
	DynaLink	poa	An ODBC client interface for DICE and IBM APL2 programs.
HMW	4XTRA	poa	Networked, Windows/Unix based Front End and Middle Office Foreign Exchange and Money Market Dealing System. Scalable from 1 user to 120+.
	Inca	poa	Software Change Management System. Enables the user to co-ordinate development work from several sources, resolve clashes, promote work items for testing and configure releases to a live environment.
	Maya	poa	APL code file manager. A comprehensive suite of tools giving a multi-window IDE style interface to file based APL code. Offers features such as copying from file to file, object comparison, string search, style formatting, hot-spot editor for filed objects (including variables), etc.
	Aztec	poa	System shell for APL development. Manages real-time and batch applications across multiple platforms. Offers standardised error trapping, job scheduling, task communication and recovery/restart features.
I-APL Ltd	Educational workspaces	5	PC format disks with the examples from: Thomson, Espinasse (Kits 1-4), Kromberg, Jizba & FinnAPL. All the examples to save your fingers!
IBM APL Products	A Graphical Statistical System	\$250	for DOS, Product Number 5764-009
	(AGSS)	\$500	for Workstations (OS/2, Aix, Solaris), Product Number 6764-092
		\$2500	for CMS, Product Number 5764-011
Insight Systems	Causeway	poa	Leading distributor of Causeway products in Denmark
			<i>All our old products are now either OEM'd, in the public domain, out of date, or all of the above. We'll be back!</i>
JAD Software	JAD SMS	poa	JAD SMS is a multi-user software management system for Dyalog APL™ based on shared, hierarchical databases. JAD SMS databases let you keep historical versions of apl items as well as attributes such as timestamp, user name and documentation. The software includes a graphical user interface as well as specialized functions for inclusion in applications. No charge for single-user version; \$100/user for multiple users
Lescasse Consulting	APL+Win Monthly Training	\$600	Download 50+ page document about APL+ programming each month. You also get one or more workspaces full of re-usable APL code and sometimes additional files or products.
	Advanced Windows Programming	\$95	200-page book plus companion disk on interfacing APL and Delphi. Contains full coverage of Delphi-2, +Win and Dyalog.

	DLL parser for APL	\$250	Parse any Visual Basic DLL declaration file into a set of quadNA definitions. Turn constants and structures into APL variables. Available for APL+Win and Dyalog/W.
	Delphi Forms Translator	\$195	Design forms with Delphi and turn them automatically into APL programs which recreate the same form (+Win and Dyalog/W).
	APL+Link Pro	poa	ODBC interface for APL+Win
	SQAAPL Pro	poa	ODBC interface for Dyalog APL/W
	RainPro	poa	Highly customisable 2D and 3D publication graphics for APL+Win and Dyalog APL/W
	NewLeaf	poa	Page layout and printing tools for APL+Win and Dyalog
	GraphX and ChartFX	poa	High-quality business graphics for APL+Win
	Formula One and Dyalog APL	\$95	100-page book + companion disk on how to use the Formula One VBX with Dyalog APL/W
Lingo Allegro	FACS	poa	EMMA-like interface to DB2 or ODBC databases
	QWIN	poa	Legacy DOS Windowing support for APL+Win
	ODBC/127	poa	IBM AP127-like ODBC Interface for APL+Win and Dyalog APL/W
Qualedi	Qualedi	\$850-\$5,500	Electronic Data Interchange (EDI) translation software for the PC, with strict compliance checking.
RE Time Tracker Oy	UIT/W	poa	Comprehensive high-level Windows User Interface library for APL+Win and +II v 5.1. Comprehensive spreadsheets, replicated fields, special field types, etc. 16 and 32 bit versions available.
	AJGRAPH	poa	Graphpak-compatible 2D graphics package for +Win and +DOS. Includes multi-window support, print and metafile support. No DLLs required.
	ECCO PRO with APL	poa	Leading group and personal information management system with comprehensive customising. Supplied with sample +Win workspace to interface to ECCO databases via DDE.
	NEWT TCP/IP SDK with APL	poa	Lead TCP/IP SDK with interfaces to all protocols. Supplied on 3 CD ROMS together with a sample +Win workspace.
	DB+	poa	Database interface for APL+DOS under Windows. Allows combining character-based APL applications with ODBC-compliant databases such as Oracle and SQL-server.
Warwick University	BATS	250	Menu driven system for time series analysis and forecasting using Bayesian Dynamic modelling. Price is reduced to £35 for academic institutions.
	FAB	free	Training program for the above.
Weighahead Systems	Weighahead Windows Weighing System (3WS)	poa	Recipe Weighing System for Manufacturing Industries. Pharmaceutical, Cosmetics, Foods etc. Works without keyboard or mouse. Uses Electronic Balances, Laser scanners, bar codes and label printers.
Zark	APL Tutor (PC)	\$299	APL computer-based training. Available for APL+PLUS PC & APL+PLUS II. Demo disk \$10.
	APL Tutor (MF)	\$5000	Mainframe version.
	Zark ACE	\$99	APL continuing education. APL tutor news and hotline phone support.
	APL Advanced Techniques....	\$59.95	488pp. book, (ISBN 0-9619067-07) including 2-disk set of utility functions (APL+PLUS PC format).
	Communications	\$200 pc, \$500 mf	Move workspaces or files between APL environments.

APL CONSULTANCY AND DEVELOPMENT

COMPANY	PRODUCT	PRICES(£)	DETAILS
Adfee	Consultancy	poa	Development, maintenance, conversion, migration, documentation, of APL products in all APL environments
Ajay Askodum	Consultancy	poa	APL+Win development and migration of actuarial, financial, mathematical applications.
Andrews	Consultancy	poa	APL programming and analysis, Year-2000 legacy systems, algorithms, tree-processing.
APL Borealis Inc.	Support and Development	poa	APL Software Support and Development. Specialists since 1979 in Sharp APL, APL*Plus, APL+Win, Dyalog APL
APL Solutions Inc	Consultancy	poa	APL systems design, development, maintenance, documentation, testing and training. Providing APL solutions since 1969.
APL Systems IDC SL	Consultancy	poa	Consultancy and maintenance available by retainer or on call.
AUSCAN Software	Consultancy	poa	APL software development, training
Camacho	Consultancy	poa	Manuals; feasibility reports and estimates; analysis and programming; APL and MS Windows applications; Sharp, ISI APL, APL*PLUS, APL2/PC and other APLs spoken. Fixed price systems a speciality
Ray Cannon	Consultancy	poa	APL, C, Assembler, Windows, Graphics: PC and mainframe
Causeway	Consultancy and Training	poa	On-site training for Causeway, RainPro and NewLeaf. Customisation and enhancement to meet local needs. Code review and pre-implementation check of Causeway applications.
Paul Chapman	Consultancy	250-500	24-hour programmer: APL, Smalltalk, C; Windows front end design a speciality.
CODEWORK	Consultancy	poa	Development, maintenance, migration, documentation of APL applications. Speciality: info systems for top executives, internet applications.
CoSy	Consultancy	poa	CoSy.com, Coherent Systems, provides rapid development in the K language and associated data base products, with a particular interest in quantitative (financial) modeling. A great deal can be done in a day.
David Crossley	Consultancy	poa	Experienced in large APL system developments since 1969 for PC or mainframe.
Dinosoft Oy	Consultancy	poa	Specialised in very large databases.
Dittrich & Partner	Consultancy	poa	APL programming and analysis; APL workshops and training on the job
Dyadic	Consultancy	poa	APL and Unix system design, consultancy, programming and training.
DynArray	Consultancy	poa	DynArray offers consulting in the areas of DSS, Y2K and APL programs upgrade/conversion to modern Web enabled platforms.
Evestic AB	Consultancy	poa	Excellent track record from 15+ years of APL applications in banking, insurance, and education services. All dialects, platforms and project phases. SQL expertise.
First Derivative Analytics Ltd.	Consultancy	poa	Analysis, design, prototyping, development & testing of APL (especially financial) applications: Sharp, Dyalog APL/W.
General Software	Consultancy	from 200	Over 20 years experience with every version of APL, large mainframe systems and small PC based programmes.
Godin London Inc	Software Development	poa	We have applications in the food manufacturing field, travel agency and airline bookings field and in product lease management.
HMW	Consultancy	poa	System design consultancy, programming. HMW specialize in banking and prototyping work.
Hoekstra Systems	Consultancy	poa	APL consultancy, programming, etc. Also UNIX system administration

Michael Hughes	Consultancy	poa	APL consultant with 20 years experience with all versions of APL. I can create your dynamic Web sites using the full power of APL working with Microsoft IIS (Internet Information Service) on Windows NT or 2000. I also undertake System design, Programming and Maintenance on all platforms, particularly MS Windows.
INFOSTROY	Consultancy	poa, competitive	Broad experience in various APL platforms. Special skills and knowledge in developing complex applications for investment, financial and construction markets. Implementation of hybrid solutions based on APL, Delphi, C++, VBA, SQL servers.
Insight Systems	Consultancy	poa	We have experience with just about every APL system and platform in common use during the last 20 years, from SHARP APL under MVS or Linux to APL+Win and in particular Dyalog APL under Windows 9x, NT or 2000. If you have decisions to take about adapting your APL application to take advantage of emerging technologies, or would like your strategy reviewed, give us a call. We have extensive experience in all areas of APL development, from legacy systems, up, down and sideways migrations, to the development and support of shrink wrapped solutions based on APL. Even if we don't have time to do the work ourselves, we will know where to find someone who is an expert in your version of APL and your application area, on your continent.
JAD Software	Consultancy	poa	Systems design and development, project management, technical manuals, financial and actuarial expertise in APL.
KJK	Consultancy and software development	poa	APL-based data management: conversions, ad hoc-analyzing tools, well-interfaced methods for defining, processing and browsing of multi-dimensional reports. Rapid custom software development based on proven modular toolset approach.
Phil Last	Consultancy	poa	APL consultancy, modelling and programming.
Lescasse Consulting	Consultancy	poa	A range of consultants, experts in Windows programming, with APL+Win and Dyalog APL/W. More than 100 major APL applications already developed. We all have additional expertise in Formula One and Delphi.
Lingo Allegro	Consultancy	poa	General APL consulting, internet website development, migration and downsizing, performance tuning, education and training.
Lucas Solutions	Consultancy	poa	Rates depend on task and location.
George MacLeod	Consultancy	poa	Design and programming of new APL applications. Enhancing and maintaining existing APL applications. Porting existing APL applications from one APL system to another. Supporting users of APL applications. Experienced on both mainframe, UNIX and PC APL interpreters.
Mackay Kinloch Ltd	Consultancy	poa	Design, analysis and programming for banking, insurance and pensions, financial planning and modelling, corporate performance and legal reporting
MasterWork Software	Consultancy	poa	Consulting and J programming for econometrics and statistics in public policy, health and food industries.
Milinta Inc	Consultancy	poa	Design, development, maintenance, conversion, documentation in all APLs, most APs and some specific Sharp products (LOGOS, ViewPoint, Retrieve). Experience in multi-user, multi-task systems, databases, Windows programming.
Ellis Morgan	Consultancy	250-500	Business Forecasting & APL Systems.
Oasis	Consultancy	poa	Expertise in APL system design, Project management, conversion, migration, tuning; for all APL versions (10+ years experience)
Omega Computing	Consultancy	poa	APL consultancy, programming, etc.
Optima	Consultancy	poa	A range of consultants specialising in all areas of pharmaceutical, industrial and financial systems with 5-15 yrs experience on both PC and mainframe.
RadSys Technologies	Consultancy	poa	Areas of expertise: financial systems, risk analysis systems, healthcare systems.

Resources & Results	Consultancy	poa	Knowledge management company builds decision support, data warehouse, data mining and strategic planning systems for CFO's, CEO's, and senior management, using our Rapid Application Development (RAD) methods and tools. Extensive experience in large-scale system development and ad hoc executive support for Fortune 500 clients.
RE Time Tracker Oy	Consultancy	poa	APL application conversions, APL Windows interfaces, APL to APL-level interfacing to any system under Windows, TCP/IP network and database connectivity.
Rex Swain	Consultancy	poa	Independent consultant, 20 years experience. Custom software development & training, PC and/or mainframe.
Rochester Group	Consultancy	poa	Specialise in MIS using Sharp APL
Shepp & Associates	Consultancy	poa	APL applications development and consulting, especially in the travel industry, especially on small computers. 25 years experience in APL programming.
Snake Island Research Inc	Consultancy	poa	APL interpreter and compiler enhancements, intrinsic functions, performance consulting. APL parallel compiler APEX is giving very good initial performance tests with convolution somewhat faster than FORTRAN.
SovAPL	Consultancy	poa	Offshore APL development service.
Strand Software	Consultancy	poa	Advice on migrating to and from all flavours of APL and hardware platforms. Full-screen interface implementation, APL utilities, benchmarking, efficiency analysis, actuarial software, system development tools, valuation, pricing and modelling systems.
Sykes Systems Inc	Consultancy	poa	Complete APL services specialising in audit, optimisation and conversion of APL systems. Excellent design skills. All dialects and platforms. 17-23 years experience.
Weighahead Systems	Consultancy	poa	Specialising in industrial systems. Links to PLCs, laser scanners, bar codes, weigh scales, label printers etc. Also programmable hand held scanners.
Stephen Wynn	Consultancy	poa	Most experience of financial planning, and mathematical areas: operational research, quality control, experimental design.

OTHER PRODUCTS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Adfee	Employment	poa	Contractors and permanent employees
APL-385	Typefaces	poa	Variants of the APL2741 typeface available to specification.
APL Borealis Inc.	APL Training	poa	Hands-on courses in Introductory, Intermediate, Advanced and Windows APL. Courses are customized and flexible, and may be delivered on-site, with strong emphasis on methods for efficient and maintainable APL systems development.
ComLog	Comic-Logger	\$25.95+p&p	APL*PLUS II comic-book inventory system. Shareware version available on America OnLine.
HMW	Employment	poa	Contractors and permanent employees placed.
I-APL Ltd	Books	poa	I-APL stocks books written to go with the I-APL interpreter and some APL Press books. For a list write to 11 Auburn Road, Bristol BS6 6LS, ring 0117 973 0036 or email 100612.1057@compuserve.com.
Oasis	Training	poa	Introductory courses in APL Advanced courses for different APL versions
Renaissance Data Systems	Booksellers		The widest range of APL books available anywhere. See Vector advertisements.

OVERSEAS ASSOCIATIONS

GROUP	LOCATION	JOURNAL	OTHER SERVICES	Ann.Sub.
ACM SigAPL	International	APL QuoteQuad	Conferences; APL white pages; web site	\$30
APL Bay Area	USA N. California	APLBUG	Monthly Meetings (2nd Monday)	\$20
APL Club Austria	Austria	-	Quarterly Meetings	200AS(indiv), 1000AS(corp)
APL Germany e.V.	Germany	APL Journal	Semi-annual meetings	DM60
Ass. Francophone pour la promotion d'APL	France	Les Nouvelles d'APL	FF350 (private) FF2800 (Company)	
BACUS	Belgium	APL-CAM	Conferences & Seminars	£18 (\$30)
Capital PCUG	Washington, D.C.	Monitor	Monthly meetings, occasional classes	free
Danish SIG	Denmark			
Dutch APL Assoc.	Holland	Vector provided	Mini-congress, APL ShareWare Initiative	
FinnAPL	Helsinki, Finland	FinnAPL Newsletter	Seminars on APL	100FIM(private), 30(student), 1000 (Co)
Japan APL Assoc	Tokyo	APL Journal	Monthly meetings (4th Sat)	10,000yen to join
NY SigAPL	New York, USA	Big Apple APL	Monthly meetings	\$35/\$25(ACM)
Rome/Italy SIG	Roma, Italy			
SE APL Users Grp	Atlanta, Georgia	SEAPL Newsletter	Quarterly meetings	\$10
SovAPL	Moscow, Russia	-	Seminars and Annual Meeting	
SwedAPL	Sweden	SwedAPL Nytt	Semi-annual meetings, seminars	SEK 75
SWAPL	Texas, USA	SWAPL		\$18
Swiss APL (SAUG)	Bern	Part of Qtiy SI-Info	SF60 (SI) + SF20 (SAUG)	
Toronto SIG	Toronto, Canada	Gimme Arrays!	Monthly Meetings, APL skills database, J SIG, Toronto Toolkit	\$25

ADDRESSES

ORGANISATION	CONTACT	ADDRESS, TELEPHONE, FAX, EMAIL etc.
ACM SigAPL	David Siegel	ACM, 1515 Broadway, 17th Floor, New York, NY 10036, USA (Subs only)
ADAPTA Software GmbH	Michael Baas	Stellinger Weg 19, 20255 Hamburg, Germany. Tel: +49 40 40170951 Fax: +49 40 40170952. Email: info@adapta.de
Adaptable Systems	Lois & Richard Hill	49 First Street, Black Rock 3193, Australia. Tel: +61 3 9589 5578 Fax: +61 3 9589 3220 Email: hillrj@melbpc.org.au
Adaytum Limited	Douglas Rowley	Castlegate, Tower Hill, BRISTOL BS12 0JA. Tel: 0117 921 5555 Fax: 0117 922 7749. Email:sales@adaytum.co.uk
Adfee	Bernard Smoor	Dorpsstraat 50, 4128 BZ Lexmond, Netherlands. Tel: +31 347 342 337 Fax: +31 347 342 342 Email: adfee@concepts.nl
Andrews	Dr Anne D Wilson	12 Thorny Hills, Kendal, Cumbria LA9 7AL, UK. Tel: 01539-731205 Email: ADWilson@kencomp.net
APL-385	Adrian Smith	Brook House, Gilling East, York YO62 4JJ, UK. Tel: 01439-788385 Email: apl385@compuserve.com
APL2000 (Europe)	Fred Honea	see APL Systems IDC SL.
APL Bay Area APLBUG	Curtis Jones (Sec)	228 South 15th Street, San Jose, CA 95112-2150, USA Tel: +1 (408) 292-4060 Email: jonesca@vnetlibm.com
APL Borealis Inc.	Richard Procter	381 Manor Road East, Toronto, Ontario M4S 1S7, Canada. Tel: (416) 457-7828. Fax: (416) 482-6582 Email: info@aplborealis.com
APL Club Austria	Harald F. Nelson	c/o N-TECH, Siebenbrunnfeldg. 4-6, A-1050 Wien, Austria. Tel: +43 1 5458063 Fax: +43 1 5458063-17
APL Germany e.V.	Dieter Lattermann	Rheinstraße 23, D-69190 Walldorf, Germany. Tel: +49 6227-63469 Compuserve: 100332,1461
APL Software/Services	Dick Holt	3802 N Richmond St, Suite 271, Arlington, VA 22207 USA Tel: +1 (703) 528-7624; Fax: +1 (703) 528-7617; Email: dick.holt@juno.org

APL Solutions Inc	Eric Landau	1107 Dale Drive, Silver Spring, MD 20910-1607 USA Tel: +1 (301) 589-4621 Fax: +1 (301) 589-4618 Email: elandau@cais.com
APL Systems IDC SL	Fred Honea	Alfredo Marquerie, 12 - 2 F, 28034 Madrid, Spain. Tel: +34 91 730 7008 (Office) +34 60 680 5949 (Mobile). Fax: +1 775 743 6131. Email: uksales@apl2000.net
Ajay Askoolum	Ajay Askoolum	42 Hanworth Road, Redhill, Surrey RH1 5HT Tel: 01737-771643 Email: ajay@askoolum.freeserve.co.uk
Association Francophone pour la promotion d'APL	Ludmila Lemagnen	174 Boulevard de Charonne, F-75020 Paris, FRANCE Email: lemagnen@aol.com
AUSCAN Software Ltd	Richard Procter	PO Box 39, Mansfield, Ontario L0N 1M0 Canada Tel: +1-705-434-1239 Email: rjp@interfog.com
BACUS	Joseph De Kerf	Rooiberg 72, B-2570 Duffel, Belgium. Tel: +32 15 31 47 24
Beautiful Systems, Inc.	Jim Goff	308 Old York Road, Suite 5, Jenkintown, PA 19046, USA Tel: +1 (215) 886-2636; Fax: +1 (215) 886-4888 Email: BeautifulSystems@golfs.net
Camacho	Anthony Camacho	11 Auburn Road, Redland, Bristol BS6 6LS, UK. Tel: 0117-973 0036. email: acam@tesco.net
Ray Cannon		21 Woodbridge Rd, Blackwater, Camberley, Surrey GU17 0BS, UK. Tel: 01252-874697 Email: ray_cannon@compuserve.com
Causeway Graphical Systems Ltd	Adrian Smith	The Maltings, Castlegate, MALTON, North Yorks YO17 7DP, UK. Tel: 01653-696760 Fax: 01653-697719 Email: adrian@causeway.co.uk
Paul Chapman		51B Lamba Conduit Street, London WC1N 3NB, UK. Tel: 020 7404 5401, Compuserve: 100343,3210
Cinerea AB	Rolf Kornemark	Box 61, S-193 00 Sigtuna, Sweden. Tel/Fax: +46 859 255 421 Email: rolf@cinerea.se
Ian Clark	Ian Clark	1, Heifer Mill Cottages, Mosterton, Beaminstor, Dorset DT8 3HG, England. Tel: +44 (0)7931 370304. Email: earthspot@aol.com
CODEWORK	Mauro Guazzo	Corso Cairoli 32, 10123 Torino, Italy. Tel: +39 11 885168 Fax: +39 11 812 2652 Email: codework@codework.it.com
ComLog Software	Jeff Pedneau	18728 Bloomfield Road, Olney, MD 20832 USA Tel: +1 (301) 260-1435 Email: jeff@softmed.com
CPCUG	Lynne Sturtz	Capital PC User Group, 51 Monroe Street, Suite PE-2, Rockville, Maryland 20850-2421, USA. Tel: +1 (301) 762-9372 Fax: (301) 762-9375.
CoSy.com	Bob Armstrong	42 Peck Slip #4B, New York, NY 10038-1725, USA. Tel: +1 212-285-1864 Fax: +1 212-285-1864. E-mail: bob@CoSy.com.
David Crossley	David Crossley	187 Le Tour du Pont, 84210 ST DIDIER, France. Tel: +33.4.90.66.08.87 Email: crossley@au-village.com
Danish User Group	Helene Boesen	c/o Insight Systems ApS, Nordre Strandvej 119G, Hellebæk, Denmark
Dinosoft Oy	Pertti Kalliojärvi	Lönnrotinkatu 21C, 00120 Helsinki, FINLAND. Tel: +358 9 70028820 Fax: +358 9 70028824 Email: dinosoft@dinosoft.fi
Dittrich & Partner Consulting GmbH	Axel Holzmüller	Kießer Strasse 17, D-42697 Solingen, Germany. Tel: +49 212-260 660 Fax: +49 212-260 6666; Email: info@dpc.de
Dutch APL Association	Bernard Smoor (Sec)	Postbus 1341, 3430BH Nieuwegein, Netherlands. Tel: +31 347 342 337 Fax: +31 347 342 342
Dyadic Systems Ltd.	Peter Donnelly	Riverside View, Basing Road, Old Basing, Basingstoke, Hants RG24 0AL, UK. Tel: 01256-811125 Fax: 01256-811130
DynArray Corporation	Dr James Brown	16360 Monterey Rd, Suite 260, Morgan Hill, CA 95037, USA Tel: +1 (408)-782-6648 Fax: +1 (408)-782-6627 Email: info@DynArray.com
Evestic AB	Ole Evero	Bertellusvagen 12A, S-146 38 Tullinge, Sweden Tel & Fax: +46 778 4410 Email: ole.evero@mailbox.swipnet.se
FinnAPL	Olli Paavola	Suomen APL-Yhdistys RY, FinnAPL RF, PL 1005, 00101 Helsinki 10, Finland Email: olli.paavola@pyr.fi
First Derivative Analytics Ltd.	Ken Chakahwata	114 Lemsford Lane, Welwyn Garden City, Herts AL8 6YP, UK Tel/Fax: 01707-339620. Email: KenChakahwata@compuserve.com
General Software Ltd	M.E. Martin	Little Wester House, Westerhill Road, LINTON, Kent ME17 4BS Tel: 01622 749328 Fax: 01622 749365 E-mail: martin@gsoft.freeserve.co.uk

Godin London Incorporated	Gaštan Godin	12 Gerrard St., London, Ontario, Canada N6C 4C5 Tel: +1 (519) 679-8290 Fax: +1 (519) 438-6381 Email: info@godin.on.ca
HMW Computing	Chris Hogan	Hamilton House, 1 Temple Avenue, London EC4Y 0HA, UK. Tel: 0870-1010-468; Email: HMW@4xtra.com
Hoekstra Systems Ltd	Bob Hoekstra	Dominique, Salsbury Road, Woking, Surrey, GU22 7UR, UK. Tel: 01483-771028. Fax: 01483-837324 Email: Bob.Hoekstra@HoekstraSystems.Ltd.uk
Michael Hughes		28 Rushton Road, Wilbarston, Market Harborough, Leics. LE16 8QL, UK. Tel: 01536-770998 Email: Michael@Hughes.uk.com
I-APL Ltd	Anthony Camacho	11 Auburn Road, Redland, Bristol BS6 6LS, UK. Tel: 0117-973 0036. Email: 100612.1057@compuserve.com
IBM APL Products	Nancy Wheeler	APL Products, IBM Santa Teresa, Dept REN/F40, 555 Bailey Avenue, San Jose CA 95141, USA. Tel: +1 (408) 463-APL2 [+1 (408) 463-2752] Fax: +1 (408) 463-4488 Email: APL2@vnet.ibm.com
INFOSTROY	Alexey Miroshnikov	3 S. Tulen Lane, St. Petersburg 191186 Russia. Tel: +7 812 325-9797 Fax: +7 812 311-2184 Email: aim@infostroy.ru
Insight Systems ApS	Helene Boesen	Nordre Strandvej 119G, DK-3150 Hellebæk, Denmark Tel: +45 70 26 13 26 Fax: +45 70 26 13 25 Email: info@insight.dk
JAD Software	David Crossley	175 East 96th St., Apt. 17G, New York, NY 10128 Country: USA Tel: +1 (212) 369-6713 Fax: +1 (212) 761-0124 Email: jadsms@usa.net
Japan APL Assoc	Toshio Nishikawa	1-8-13 Masujima Buld.6/F Higashi Gotanda Shinagawa-ku, Tokyo Japan 141-0022. Tel: +81 (03) 3280-0411 Fax: +81 (03) 3280-0418 Email: KYY00361@niftyserve.or.jp
J Austria	Joachim Hoffmann	Hochsteltengasse 13/26, 8010 Graz, Austria. Tel: 0043 (0)316 91 42 51 Mobile: 0043 (0)699 1 91 42 51 2. Email: joachim.hoffmann@gmx.at
JSoftware Inc.	Eric Iverson	33 Major Street, Toronto, Ontario, Canada M5S 2K9. Tel: +1 (952) 470-7345 Fax: +1 (952) 470-8202 Email: info@jsoftware.com
KJK-Iieto Oy	Kimmo Kekäläinen	Merikasarminkatu 10 B 56,00160 Helsinki, Finland. Tel: +358 50 55 27 207; Email: Kimmo.Kekalainen@pp.htv.fi
Phil Last Ltd	Phil Last	146 Crossbrook Street, Cheshunt, Herts, EN8 8JY, UK. Tel: 01992-633807 Fax: 0121-359 0375 Email: phil.last@net.ntl.com
Lescasse Consulting	Eric Lescasse	18 rue de la Belle Feuille, 92100 Boulogne, France. Tel: +33.1.45.05.10.76 Fax: +33.1.46.04.60.23 Email: eric@lescasse.com
Lingo Allegro USA, Inc	Walter G. Fil	203 N. LaSalle Street, Suite 2100, Chicago, Illinois 60601, USA. Tel: +1 (800) 546 4621 E-mail: lingo-allegro@visto.com
Lucas Solutions	Jim Lucas	Stubbendamvej 9C, 3.tv., 3000 Helsingør, Denmark Tel: +45 49 26 52 42. Email: jel@danbbs.dk
Mackay Kinloch Ltd	Alastair Kinloch	519 Webster's Land, Edinburgh EH1 2RX, Scotland, UK. Tel: +44 (0)131 228 5235 Email: akinloch@globalnet.co.uk
George MacLeod	George MacLeod	37 Newhouse Rd, Bovingdon, Herts, HP3 0EJ, UK. Tel: 01442-831385 Fax: 01442-831388 Email: george.macleod@ntlworld.com
MasterWork Software Ltd	Fraser Jackson	PO Box 56-036, Tawa, Wellington, New Zealand. Tel: +64 (4) 232-4440 Fax: +64 (4) 232-4452 Email: fraser.jackson@xtra.co.nz
Mercia Software Ltd.	Gareth Brentnall	Mercia House, Ashted Lock, Aston Science Park, Birmingham, B7 4AZ, UK. Tel: 0121-359 5096. Fax: 0121-359 0375
MicroAPL Ltd.	Richard Nabavi	The Roller Mill, Mill Lane, Uckfield, E.Sussex TN22 5AA Tel: 01825 768050. Fax: 01825 749472 Email: MicroAPL@microapl.demon.co.uk
Milinta Inc.	Dan Baronet	Contact Dan Baronet at danb@milinta.com
Ellis Morgan	Ellis Morgan	Myrtle Farm, Winchester Road, Stroud, Petersfield, Hants GU32 3PE, UK. Tel: 01730-263843 Email: Ellis@mtlfirm.demon.co.uk
NY Sig	David Siegel	PO Box 2697, New York, NY 10163-2697, USA. Email: NYSIGAPL@ACM.ORG
Oasis b.v.	Louis Rijkse	Lekstraat 4, 3433 ZB Nieuwegein, Holland. Tel: +31 30 60 66 336 Fax: +31 30 60 65 844 Email: rijkse@oasis.nl
Omega Computing inc	Alan Graham, Andrew Chou	3 Columbus Avenue, Edison, NJ 08817, USA. Tel: +1 (732) 985 9519 Email: alangraham@mindspring.com
Optima Systems Ltd	Paul Grosvenor	1st Floor, 10 Raleigh Court, Priestley Way, Crawley, West Sussex, RH10 2PD, UK. Tel: 01293 562700 Fax: 01293 562699 Email: mailbox@optima-systems.co.uk

Qualedi Inc.	Ann Wilkie (Marketing) Ed Shaw (Technical)	121 West Main Street, Milford, CT 06460, USA. Tel: +1 203-874-4334. Email: awilkie@qualedi.com; eshaw@qualedi.com
RadSys Technologies AB	Randolph Schrab	Lovsångarv. 18, S-756 52 Uppsala, Sweden. Tel: +46 18 32 41 53 Fax: +46 708 1996 11 Email: randolph.schrab@radsys.se
Renaissance Data Systems	Ed Shaw	P.O. Box 313, Newtown, CT 06470, USA. Tel: +1 (203) 270-9729 Email: rendata@aplbooks.cncnhost.com or orders@aplbooks.cncnhost.com
Resources & Results	Frank Rhodes	2438 W. Northgate Dr., Irving, TX 75062, USA. Tel: +1 972 523 5117 Email: frank@decision-support.net
RE Time Tracker Oy	Richard Eller	Mikonkatu 8 A, 2.krs, PL 363, 00101 Helsinki, Finland. Tel: +358 9-621 3300 Fax: +358 9-621 3378 Email: re@rett.fi
The Rochester Group Inc.	Robert Miller	600 Park Avenue, Rochester, NY 14607-2926, USA. Tel: +1 (716) 271-1110. Fax: +1 (716) 271-1230
Rome/Italy SIG	Mario Sacco	Casella Postale 14343, 00149-Roma Trullo, Italy Email: mario.sacco@tin.it
SE APL Users Group	John Manges	413 Comanche Trail, Lawrenceville, GA 30044, USA Tel: +1 (770) 972-3755 Email: seapldoc@aol.com
Shepp & Associates LLC	Andrew Shepp	1312 Washington Avenue, 6th Floor St. Louis MO 63103, USA Tel: +1 (314) 621-3272 Fax: +1 (314) 621-4267 Email: ashepp@compuserve.com
Snake Island Research Inc	Bob Bernecky	18 Fifth Street, Ward's Island, Toronto, Ontario M5J 2B9 Canada Tel: +1 (416) 203-0854 Fax: +1 (416) 203-6999 Email: bernecky@interlog.com
SOCAL (South California)	Roy Sykes Jr	Sykes Systems Inc, 4649 Willens Ave, Woodland Hills, CA 91364-3812 USA. Tel: +1 (818) 222-2759 Fax: +1 (818) 222-9250
Soliton Associates	Benoit Paquin	Soliton Denmark, Havsgårdsvej 4, 2900 Hellerup, Denmark Email: sales@soliton.com
SovAPL Russian Chapter of SIGAPL	Alexander Skomorokhov	PO Box 5061, Obninsk-5, Kaluga Region 249020, Russia Tel: +7(08439)47109 Fax: +1 (530) 6885510 Email: askom@obninsk.com
Strand Software Inc	Anne Faust	19235 Covington Court, Shorewood MN 55331 USA Tel: +1 (952) 470-7345 Fax: +1 (952) 470-9202 Email: info@strandsoft.com
Rex Swain	Rex Swain	8 South Street, Washington, CT 06793 USA. Tel: +1 (860) 868-0131 Fax: +1 (860) 868-9970 Email: rex@rexswain.com
SwedAPL	Christer Ullhielm	Novator Consulting Group AB, Svårdvägen 11C, S-182 33 Danderyd Sweden. Tel: +46 8 622 63 50 Fax: +46 8 622 63 51 CService: 100341,404
Swiss APL User Group		Swiss APL User Group, CH-3001, Bern 1, Switzerland Email: si@ifi.unizh.ch
Sykes Systems Inc	Roy Sykes Jr	4649 Willens Ave., Woodland Hills, CA 91364, USA Tel: +1 (818) 222-2759 Fax: +1 (818) 222-9250
Toronto SIG	Richard Procter	PO Box 55, Adelaide St. Post Office, Toronto Ontario M5C 2H8, Canada Email: info@torontoapl.org
Weighahead Systems	Phillip Bulmer	Camberley House, 1 Portesbury Road, Camberley, GU15 3RB, UK. Tel: +44 1276 20789 Email: sales@weighahead.com
Stephen Wynn		8 Clarence Gardens, Brighton, Sussex BN1 2EG, UK. Tel: 01273-327238 Email: centre@cwcom.net
Zark Incorporated	Gary A. Bergquist	23 Ketchbrook Lane, Ellington CT 06029, USA. Tel: +1 (860) 872-7806

FTP SITES

IBM APL2	ftp.software.ibm.com/ps/products/apl2
Waterloo Archive	archive.uwaterloo.ca/ftparch/languages/apl
APL-to-ASCII	archive.uwaterloo.ca/languages/apl/workspaces/aplascii

WORLD WIDE WEB SITES

ACM SigAPL	www.acm.org/sigapl/
Adapta Software	www.adapta.de/
Adaptable Systems	www.assuredsystems.com.au/
Adaytum Limited	www.adaytum.com/

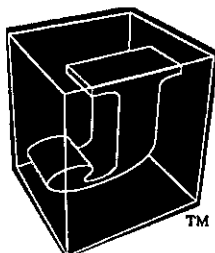
AFAPL	www.afapl.asso.fr/ (Journal available on line)
APL2000	www.APL2000.com/
APL-385	www.demon.co.uk/apl385/
APL Journal, Germany	www.rhombos.de/apljourn.htm
APL Systems IDC SL	www.apl2000.net
AUSCAN	www.interlog.com/~rjp/auscan/
Eke van Batenburg	www.bio.LeidenUniv.nl/~Batenburg/index.html
Capital PC User Group	http://cpcug.org/
Causeway	www.causeway.co.uk/
CODEWORK	www.codework-it.com/tangram/eng/
CoSy (Bob Armstrong)	CoSy.com/
Dinosoft Oy	www.dinosoft.fi/
Dittrich & Partner	www.dpc.de ; www.apl-online.de
DMOZ - Open Directory	http://dmaz.org/Computers/Programming/Languages/APL/
Dyadic Systems Ltd	www.dyadic.com/
DynArray	www.dynarray.com/
FinnAPL	www.pyr.fi/apl/
Godin London Inc	www.godin.com/
Houben (IQL)	www.apl.olap.club.tip.nl
Hoekstra Systems	www.HoekstraSystems.ltd.uk/
IBM APL2	www.ibm.com/software/ad/apl
Infostroy	www.infostroy.ru
Insight Systems ApS	www.insight.dk/
Japan APL Association	www.naska.co.jp/JAPLA/
JSoftware Inc	www.jsoftware.com/
KJK-tieto Oy	www.kjk-tieto.com
Lescasse Consulting	www.lescasse.com/
Lingo Allegro USA Inc	www.lingo.com/
Mackay Kinloch	www.mackaykinlochltd.co.uk/~admkl/
MicroAPL Ltd	www.microapl.co.uk/apl
Milinta Inc	www.milinta.com
Oasis b.v.	www.oasis.nl/
Optima Systems Ltd	www.optima-systems.co.uk
Qualedi, Inc.	www.qualifiedi.com
Renaissance Data	www.aplbooks.cnchost.com/
Resources & Results	www.decision-support.net
RE Time Tracker Oy	www.rett.fi/
The Rochester Group Inc.	www.rochgrp.com/
Shepp & Associates	www.digitravel.com/
SigAPL	www.acm.org/sigapl/
Soliton	www.soliton.com/
Snake Island Research Inc.	www.snakeisland.com
Strand Software Inc.	www.strandsoft.com/
Rex Swain	www.rexswain.com/
Toronto SIG (for Toolkit)	www.torontoapl.org/
Jim Weigang	www.chilton.com/~jimw/
Weighahead Systems	www.weighahead.com

Jsoftware Inc.
19235 Covington Ct.
Shorewood, MN 55331
info@jsoftware.com
Tel (952) 470-7345
Fax (952) 470-9202

Announcing Jsoftware Version 5.01™

**Watch our web site for the most
advanced Jsoftware release
yet—version 5.01!**

- ◆ **Session Manager, window driver, IDE, and GUI support in Unix and Linux**
- ◆ **GUI event support now extended to closely match VB and Java**
- ◆ **Improved online documentation**
- ◆ **Performance Improvements**
- ◆ **Online documentation available in hardcopy books**
- ◆ **New debug facilities for Pro users**



www.jsoftware.com

Review of "J: the Natural Language for Analytic Computing"

book by Norman Thomson, review by Cliff Reiter

Norman Thomson's book, *J: the Natural Language for Analytic Computing*, which we will abbreviate *JtNL*, is the first J book to be published independently from the language developers. The beautiful title makes Thomson's zeal for J clear. The publisher, Research Studies Press Ltd., Baldock, Hertfordshire, England (RSP) targets specialist areas. The editor states in his foreword that "Norman Thomson has prepared a text for the J community, which I am confident will be appreciated by the J community." I think the editor's words are true.

In the simplest terms, *JtNL* is a J language book. Its organization is heavily influenced by parts of speech: verbs, nouns, adverbs and conjunctions. *JtNL* touches on most aspects of the J language, introducing primitive features and common phrases. It could also be viewed as a less formal, alternate dictionary [2] organized around parts of speech. However, the book is deeper than either of those views: it flows forward in an organized manner and includes a number of topics that illustrate solving larger problems.

The reviewer is the author of [5], which is a book designed to teach mathematical visualization using J. Thus, readers should understand that I have my own ingrained preferences on how to introduce J. I would not have chosen to organize an introduction to J like *JtNL*. Still, in reading the book I found myself learning a lot and admiring the ways in which Thomson made his organization work.

The cover of *JtNL* has a sharp magenta abstract design with a collage of J expressions and J results scattered in the background. It has a solid feel, a professional appearance, and it contains approximately 300 pages. *JtNL* is divided into four parts: 1. Basic J, 2. Advanced J, 3. Data Processing with J, and 4. Mathematics with J. These parts are divided into a total of 29 chapters with each chapter containing several sections. In general, I found the book more interesting as it progressed, but even the beginning was thought provoking.

Space doesn't allow the mention of each section and only the barest mention of most chapters; however, each section illustrates the rich ideas of J and hence there is a lot packed into the book. Nonetheless, I will try to give a sense of the flow of the book along with a few examples.

Basic J

The first section of the first chapter discusses language in general, both natural and formal. It goes on to describe word formation in J along with the primary parts of speech: nouns, verbs, adverbs and conjunctions. The case is made that verbs are the powerhouse of the language and the rich rules for composition of verbs forms the heart of J. Thus, Chapter 1 is: *In the beginning...verbs*. The first example discusses the common mathematical notion of "plus or minus". Thomson notes this is almost always really means plus joined to minus in the sense that the list of both the sum and the difference is desired. After noting the meaning of comma to join together data pieces, he notes that (+, -) expresses the notion of plus joined to minus in J. He gives examples of the use of that fork both without and with naming it. Choosing to introduce a fork as the first example of J seems brave, but Thomson's example clearly illustrates the power of J to clarify ordinary thinking. The classic fork for computing averages, (+/ % #), is often used to motivate tacit constructions and it appears as his next example. The first chapter goes on to briefly introduce the *hook*, *at*, *atop*, *cap* and long *trains*. The heart of J appears in the first chapter.

Chapter 2 is *...and then nouns*. Character data, lists, and boxed arrays in general get discussed. Frames and cells and rank are described and the order of execution of verb-noun phrases is made clear. Empty arrays are explained. The second chapter, like the others, finishes with a very useful chapter summary and exercises. The summary in Chapter 2 lists (among several other things) the primitive verbs introduced or extended in the chapter:

i .	integers
\$	shape of (monadic), shape (dyadic)
{	from (dyadic)
#	tally (monadic), copy (dyadic)
<	box
>	open
;	raze (monadic), link (dyadic)

We see that even though this chapter focused on nouns, experience with the language's primitive verbs is evolving. Indeed, while the general organization is at first around parts of speech, each section is a coherent exposition and together the sections move toward a high quality understanding of J.

Chapter 3 is *Extensions to verbs and adverbs*. It discusses the eight adverbs and what they do. On a first reading, it was probably the chapter that was most jarring

organizationally. Nonetheless, the illustrations and examples are quite clear. And if I ever want to read up an adverb, I know where I can look. Conjunctions are not so easy. Chapter 4, *Basic conjunctions and composition*, is a deeper treatment than appeared in the first chapter. Chapter 5, *More on conjunctions and composition*, is in the same spirit. It includes a nice section: *Two traps for the unwary!* These traps are forgetting to use @ or @: when applying compositions and the second is forgetting parentheses when constructing complex tacit verbs. This isn't a magic wand which allows readers to avoid all errors in defining tacit functions, but it is good, timely advice to a reader studying the book.

Chapter 6, *Dictionary and foreign conjunction*, contains terse listings and is thereby similar to the on-line vocabulary; however, it is organized by parts of speech. There was a lengthy recent Jforum discussion of "what IS *cap*?"; in light of the discussion, it is not surprising to see *cap* listed in the section: *Other symbols*. Some basic useful foreign conjunctions are illustrated and scripts and locales are mentioned, but the treatment is very brief and only generalities are discussed.

Advanced J

The second part of the book, *Advanced J*, begins with Chapter 7, *Control Words*. But the chapter might be better titled "explicit definition". Explicitly defined functions, adverbs and conjunctions appear. I did not see any mention of 4 : type definitions of dyads. Also, all of the explicitly defined adverbs and conjunctions use x. and y. for the adverb/conjunction arguments. The use of u. v. m. n. for adverb and conjunction verb and noun arguments appears to be missing and the richness of using conjunctions with both the conjunction arguments and the derived arguments available is thereby shortchanged. Also, automatic conversion to tacit via 13 : and the like is not mentioned. Indeed, it seems like the automatic conversion of adverbs via 12 : would be a good reason to mention x. as the adverb argument. The chapter ends with illustrations of using control words if. while. and try. and a listing of the other control words is given.

The next chapter is on gerunds; agenda, recursion and gerund inserts are clearly illustrated. Gerunds, and user defined adverbs and conjunctions are often used throughout the remainder of the book. This is consistent with my earlier observation that the material is introduced in an evolving fashion. Formatting data, executing strings and files and input/output in general are discussed in Chapter 9. Full chapters (10-12) on each of the conjunctions: dot, power, and rank appear. These include many nice, thoughtful illustrations. For example, dot where the left argument is not an insert is discussed (this is one of the first reasons that I ever looked at J [4]). The examples with power include gerunds and many

phrases that experienced users would expect to see appear in these chapters. The discussion of rank includes the illustration of using two applications of rank, one where the left and right ranks are different. On page 119 we see the description and example: "This is a convenient way of, for example, joining each item in one list with all items in another:"

```
lets=.2 2$'abcd'
nos=.3 3$'123456789'
lets,"1"1 _ nos
ab123
ab456
ab789
cd123
cd456
cd789
```

This construction is of a type that users are likely to face from time-to-time, but finding a working phrase might be a struggle for a new user. It isn't that *J1NL* explains all the important phrases, its strength is that it exercises them at a level that a learner needs. These chapters of *J1NL* on individual conjunctions are relatively easy reading for these important, multi-layered facilities.

Data Processing in J

This third section recognizes the expressiveness of J in effecting data processing. Chapter 13, *Selection*, explicates using *from*, *{*, and complex *copy*, *#*, to make selections. The use with rank and boxed arguments is suitably included. Chapter 14, *Data Structuring*, is devoted to functions such as: rotate, reverse, transpose. Chapter 15, *Ranking and Ordering*, illustrates grade up/down, sorting, and ranking via grade atop grade (an powerful phrase). Chapter 16, *Searching, finding and updating*, contains a fine discussion of partially modifying data under boxing. That discussion ends with an example of using gerund arguments to amend.

```
USA=. 'Florida'; 'California'; 'Alaska'
MASCA=. ['(<:@#)]' }
'o' MASCA &> USA
Florida
Califonio
Alasko
```

A typographic error appears in the result *J1NL* shows for the last phrase. The result of 'o' MASCA &. > USA is shown in *J1NL* where the phrase was 'o' MASCA &> USA as shown above. This is typical of the typographic errors in

JtNL. They appear, readers will notice them, but they seem not to be very disruptive.

Chapter 17, *Joining Data* appears; here several useful facilities are explained; multiple assignment (p153) is done in an old style which is not supported in current versions of J. Chapter 18, *Partitioning*, is primarily on cut. I love cut and am glad to see lots of simple, clear illustrations of its power. Not every detail needed to define cut is given, but this discussion covers substantial ground and this chapter is easier reading than the dictionary. *Raze-in* is also illustrated in this chapter. Here are some of the pretty examples of computing structured binary patterns that appear there.

```
e. i. 4
1 0 0 0
0 1 0 0
0 0 1 0
0 0 0 1
```

```
(e. 0 1 1 0);(e. 0 1 2 1 0);(e. 0 0 1 1 1)
+-----+
|1 0 0 1|1 0 0 0 1|1 1 0 0 0|
|0 1 1 0|0 1 0 1 0|1 1 0 0 0|
|0 1 1 0|0 0 1 0 0|0 0 1 1 1|
|1 0 0 1|0 1 0 1 0|0 0 1 1 1|
|      |1 0 0 0 1|0 0 1 1 1|
+-----+
```

Chapter 19, *Opposites*, discusses topics such as obverse, inverse, adverse and identity verbs. Chapter 20, *Some programming principles* appears. It discusses various approaches to selected problems. Mostly this amounts to putting together several tacit definitions to accomplish some task. In the *J Introduction and Dictionary* [1, p4] or the electronic *J Dictionary* [2, Intro., pt. 3], *by* and *over* are used as "black boxes" as illustrated by the following from *JtNL*.

```
over=.{.;}.)@":@,
by=. ' '&;@,.@[,.]
'abc' by 1 2 3 4 over i.3 4
+-----+
| |1 2 3 4|
+-----+
|a|0 1 2 3|
|b|4 5 6 7|
|c|8 9 10 11|
+-----+
```

A section in Chapter 20 of *JtNL* describes how *by* and *over* work, which removes the mystery of the black box. A worthy exercise for a growing learner.

Maths in J

The last part of the book discusses mathematics with J. New primitives appear and modest applications are developed in the nine chapters that appear here. Thus the introduction to J continues. However, these chapters are much more topical. Chapter 21, *Complex numbers* introduces primitives and basic applications, such as computing roots of unity and transformations of the plane. Chapter 22, *Numbers bases and Polynomials*, describes polynomial evaluation, changing bases and applications such as clock time. Chapter 23, *Series*, includes geometric series and mortgage payments. No discussion of the Taylor series primitives is given. Chapter 24, *Calculus and the D adverb*, introduces some of the variants of D. along with a discussion of partial derivatives. A section on Newton-Raphson's method for approximating roots appears there. How easy it is:

```
Newton=. 1 : ']-x.%x.D.1' (^:_)("0)
f2=.2&o.-(*^)
f2 Newton 1
0.517757
```

Notice Newton- Raphson is a single, easy to explain line; but it uses a mixture of an explicit adverb embedded in a tacit definition. It has a function argument — limits are computed — the rank is good. Powerful stuff from one line.

Chapter 25, *Numerical analysis*, consists of best fitting data, root approximation and numerical integration. Here the function for Simpson integration uses a specific global function, *fn*, as integrand. It seems like a missed opportunity to use an adverb; strangely, a defined conjunction is used for an intermediate step in the construction, but the globally defined integrand remains *fn*.

JtNL primarily uses local assignments, even for the many multi-line definitions appearing in Chapter 25. Of course global and local assignments are equivalent for direct interactive definitions and one could argue the one line functions should just be typed directly, but local assignments are not what one generally wants in scripts, so I am left wondering if *JtNL* is really supposing the multi-line definitions should be keyboarded with each use and not stored in a script. My guess is this chapter is based upon very early J writings and was not brought up to date. It is too bad, since each of these examples could have been as sweet as Newton-Raphson's method was in the previous chapter.

Chapter 26, *Randomness*, describes roll and deal and some useful utilities for random sampling. Chapter 27, *Permutations and Combinations* introduces nub, nubsieve, atomic permute, and cyclic primitive. Work with permutation groups appears. Chapter 28, *Groups and Symmetries*, takes a more abstract look at groups, primarily cyclic and dihedral groups. Chapter 29, *Logic*, discusses logical operations and testing tautologies.

The book has two appendices; the first gives solutions to the exercises and the second is a summary of the J phrases used throughout the book. This overlaps the chapter summaries, but puts the key functions together in one place for browsing.

Conclusion

Thomson's *JtNL* is not a replacement for the Dictionary — it isn't complete. It could even be argued that all the types of examples done in *JtNL* appear somewhere in the *J Dictionary* [2] or *Primer* [3]. *JtNL* has faults: there are typographic errors and some sections are outdated. Still, *JtNL* has many easy to read examples and most topics appear. It is in a handy to read hard-bound book. It has helpful summaries and reminders. More importantly, for someone trying to learn J, the level of sophistication rises as *JtNL* progresses. The most common phrases that J users need are illustrated. Readers who need to understand something completely before moving onward might be frustrated by the informality of *JtNL*, but readers, like me, who read things repeatedly, gaining further understanding at subsequent passes, probably would do well to study *JtNL*. I think it is a valuable resource for J users who want an introduction to, or refresher on, the J language.

References

- [1] Kenneth E. Iverson, *J Introduction and Dictionary* (print version), Jsoftware Inc., Toronto, 1995.
- [2] Roger K. W. Hui and Kenneth E. Iverson, *J Dictionary* (electronic version), Jsoftware Inc., Toronto, 2001.
- [3] Eric Iverson, *The J Primer*, (electronic version), Jsoftware Inc, Toronto, 2001.
- [4] Clifford A. Reiter, *Fractals RYIJ*, *Vector*, 11.2 (1994) 86-104.
- [5] Clifford A. Reiter, *Fractals, Visualization and J*, 2nd Edition, Jsoftware, Inc., Toronto, 2000.

ZARK Newsletter Extracts

introduced by Jon Sandles

Utility Corner: Killing Comments

(The purpose of this column is to make you more productive by introducing you to utility functions. Think of utility functions as APL functions that have names instead of symbols. By expanding your function vocabulary, you'll be able to write APL code that's more concise, more efficient, and more readable.)

In last issue's LIMBERING UP column, we asked you to design and write a function named *UNCOMMENT* that eliminates the comments and all-blank lines from a specified function. Normally, we encourage you to include more comments, not to strip them out. Why the change of heart?

Strangely, having an *UNCOMMENT* utility in your bag of tricks may encourage you to include *more* comments. Some APL programmers rationalise their lack of comments by claiming comments consume valuable workspace. Less workspace increases the chances of *WS FULL* errors and may even cause the functions to run more slowly. By omitting comments, they say, the functions run "better". Fine, we say, omit the comments from the version of the functions that you *run*. But don't omit them from the version you *maintain*.

Construct a "development workspace" that contains highly commented functions. Then pass each of the functions through the *UNCOMMENT* utility function to strip out the comments. Save the uncommented functions in a "production workspace." You'll have two workspaces: the development workspace is big and readable and easy to maintain. The production workspace is small and streamlined, but unreadable.

The general approach of the *UNCOMMENT* utility is to convert the specified function to a character matrix by providing its name as the right argument of the system function $\square CR$ (Canonical Representation).

Here's an illustration:

```

      ▽ R←AVG V
[1]   ⍝ Returns the average of the numbers
[2]   ⍝ in vector V.
[3]   R←(+/V)÷ρV ⍝ Doesn't work on scalars
      ▽
      □CR 'AVG'
R←AVG V
⍝ Returns the average of the numbers
⍝ in vector V.
R←(+/V)÷ρV ⍝ Doesn't work on scalars
      ρ□CR 'AVG'
4 36

```

Next, use the Boolean techniques discussed in the Primitive Pointers sections of this News to identify and blank out comments. Finally, supply the resulting character matrix as the right argument of the system function `□FX` (Fix). `□FX` defines (fixes?) the new function in the workspace, using the rows of the character matrix as the lines of the function. The explicit result of `□FX` is the character vector name of the function defined. Like so:

```

      CMAT←2 6ρ'R+DZ NR+N~0 '
      CMAT
R+DZ N
R+N~0
      ρCMAT
2 6
      □FX CMAT
      ▽ DZ[ ] ▽
      ▽ R+DZ N
[1]   R+N~0
      ▽

```

What should be the syntax of the `UNCOMMENT` function? We left the design up to you. Of the solutions sent us, two designs were proposed.

In the first, `UNCOMMENT` is a monadic function whose argument is the name of the function to be "uncommented." The `UNCOMMENT` function uses `□CR` to convert the name to a character matrix representation of the function, applies the Boolean techniques to replace the comments by blanks, applies `□FX` on the resulting matrix, and returns as its explicit result the result of `□FX`, that is, the name of the redefined function.

A disadvantage of this design surfaces when you try to uncomment a function whose name happens to match the name of one of the local variables in *UNCOMMENT*. For example, suppose the header of *UNCOMMENT* looks like this:

```
▽ R←UNCOMMENT NAME;A;B;C
```

If you try to uncomment a function named *R*, *NAME*, *A*, *B*, or *C*, the function won't work. In particular, when *UNCOMMENT* does $\square CR$ *NAME* the function having that name is "shadowed" by the local variable of the same name. So, $\square CR$ tries to return the canonical representation of a variable rather than a function, and gets all fouled up. (It returns an empty character matrix.)

In the second design, this problem is eliminated, at the expense of convenience. *UNCOMMENT* is a monadic function whose argument is the canonical representation of a function to be uncommented, rather than its name. The explicit result of *UNCOMMENT* is the amended character matrix. *UNCOMMENT* does not perform $\square CR$ or $\square FX$. Those operations are left to you. For example,

Design 1:

```
UNCOMMENT 'AVG'
AVG
```

Design 2:

```
 $\square FX$  UNCOMMENT  $\square CR$  'AVG'
AVG
```

Gary Logan submitted a solution that incorporates both designs. If the right argument is a character vector (or scalar), *UNCOMMENT* takes it as a function name, and applies $\square CR$ to it. If the right argument is a character matrix, *UNCOMMENT* takes it as the function's canonical representation, and applies the Boolean techniques directly. To show us the effectiveness of his *UNCOMMENT* function, Gary printed the result of *UNCOMMENT* $\square CR$ '*UNCOMMENT*' and sent it to us, with the explanation, "It is clear that *UNCOMMENT* works very well on itself." True, though we couldn't help but wonder whether this *UNCOMMENT* function ever contained any comments.

The following *UNCOMMENT* function is a composite of the functions we received.

It contains plenty of comments (why not?).

```

▽ R←UNCOMMENT A;N
[1]  A Remove the comments from the func-
[2]  tion name A (if A is a scalar or
[3]  a vector) or from the □CR canonical
[4]  a representation A (if A a matrix).
[5]  A The result is the name of the newly
[6]  a defined function (via □FX) if A is
[7]  a a name, or is the stripped charac-
[8]  a ter matrix, which can be passed to
[9]  a □FX, if A is a matrix. The number
[10] a of local names is kept to a minimum
[11] a to reduce the possibility of a
[12] a named function conflicting with a
[13] a local name.
[14]  A
[15]  A Convert name to □CR if needed:
[16]  A(N+1zppA)/'A+□CR A'
[17]  A Flag leading (^\) noncomments or
[18]  a text in quotes:
[19]  R←,^\'A≠'A')v×\A=''''
[20]  A Squeeze/expand out comments:
[21]  R←(pA)pR\R/,A
[22]  A Eliminate all blank rows:
[23]  R←(Rv,≠' ')≠R
[24]  A Convert the □CR back to a function
[25]  a if a name given
[26]  A N/'R+□FX R'
[27]  →N/0
[28]  A Otherwise, eliminate trailing all-
[29]  a blank columns before returning the
[30]  a matrix:
[31]  R←(φv\φ' 'v,≠R)/R
▽

```

The last expression in *UNCOMMENT* is executed only if the argument and result of *UNCOMMENT* are character matrices. Because the logic in *UNCOMMENT* does not squeeze out comments, but rather replaces them by blanks, it's possible for the resulting character matrix to contain a number of trailing all-blank columns, where comments previously existed. As a courtesy to the user of *UNCOMMENT*, these trailing blank columns are eliminated.

Finally, let's try *UNCOMMENT* on itself:

```

UNCOMMENT ⍵CR'UNCOMMENT'
R←UNCOMMENT A;N
⍺(N←1≥ρA)/'A←⍵CR A'
R←,^(\A≠' ')/'A=' '
R←(ρA)ρR\R/,A
R←(R∨.≠' ')/R
⍺N/'R←⍵FX R'
→N/0
R←(ϕ∨\ϕ' '∨.≠R)/R

```

What's the size difference between the commented and uncommented versions of *UNCOMMENT*? On APL*PLUS PC, the commented version occupies 1776 bytes of actual workspace; the uncommented version takes 384 bytes, or 22% of the original.

What about readability? The commented version is five times the size of the uncommented version. Is it five times more readable? Ten times? Fifty times?

Comments are priceless. Don't omit them.

Limbering Up: Generalised Searching

(The purpose of this column is to work some flab off your APL midsection. Like muscles, your APL skills can atrophy if not exercised with adequate frequency and variety. This column presents a task for you to perform. Set aside a few minutes from your busy schedule and work the task. Mail in your solution and stay tuned for the results.)

In a previous issue of the Zark APL Tutor News, we developed a number of functions for searching through the rows of a character matrix for the first occurrence of each row of another matrix. Different algorithms excelled in different circumstances. However, one algorithm stood above the rest when used on large arguments.

In case you missed that issue, here's a listing of the function:

```

    □vr'CMiota'
  ▽ R←A CMiota B;G;J;K;N;S;T;Z
[1]  A Returns row indices of character
[2]  A matrix A where each row of B is
[3]  A found. Like dyadic 1, CMiota re-
[4]  A turns one greater than the largest
[5]  A index in A for rows of B not found.
[6]  A
[7]  A Index returned if not found:
[8]  Z←□IO+1+ρA
[9]  N←1+ρB A No. rows in B
[10] A Stick 1+□AV as new row of A:
[11] A+A,[□IO]□AV[255+□IO]
[12] G←□AV⊥B,[□IO]A A Grade vec
[13] A Which ones come from B (∧/K∈N):
[14] K←(S+G<N+□IO)/G
[15] A The relative ind of the next A row:
[16] J←□IO+S/∖T←-S
[17] R←J+(T/G)[J]-N A The absolute A inds
[18] R[K]+J A ...in the orig order of B
[19] A Use the "not-found" value for those
[20] A not matching:
[21] R[(∨/B≠A[R;])/N]+Z
  ▽

```

The function is named *CMiota* (Character Matrix IOTA) because of its similarity in syntax with the primitive dyadic APL function 1 (iota). For example:

```

    38 42 75 84 1 42 15 38
  2 5 1
    (4 3ρ'REDBOYCANSIT') CMiota (3 3ρ'BOYWASRED')
  2 5 1

```

Each function searches through the items (elements or rows) of its left argument for the first occurrence of each item (element or row) of its right argument. The result has one index per item of the right argument, telling where in the left argument the item was first located. If the item of the right argument is not found in the left argument, the corresponding number in the result is the index just beyond the last item of the left argument. In origin 1, that "not-found" index is one more than the number of items in the left argument. In origin 0, it's the number of items.

In this issue, we're going to suggest another way to view APL's searching capability. Instead of saying, "1 returns the index in the left argument where the items of the right argument are first found," let's say, "1 returns the index in the

left argument where the items of the right argument are first *equal*." Stated this way, searching sounds more like an *operator* than a *function*. After all, there's a function (=) involved.

Just as the reduction (/) operator requires a function operand (such as +/ or ×/ or [/), perhaps the searching operator should also require an operand (such as =₁ or >₁ or <₁). Unfortunately, this notation conflicts with the existing definition of monadic ₁ (as in, 3 4 5=₁3). This conflict doesn't bother us a whole lot, because we're not really suggesting a new operator. We're suggesting a new way to look at searching. For now, to avoid the ambiguity, we'll slap the operator and operand in parentheses, like so:

```
      38 42 75 84 (=1) 42 15 38
2 5 1
```

This is the "where is it first equal" function. Let's try a different relational function:

```
      10 20 35 50 (≤1) 12 57 35
2 5 3
```

This is the "where is it first less than or equal" function. The result contains 1 (in origin 1) for each number of the right argument less than or equal 10, 2 for each number greater than 10 but less than or equal 20, 3 for each number greater than 20 but less than or equal 35, and so on. In more concise notation:

```
1 if X≤10
2 if 10<X≤20
3 if 20<X≤35
4 if 35<X≤50
5 if 50<X
```

Notice the index 5 is returned for any corresponding number in the right argument that is not less than or equal any of the numbers of the left argument. Like dyadic ₁, the result contains indices of the left argument, or one greater than the largest index if the corresponding element of the right argument does not belong to any of the defined sets.

Let's try (<₁):

```
      10 20 35 50 (<1) 12 57 35
2 5 4
```

This is the "where is it first less than" function. It returns indices according to the following scheme:

```
1 if X<10
2 if 10≤X<20
3 if 20≤X<35
4 if 35≤X<50
5 if 50≤X
```

This scheme is similar to that of ($\leq$). The difference lies in whether the upper of lower limits of the ranges are inclusive. In the case of ($\leq$), the upper limits are inclusive; for ($<$), the lower limits are inclusive.

These functions, ($\leq$) and ($<$), if available, would be quite useful in a number of applications. We'll discuss some of these in the next issue of the Tutor News.

In the meanwhile, your task is to implement one or both of the functions *LEIOTA* and *LTIOA* to emulate ($\leq$) and ($<$) respectively:

```
10 20 35 50 LEIOTA 12 57 35
2 5 3
10 20 35 50 LTIOA 12 57 35
2 5 4
```

Try to make the functions as efficient as possible, especially when the right argument is long. The function does not need to work properly for non-vector right arguments or for left arguments that contain duplicates or are not in ascending order. However, if you do handle these cases, without forsaking efficiency, you'll win a special prize (our admiration, and that of our readers). Naturally, comments are a welcome surprise.

Send your solutions to:

Vector Production
Brook House
Gilling East
York YO62 4JJ
UK email apl385@compuserve.com

The notable functions and their authors' names will be printed in the next issue of *Vector*. Good luck and happy limbering.

Letters

From: Claude Henriod

My idea to resolve the Monthly Discount Rate proposed in Vector Vol.18.1 page 37-38. 74 years old, but not so atrophied, it seems ?

Each *IDEA*n variable is to be executed (1)

Mortgage is an IBM-APL2 operator

Syntax is : *LA (LO MORTGAGE) RA*

Syntax is : *INT (PRIN MORTGAGE) TERM*

If $\rho INT = \rho TERM$, it is 2 possibilities :

(.08 .09)(90000 MORTGAGE) 20 25 give a 2x2 Matrix

(.08 .09)(90000 MORTGAGE) 20 25 give a Vector of 2 terms

```

      VMORTGAGE[[]]V
[0] PMT←INT(PRIN MORTGAGE)TERM;V;W
[1]  A POLYADIC FUNCTION. VECTOR VOL.18.1
[2]  A PMT←PRIN÷(V-V*1+12*TERM)÷1-V÷1+INT÷12 A PROPOSAL
[3]    V÷1+INT÷12
[4]    W+V÷1+12*TERM
[5]    V÷1+INT÷12
[6]    PMT←PRIN÷(V-W)÷1-V
[7]    A W+(V÷1+INT÷12)÷1+12*TERM A TWO-
[8]    A PMT←PRIN÷(V-W)÷1-V÷1+INT÷12 A STMT
[9]    A CHI, SWISS. AN IBM-APL2 OPERATOR.
      V 2001-10-09 21.03.27

```

```

      IDEA1
(9÷100)(90000 MORTGAGE)30
      1IDEA1
724.1603553
      IDEA2
(9÷100)(90000 MORTGAGE)15 20 25 30
      1IDEA2
912.8399258 809.7533603 755.2767273 724.1603553
      IDEA3
(8 9 10÷100)(90000 MORTGAGE)15 20 25 30
      1IDEA3
860.0868759 752.7960621 694.6345974 660.3881165
912.8399258 809.7533603 755.2767273 724.1603553
967.1446059 868.5194806 817.830671 789.8144131

```

```

      IDEA4
(8 9 10÷100)(90000 MORTGAGE)20 25 30
  ⚡IDEA4
752.7960621 694.6345974 660.3881165
809.7533603 755.2767273 724.1603553
868.5194806 817.830671 789.8144131
      IDEA5
(8 9 10÷100)(90000 MORTGAGE)20 25 30
  ⚡IDEA5
752.7960621 755.2767273 789.8144131

      2 3p⊞TS
2001 10 9
      21 38 9

```

CHI - Claude Henriod Informatique
 1965, Savièse, Switzerland
 chenriod@vtx.ch

From: Gianluigi Quario

I'm not sure having understood your trial. Anyway I prepared 2 functions.

1) *MORTGAGE* is a dyadic function; the operator *ofAmount* allows a kind of third argument

```

0.9 MORTGAGE 30
0.9 (MORTGAGE ofAmount 900000 1000000) 30

```

2) *myMORTGAGE* is a dyadic function with 3 arguments (the right one is a nested array). May I consider it like a polyadic function? Please note:

```

3 2  ++ p90000 100000 200000      myMORTGAGE (0.9 0.85 0.8) (20 30)
4 2 3  ++ p90000 100000 200000 500000 myMORTGAGE (0.9 0.85) (20 25 30)
4 2 2 3 ++ p90000 100000 200000 500000 myMORTGAGE (2 2p0.9 0.85) (20 25 30)

▽ Z+LO(FN ofAmount arg)RO;xFN;PRIN
[1]  a companion operator of dyadic function MORTGAGE
[2]  PRIN+arg
[3]  Z+LO FN RO
[4]  a Gianluigi Quario 2001 10 18
▽

```

```

V PMT←INT MORTGAGE TERM;V
[1]  A mortgage for 1 unit of amount borrowed
[2]  A if global variable PRIN found, PRIN is amount borrowed
[3]  A INT ←nom. annual interest rate  TERM←number of years of loan
[4]  A PMT ←monthly payment
[5]  A PRIN←principal(amount borrowed) can be GLOBAL
[6]
[7]  A try: 0.9 MORTGAGE 30
[8]  A      0.9 (MORTGAGE ofAmount 900000 1000000) 30
[9]  A Polyadic Functions
[10]
[11]  :If 2*□NC'PRIN' ◊ □SHADOW'PRIN' ◊ PRIN+1 ◊ :End
[12]
[13]  V←1+INT÷12                                AMonthly discount rate
[14]  PMT←PRIN÷(V-V÷1+12×TERM)÷1-V
[15]  A Gianluigi Quario 2001 10 18
V

V PMT←X myMORTGAGE Y;V;INT;TERM;PRIN;Xminus;Xdivide;Xpower;accordingly
[1]  A triadic function: one left arg (PRIN) + two right args (INT TERM)
[2]  A PRIN ←principal  INT←nom. annual interest rate  TERM←years
[3]  A the args can have different ranks and dimensions
...
[10]  PRIN←X
[11]  INT TERM←2+Y                                A function is triadic
[12]
[13]  accordingly←{
[14]  A operator for dyadic scalar functions :
[15]  A try to preserve "scalar behaviour" by "stretching"
[16]  A the minor rank operand;
[17]  A if not possible, outer product of operands is performed)
[18]  A if args have same rank and dimensions, normal scalar behaviour
[19]  A elseif args have a common dim, arg with minor rank is stretched
[20]  A else,                                outer product.
[21]
[22]  ((,1)=p,α)∨((,1)=p,w)∨(pα)=pw:α αα w      Anormal scalar behaviour
[23]  -∨/(α+(pα)⊆pw),b+(pw)⊆pα:α◊,αα w      Aouter product
[24]  AxesWithCommonDimension←((-□IO)+((∨/a)/a),(∨/b)/b)1)+1(pα)lpb
[25]  α αα[AxesWithCommonDimension]w      Aminor rank arg is stretched
[26]  }
[27]  Xminus←-accordingly
[28]  Xdivide←+accordingly
[29]  Xpower←*accordingly
[30]
[31]  V←1+INT÷12                                AMonthly discount rate
[32]  PMT←PRIN Xdivide(V Xminus V Xpower 1+12×TERM)Xdivide 1-V
[33]  A Gianluigi Quario 2001 10 18
V

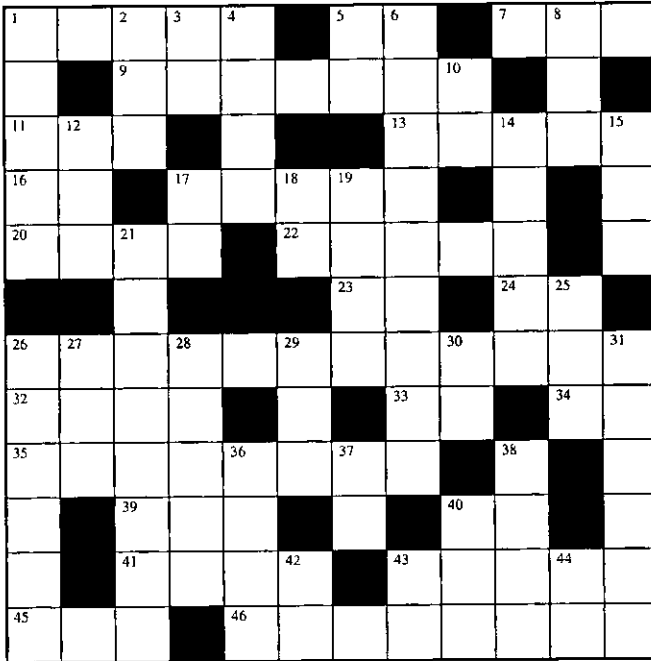
```

Solution and New Crossword

Solution to Crossword in 18.2

+	/	B	o	.	<	L		3	┐	┐	B
\		\		0	\		2	o	o	R	
T	A	B	*	5		N			D		
-	+	/	R		V	ι	ι	T		(	
G		V		3	!	!	N		+	V	1
*	T		2	ρ	ρ	A		5	×	ι	4
2		2	×	×	P		3	?	N	V	4
	A	~	~	B		5	†	5		)	
0	≡	≡	T		M	,	2		0	=	F
ρ	ρ	A		K	+	4	↓	Z		ι	I
⊂	T		B	/	ι	ρ	B			ρ	V
M				1	0	0	0	0	*	V	E

New Crossword



Across

1. $+^{\circ}MAT$
5. $V[(\rho V) + (\Box IO = 1) - \Box IO = 0) - \iota \rho V]$
7. $, 1 + \iota 10 \otimes N$ for integer scalar $N > 0$
9. Are any of the corresponding rows in matrices M and W identical?
11. $(M \times M \geq N) + (N \times M < N)$
13. Pythagoras: $(\text{_____} + B \star 2) = C \star 2$
16. $\supset, / A$ where A is a vector of simple vectors
17. $-V - 100$
20. $0, 5 \times VR$
22. $(\iota 0) \rho \Psi V$
23. $\iota 0$, using shape
24. Add up
26. $A \in V$ for vector V, using dyadic ι
32. $(\rho N) + (\rho K)$ for vectors N and K

33. $N-1 \mid N$
34. $\phi \rho \& M$
35. N random numbers from the set $(ML-1), (ML-2), (ML-3) \dots, (ML-59)$
39. The value of X, if $TP \leftarrow \odot X$
40. $\phi^{-2} + 1 \phi 'NUMBER'$
41. $V[1 \ 5]$
43. $R[I] \times \times / N[I;]$ for all I
45. 10, using reshape
46. *MATRIX*, with values formatted to one decimal place

Down

1. Are any of the values of matrix M in vector V?
2. Is either M or N true?
3. Replicate each element A times
4. Are both T and B0 true?
5. Reverse the values in each row of M
6. $(V \geq 0) \wedge (V \leq 9)$
8. '1' or '2' at random (origin 1)
10. $\square_$ tells you how much room there is
12. $-1-AR$
14. $(2 \times V[1]) * (2 \times V[2])$ for 2-element vector V in origin 1
15. What gave birth to this erroneous situation?
17. $\times / (\phi 12) / 1 + 12$ in origin 1
18. $2 \rho 2 \phi \& ' ' ' V \leftarrow 0 ' ' '$
19. $+ \backslash (\rho V) \rho^{-1}$ for vector V in origin 1
21. $I, -I \times *^{-1} 0$
25. Bit vector to index vector converter
26. $M \neq M$, but faster
27.) *VARS* or) *FNS* under program control
28. OK random numbers from 1 T5
29. ____ your boat
30. $\Rightarrow N$ in Sharp APL
31. Stick vector NX on the right side of matrix AMT as a new column
36. $1, (NP-1) \rho 0$
37. $+ / + \backslash (+ / + \backslash (+ / + \backslash 12) \rho 1) \rho 1$ in origin 1
38. *RRR*[____] is in ascending order
40. $(N \rho T[1]), (N \rho T[2]), (N \rho T[3]), \dots$ for scalar N
42. Format to V decimal places
43. $(A > 0) - (A < 0)$
44. $1 \rho I$ for scalar I

J-ottings 31: Symphony in J minor, op.31

by Norman Thomson

Composer's programme note: In response to a question on the forum "is J beautiful?", Roger Hui answered as follows :

minors = . 1&(|:\.)"2^:2

If you know mathematics, you already knew what minors are. If you know J but didn't know mathematics, you will also now know what minors are, even if you didn't before! For the benefit of listeners who fall between these two categories, any matrix with m rows and n columns can be transformed into a set of mn matrices each of size $(m-1)$ by $(n-1)$ by omitting the row and column of every element in the original matrix in turn. That much is a statement in English of the phrase above. The phrase itself forms the symphony's motif, whose meaning unfolds movement by movement as the work progresses.

First movement : *Introduction and andante syntactico.*

The end of the motif, has a repeat sign ^:2 indicating that the theme is to be played twice. The theme itself, to be found within the parentheses, is |:\. , that is a melody |: played with orchestration \. . Next the opening bar of the motif shows that there are two staves, in which & braces the melody to a pedal bass 1. The coda "2" adds dynamics to the main theme, indicating the level at which it is to be played. The structure of the motif should now be apparent, and the remaining movements bring into increasingly sharp focus how the four repeated musical elements |: \. & " combine to make a single coherent form. As these component ideas emerge and crystallise, the work itself surges forward with a compelling, indeed inevitable momentum.

Second movement : *Pastorale*

The movement opens with the folk melody |: which with its easy turn of phrase is often to be heard being hummed and whistled in its native matrix environment. \. , called *suffix*, is perhaps not quite so well known. As the name suggests, it is associated with Suffolk, the most easterly of England's counties, which slopes gently downwards from west to east until it disappears below the surface of the ever encroaching and unforgiving sea which is portrayed by the dot.

The following illustration shows how it works, as first one part and then another withdraws from the melodic strand, in the style of the Farewell Symphony:

m	NB. three strands each played to completion
life	NB. before the other begins
ebbs	
away	

,m	
lifeebbsaway	NB. .. then played altogether as one

,\,m	
lifeebbsaway	NB. .. now first three,
ebbsaway	NB. .. then two,
away	NB. .. then one

Gentle *suffix* is quite unlike the rocky coastal cliff scenery of Cornwall on the other side of the country where her cousin *oblique* evokes the music of the waves as they surge and then fall away in nature's vertical polyphony:

,/. m	NB. flow at first ..
l	NB. the three melodies enter
ie	NB. one after the other
fba	NB. in downward progression
ebw	
sa	NB. .. then ebb
y	

The music of the integers, if not the spheres, can be heard in the same fashion:

+/. 1+i.4 4	
1 0 0 0	NB. strata of sequences increasing by 3
2 5 0 0	NB. padded with fill items
3 6 9 0	NB. rising in length ..
4 7 10 13	
8 11 14 0	NB. .. and then falling
12 15 0 0	
16 0 0 0	

Third movement: Presto semantico

Now the number 1 is heard knocking at the door like Fate in Beethoven's fifth. No longer is \. a gentle ebb and flow, now decline (*suffix*) has become a relentless forward march to the rhythm of *out,fix, out,fix,out,fix,out..*, as first one and then another of the underpinning strands is left *out*:

1 ,\ . m	
ebbaway	NB. strands are now played in all
lifeaway	NB. possible combinations, leaving out
lifeebbs	NB. 1 at a time

Leave them out two at time, and there are just single strands left, not three as might be expected, but only two, since those left out must be in succession:

2 ,\ . m	
away	NB. leave out strands 1 and 2
life	NB. leave out strands 2 and 3

(Note: the bass line may be played with either positive or negative *vibrato*, that is it makes no difference if 1 is replaced by _1 or 2 by _2.)

A percussion line can be added to each strand to make , into a two-stave melody:

```
( , &'!') \ . m
life
ebbs
away
!!!!
ebbs
away
!!!!

away
!!!!
```

This illustration can be played more elegantly on the box piano <"2 :

```
<"2 ( , &'!') \ . m
+-----+-----+-----+
| life | ebbs | away |
| ebbs | away |!!!!|
| away |!!!!|      |
|!!!!|      |      |
+-----+-----+-----+
```

Adding a second stave to *oblique* transforms polyphony into modulation through a variety of *keys*, so in the next example theme A is in one key and themes B and C in another:

```

      1 2 2 ,/.m
life
ebbsaway

```

Fourth movement: *Finale grandioso*

In the final movement, the influence of Eastern music becomes apparent, since the performer is free to choose an initial foundational thread to start off each individual performance. Following this the various musical elements of the preceding movements are brought together to form a grand climax of resounding symbols. Rank 2 is the natural medium for the foundation thread, which might, for example, be

```

      i.3 2
0 1
2 3
4 5

```

but it could equally well be a matrix of any size, rank or type (character or numeric). For a rank 2 foundation thread the explicit dynamic <"2 is not strictly necessary for the first statement of the melody in which the full force of the orchestra is unleashed, slicing and transposing the rows in a crescendo which finishes at rank 3 level:

```

      (1&(|:\.)) i.3 2
2 4
3 5

0 4
1 5

0 2
1 3

```

In the repeat section, each of the inner level (<"2) matrices is sliced and transposed, the latter ensuring that the original tonality of rows and columns is reinstated:

```
t=. (1&(|:\.)) i.3 2
<"2 (1&(|:\.))"2 t
+--+
|3|2|
|5|4|
+--+
|1|0|
|5|4|
+--+
|1|0|
|3|2|
+--+
```

In the most popular performances, the foundation thread is rank 2, numeric and square :

```
<"2 minors i.3 3
+-----+
|4 5|3 5|3 4|
|7 8|6 8|6 7|
+-----+
|1 2|0 2|0 1|
|7 8|6 8|6 7|
+-----+
|1 2|0 2|0 1|
|4 5|3 5|3 4|
+-----+
```

This has led to repeated demands for the work to be performed globally wherever a popular audience for linear algebra is to be found. However, it has also been played with acclaim to audiences of a more literary inclination:

```
<"2 minors 2 4$'lasttime'
+-----+
|ime|tme|tie|tim|
+-----+
|ast|lst|lat|las|
+-----+
```

New arrangements for different combinations of instruments and platforms are constantly in demand, and it is suggested that listeners regularly scan the local Toronto papers for forthcoming concerts in this and similar series.

Encore ?? ...

APL2000 User Conference at Naples

reported by Ray Cannon (ray_cannon@compuserve.com)

photos by Ray and Jonathan Manktelow

News Flash

APL2000 announce *two* new APL interpreters at their User Conference. Look out for APL2000.JS and Comic in future issues of Vector!

When, Where, and Who

This year's APL2000 User Conference was held at the Naples Beach Hotel & Golf Club, in Naples Florida between the 4th and 7th of November 2001.



The conference proper took place between the two training sessions (which were held on Sunday the 4th and Wednesday the 7th).

Four concurrent training sessions were available. Ray Polivka ran the two-day "Introduction to APL" training as usual. Fred Waid and Jairo Lopez ran one day on "APL Web Services" and the second on "APL, Microsoft .NET and the future". The third session was a collection of different units: Jeremy Main on "ADO.NET ADO, DAO, ODBC", Patrick Parks on "Cheap Grid Tricks", Eric Baelen on "Secrets of the APL+Excel Interface" and Gary Bergquist on "Intermediate APL

Programming". The fourth session was not actually training, but reserved for re-runs (or pre-runs) of popular conference proper sessions (the equivalent of a conference VCR, allowing one to attend two sessions with overlapping schedules).

On the main days of the conference, Monday and Tuesday, each morning started with breakfast and followed with a single session (invited speakers etc.). It was then split into 3 concurrent sessions after the morning break. An excellent lunch was supplied each day, as was ice cream each afternoon. (Food and drink every two hours.) Afternoon closing sessions ended in time to watch the sunset from the poolside bar overlooking the Gulf of Mexico.



Monday had nine "papers" and Tuesday 12, some of which are /will be appearing in Vector.

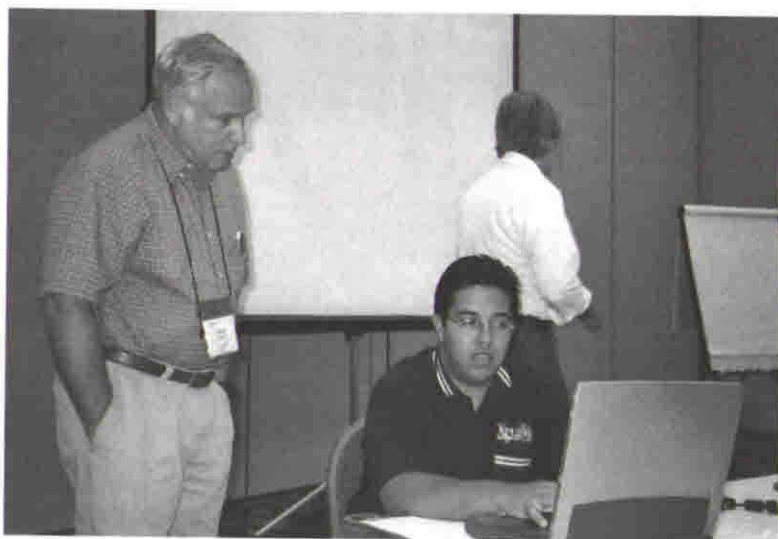
Training Days – APL2000.JS

At last year's APL2000 conference, I really enjoyed Fred and Jairo's JS training. (Question: What the difference between JavaScript and JS? Answer: A lawyer.) So this year I signed up for their sessions again. I was very surprised by how much progress they had made, especially with their new "baby", an APL interpreter written in JS called "APL2000.JS".

Hopefully Fred will get time to produce a proper write-up for Vector on this new interpreter, but in the meantime I will try to convey to you what the significance of it is and how it all fits together.

The Goal – Run APL code on your PC (the client) supplied via a web page (the server), with nothing on your machine but your web browser.

OK, using Microsoft's fully JS-enabled Internet Explorer Browser, you visit a web page. The browser downloads 3 ASCII files being an HTML page, and two JS files (the APL interpreter `APL2000.JS`, and the workspace `<wsid>.JS`). The browser displays the HTML, which invokes the JS APL interpreter, which loads and runs the APL workspace. Neat.



What is amazing is that `APL2000.JS` is only 30Kb of JavaScript, that is 30Kb of ASCII text, and the workspace is also simple text. (How big? How long is a piece of string?) For comparison Adrian's `APL2741` font is larger at 32Kb and that is after being zipped!

How is this achieved? Fred and Jairo have written a new user command `JS SAVE` which the APL programmer uses to convert functions from the workspace into the "APL JS" ASCII workspace.

What about screen interactions, forms, buttons, fields, `SI` calls etc.? JS supports them all via the browser. The `JS SAVE` command converts the APL2000 style parameters into JS so the JS engine in the browser handles them all.

This is all currently working but not yet in Beta testing, as some features still need to be finalised.

Current "issues", "problems" and "features"

This is all still under development, Fred and Jairo only got it working a week or so before the conference. By the time you receive the Vector containing this report, many changes will have occurred. However at the time of the conference Fred raise the following points:

- **No support for multi-dimensional vectors.** Vectors and Vectors of Vectors (nested arrays) ARE supported. JS currently just does not support multi-dimensional vectors of any data type, so neither can APL2000.JS.
- **RunAtServer.** Some operations, rather than running on the client, will need to be passed back over the network to the server, to be run there. This is supported, the server can have the standard APL2000 interpreter running, processing the RunAtServer requests.

However the RunAtServer operation is in realy an OPERATOR not a FUNCTION (strict APL meanings here) as it takes both functions and data as its arguments. The current APL2000 interpreter does not support User Defined Operators, which is what is really needed here. Fred and Jairo have coded round this is in the time-honoured APL tradition effectively via "execute" with the subsequent requirement of doubling up on quotes, but were open to suggestions as to how they should code it.

The result of code being RunAtServer will affect the style of writing your APL, just as happened between writing APL for running under DOS and then under "Event Driven" Windows. When running code over TCP/IP (e.g. the Internet) a result CANNOT be guaranteed. Requests time out, connections get dropped, networks go down, sites get overloaded. You cannot be sure a request has been received, or if received, actioned, until an acknowledgement has been received back. The acknowledgement itself may get lost, or delayed. As a consequence, incoming "acknowledgements" are effectively events that you use to trigger the next part of your code. (This is a requirement of writing code to run over "imperfect" networks, not a result of a poor implementation of RunAtServer.) So following a RunAtServer request, the code awaits an event that could be say, a time-out, the user aborts the process, or the acknowledgement of success. How each of these events is handled will be up to the programmer.

- **ListView.** The ListView common custom control is not available on all platforms. So running programs containing ListView controls will not work without supplying additional programs to run it on say the iMAC. (I am not clear on the exact limitations here; my notes on this point are incomplete.)

- **Zero Footprint.** (I am unsure of the exact details here, what follows is my belief, but don't take it as gospel. Much depends on the setting of your browser privacy and security "options".) The average web user, armed with IE 6.0, on visiting a web page containing APL2000.JS code, will, on the first visit, invisibly and automatically download the APL2000.JS and the associated workspace along with the HTML. On subsequent visits, these files will only be downloaded if they have changed. This is normal practice at any site with JS or JavaScript, but not quite a Zero Footprint. I believe a true Zero Footprint can be obtained if really required by setting your security/privacy at the "Paranoid" type level.
- **Offline viewing.** Can the web pages with APL2000.JS be viewed off-line? Yes, if they do not contain RunAtServer requests, and you save the web page explicitly for Offline viewing.
- **The browser BACK button.** If the user who is actively running an APL2000.JS workspace presses the BACK button on the browser, APL sees this effectively as "OFF". A subsequent press of the FORWARD button will re-load the web page and start the `DLX`. To prevent this, the initial web page should do nothing except "open in new window" the web page containing the APL2000.JS and workspace. This new window (being new) will have its BACK button greyed out, preventing any problems. By switching to the initial web page, the user can still navigate through the session history via the BACK and FORWARD buttons safely without affecting APL2000.JS.
- **Speed.** A major point of issue with APL interpreters is often speed. How fast/slow is a program? Well, in the friendly words on the back cover of the *Hitch Hiker's Guide to the Galaxy* "DON'T PANIC!". In tests using IE 6.0, APL2000.JS is FASTER than APL2000.EXE! (Shock! Horror! Gasp!)

APL2000.JS is a major achievement, with very far-reaching consequences.

Conference Days

To me the highlight of Monday's sessions (i.e. excluding the Sunset Dinner and entertainment and other meals) was the COMIC APL Interpreter sessions by Bill Rutiser (Cognos Finance) and Jonathan Manktelow (Causeway).

In "Introducing the COMIC APL Interpreter" Bill told us that this is an entirely new APL2000 interpreter (that is it contains NO legacy code left over from earlier APL*PLUS) and is an "in-process COM server, supporting self-contained APL computational components for use with Internet Explorer, Excel, Word and other COM applications".

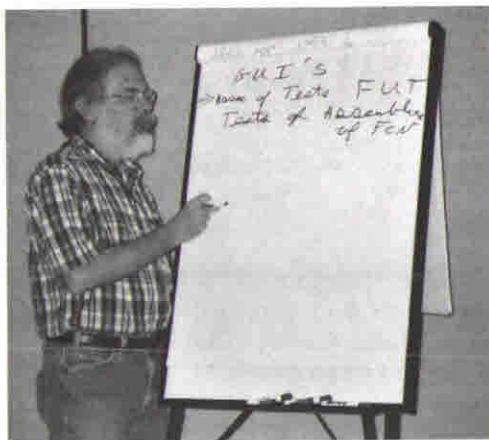
What this means is that the APL workspace is combined with the COMIC interpreter to produce a single DLL for use via other Windows applications.

Jonathan's talk on "Using the COMIC APL Interpreter" was a step-by-step guide to creating a stand-alone DLL from the current COMIC interpreter. This will be included in *Vector* when COMIC is ready for release – the interfaces in the current version are all subject to change.

The final session on Monday was Ian Clark on "The 'Killer App': How to Make Millions With Ground Breaking Software". This was a repeat of the excellent talk he gave at Berlin last year.

Tuesday's sessions included Eric Landau's "Strategies for Dealing with Multiple Arguments: The Case for Names" (see page NN of this *Vector*). As a consultant who is primarily a Dyalog APL developer, I attend the APL2000 Users' Conferences to obtain as much exposure to/experience of APL2000 code as I can get. Having read the excellent paper (supplied as part of the 300-page proceedings) and chatting with him before the session, I felt I could gain more by attending an APL2000 specific paper.

Thus I attended Marvin Renich's "NI Revisited", which taught me about APL2000's Network Interface system function for communicating with the Internet. I found this session invaluable, as the implementation methods that APL2000 and Dyadic have adopted are so fundamentally different.



Following on from last year's conference, I could not miss Bill Rutiser's "Test First: Design and Programming". This was a tutorial on a fundamental *Extreme Programming* practice. It was delivered with Bill's usual very dry wit. ("There are three types of mathematicians, those who can count, and those who can't.")

The last concurrent session I attended was again decided by APL2000/Dyalog differences. I decided against Gary Bergquist's "Limbering Up – Exercising Your APL Muscle" and went for Patrick Parks's "WCALL" session instead. This showed how to call the Win32 API, something I currently do a lot of (I wrote three new utilities in the last week).

After Hours

In addition to learning, APL conferences are about meeting old friends and making new ones. This is done over the meal breaks, at the bars, and in "Evening Seminars".



One such event occurred after the Banquet (where I was taught how to play a kind of poker using the serial numbers on dollar bills).

Lynne Shaw, Ian Clarke, and myself retired with Roy Sykes to his room, where he had a rather fine single malt whisky. This was attacked with some gusto while sitting on his balcony with a fine view of moonlight reflecting off the Gulf of Mexico framed by palm trees.

Strategies for Dealing with Multiple Arguments: the Case for Names

by Eric Landau, APL Solutions, Inc.

An earlier version of this article appeared in the proceedings of the 2001 APL2000 User Conference.

Once upon a time, an APL function could take at most two arguments. If the logic of the function required a larger number of arguments, the programmer had to come up with a scheme by which all of them were represented by at most two homogeneous arrays, putting character data in the left argument and numeric data in the right argument or vice versa.

With the introduction of nested and heterogeneous arrays and strand notation, it became trivial to write functions that, from the user's perspective, allowed arguments of any type or shape to be "strung out" as a strand. Each element could be assigned to its own variable internally, eliminating the necessity to "break" the logical arguments apart and rearrange them into their desired form. Today we can write functions that look like the following, which has, as far as a user (or calling function) is concerned, twelve arguments:

```

v Example1 args;title;per;ampl;vmin;vmax;voff;hmin;hmax;hoff;axes;ticks;color
[1]  a Arguments:
[2]  a  [1] Title (use '' for blank)
[3]  a  [2] Period
[4]  a  [3] Amplitude
[5]  a  [4] Vertical minimum
[6]  a  [5] Vertical maximum
[7]  a  [6] Vertical offset
[8]  a  [7] Horizontal minimum
[9]  a  [8] Horizontal maximum
[10] a  [9] Horizontal offset
[11] a [10] Displayed axes ('x'; 'y'; 'xy' or 'yx' for both; '' for neither)
[12] a [11] Tick interval (x,y; 0 for do not display)
[13] a [12] Colors (bg rgb triple, fg rgb triple)
[14] a
[15] a {title per ampl vmin vmax voff hmin hmax hoff axes ticks color}←args
[16] a Processing...
v

```

To invoke this function, one supplies an argument strand containing twelve values:

```
Example1 'Curve #1' (o2) 4 ^5 5 0 0 1 0 'xy' (.8,o.2)
((0 0 0) (255 255 255))
```

Chances are, though, that the user of this function doesn't really care about some, perhaps even most, of these argument values. We'd like to be able to offer our users appropriate default values for any or all of them, allowing them to supply only those arguments that they are concerned about. To do this, we need only select some value that would be inappropriate for any of the arguments, and use it as a "placeholder" to represent a suitable default value. From the user's perspective, this value represents "I don't care". The refined function looks like:

```
v Example2 args;title;per;ampl;vmin;vmax;voff;hmin;hmax;hoff;axes;ticks;color;s
[1]  A Arguments:
[2]  A [1] Title (default '')
[3]  A [2] Period (default 2*pi)
[4]  A [3] Amplitude (default 1)
[5]  A [4] Vertical minimum (default ^1)
[6]  A [5] Vertical maximum (default 1)
[7]  A [6] Vertical offset (default 0)
[8]  A [7] Horizontal minimum (default 0)
[9]  A [8] Horizontal maximum (default 2*pi)
[10] A [9] Horizontal offset (default 0)
[11] A [10] Displayed axes: 'x'; 'y'; 'xy' or 'yx' for both; '' for neither
[12] A (default 'xy')
[13] A [11] Tick interval: x,y; 0 for do not display (default (0.2,0.2*pi)
[14] A [12] Colors: bg rgb triple, fg rgb triple (default (0 0 0)(255 255 255))
[15] A Use 0 for default value.
[16] A
[17] (s/args)+(s+args#"^c0")/' (o2) 1 ^1 1 0 0 (o2) 0 'xy' (.2,o.2)
      ((0 0 0)(255 255 255))
[18] (title per ampl vmin vmax voff hmin hmax hoff axes ticks color)+args
[19] A Processing...
v
```

With this function, the user who cares about only a few values can specify only those, and allow the rest to take on their programmer-defined default values:

```
Example2 'Curve #1' 0 4 ^5 5 0 0 0 0 0 0 0 0
```

The user need only know where to put the values he (or she) does care about, and need not even know what the rest of the arguments represent.

He does, however, still need to know how many arguments are defined, and must type the appropriate number of trailing placeholders.

We can relieve him of these burdens easily enough:

```

v Example3 args;title;per;ampl;vmin;vmax;voff;hmin;hmax;hoff;axes;ticks;color;s
[1]  A Arguments:
[2]  A [1] Title (required, use '' for blank)
[3]  A [2] Period (default 2*pi)
[4]  A [3] Amplitude (default 1)
[5]  A [4] Vertical minimum (default -1)
[6]  A [5] Vertical maximum (default 1)
[7]  A [6] Vertical offset (default 0)
[8]  A [7] Horizontal minimum (default 0)
[9]  A [8] Horizontal maximum (default 2*pi)
[10] A [9] Horizontal offset (default 0)
[11] A [10] Displayed axes: 'x'; 'y'; 'xy' or 'yx' for both; '' for neither
[12] A (default 'xy')
[13] A [11] Tick interval: x,y; 0 for do not display (default (0.2,0.2*pi)
[14] A [12] Colors: bg rgb triple, fg rgb triple (default (0 0 0)(255 255 255))
[15] A Use 0 for default value. Trailing arguments to be defaulted may be elided.
[16] A
[17] :IF 02=0DR args 0 args+.cargs 0 :END A In case title was the only argument
[18] args+12+args,12p<0
[19] (s/args)+(s+args=""c0)/'' (02) 1 -1 1 0 0 (02) 0 'xy' (.2,0.2)
((0 0 0)(255 255 255))
[20] (title per ampl vmin vmax voff hmin hmax hoff axes ticks color)+args
[21] A Processing...
v

```

In this version, the user can type:

Example3 'Curve #1' 0 4 -5 5

This produces the same result as does the previous example.

Notice that, as in Example2, the code permits the user to give the first argument as 0, which will set the title to '', but the documentation in the function has been deliberately modified to conceal this. If we were to allow the user to supply a placeholder for any argument, and then allow him to omit trailing placeholders, he would be entitled to assume that he could type:

Example3

to run the function with all of the arguments taking on their default values. In reality, of course, doing so would raise a syntax error, as APL does not permit us to define functions which are ambivalently either niladic or monadic. A user who really wants to default all of the arguments must nevertheless supply either a value or a placeholder for (only) the first one, which is counter-intuitive. Rather than document this as a special-case exception, it is more readily understandable to simply tell the user that the first argument must be supplied. It helps if the first

argument has an easy and obvious (and documented!) "logical default" value, such as a blank title.

All of the default values in Example2 and Example3 are assigned "simultaneously", i.e. in a single statement. This is easy enough, but may be unrealistic. Sometimes the best (or only appropriate) default value for a particular argument will not be a constant, but rather some function of the other arguments. In that case, we will need to assign default values individually. If the computation of these default values from some set of other argument values is trivial, we can simply replace the statement in Example3[19] with twelve individual assignment statements. Sometimes, however, a suitable default may be the result of a complex calculation that we need to do to get the result, and so we'd like to set the default as a function of some intermediate result of some prior calculation. In that case, we would want to scatter the individual assignments of the defaults throughout the function.

When we do that, we wind up with a function that looks something like:

```

v Example4 args;title;per;ampl;vmin;vmax;voff;hmin;hmax;hoff;axes;ticks;color
a Arguments:
[2] a [1] Title (required, use '' for blank)
[3] a [2] Period (default 2*pi)
[4] a [3] Amplitude (default 1)
[5] a [4] Vertical minimum (default -arg[3])
[6] a [5] Vertical maximum (default arg[3])
[7] a [6] Vertical offset (default 0)
[8] a [7] Horizontal minimum (default 0)
[9] a [8] Horizontal maximum (default arg[2])
[10] a [9] Horizontal offset (default 0)
[11] a [10] Displayed axes: 'x'; 'y'; 'xy' or 'yx' for both; '' for neither
[12] a (default 'xy')
[13] a [11] Tick interval: x,y; 0 for do not display (default (0.2*arg[3]),
[14] a 0.1*arg[2])
[15] a [12] Colors: bg rgb triple, fg rgb triple (default (0 0 0)(255 255 255))
[16] a Use 0 for default value. Trailing arguments to be defaulted may be elided.
[17] a
[18] :IF 82=DR args o args+,<args o :END a In case title was the only argument
[19] args+12+args,12p=0
[20] (title per ampl vmin vmax voff hmin hmax hoff axes ticks color)+args
[21] :IF title=0 o title+'' o :END
[22] :IF per=0 o per+02 o :END
[23] :IF ampl=0 o ampl+1 o :END
[24] a Processing...
[25] :IF vmin=0 o vmin+ampl o :END
[26] :IF vmax=0 o vmax+ampl o :END
[27] :IF voff=0 o voff+0 o :END
[28] a Processing...
[29] :IF hmin=0 o hmin+0 o :END
[30] :IF hmax=0 o hmax+per o :END
[31] :IF hoff=0 o hoff+0 o :END

```

```

[32]  a Processing...
[33]  :IF axes=0 o axes+'xy' o :END
[34]  :IF ticks=0 o ticks+ (.2*ampl),.1*per o :END
[35]  :IF color=0 o color+(0 0 0) (255 255 255) o :END
[36]  a Processing...

```

▼

We could mix these techniques, assigning constant defaults in one statement, "easy" calculated defaults in separate statements at the top of the function, and complex calculated defaults as the intermediate results become available, but our function will be much easier to read and follow if we handle all of the defaults the same way. In addition, we make the function somewhat easier to follow by putting the setting of the default values for the arguments near the sections of code where the arguments first get used, so the reader doesn't have to refer back to the top of the function repeatedly to see what they are. These refinements provide us with a lot more flexibility in choosing default values, but are transparent to the end user, who invokes Example4 identically to Example3:

Example4 'Curve #1' 0 4 ~5 5

Example4 is a significant improvement over Example1, but we can use the power of nested arrays and strand notation to do a lot better. In Example4, the user must still remember (or look up, each time) the positions of the particular elements of the argument strand that he is interested in, and must still type a placeholder 0 for each non-trailing argument he doesn't care about. We can relieve him of these burdens as well. To do this, we must abandon the traditional view of formal arguments as collections of argument values.

To implement our final strategy, which we refer to as "named arguments", we assign each argument a meaningful name. Our user will no longer need to remember some set of meaningless index values that are arbitrarily associated with the arguments of interest to him. Instead, he need remember only the names. (If there are users out there who find remembering that amplitude and vertical maximum are in the 3rd and 5th positions respectively easier than remembering that amplitude and vertical maximum are called 'ampl' and 'vmax' – or 'amplitude' and 'vertical_maximum' – respectively, I'm not at all sure I want to meet them.) The formal arguments to the function are used to pass associations between names and values rather than just values. When our function needs to assign a value to a logical argument, it uses the associated name to select the appropriate value instead of using its position in the argument strand. The position becomes meaningless to the programmer (as it already was to the user), and the user can enter the arguments of interest to him in any order he chooses.

We can write a utility function that will take a vector of pairs of names and associated values and, for a given name, return the associated value if there is one, otherwise return a given default value. While that's all we need, I have found it helpful for debugging purposes to have it also remove the name whose value is being extracted (along with its value) from the argument vector, so that the current value of the variable that represents the argument to the calling function contains only those logical arguments that have not yet been extracted. In addition, I find it helpful for it to incorporate a "meta-default" value, which can, among other uses, serve as a flag to indicate that a required argument has been omitted, so that the lack of an explicitly passed default value lets the reader of the calling function know that an explicit value is expected. The following function does all of this:

```

v R←ad ArgValue args;arg;default;s
[1]  a Separates the value associated with a name from a vector of name-val pairs.
[2]  a args is a vector of name-value pairs. Names must be character scalars
[3]  a or vectors.
[4]  a ad[1] is the name for which the associated value will be separated.
[5]  a ad[2] (optional) is the value to be returned if ad[1] does not appear as
[6]  a a name in args (default 0).
[7]  a R[1] is the value associated with ad[1], or its default (ad[2] or 0).
[8]  a R[2] is args with the pair corresponding to ad[1] removed.
[9]  a
[10] :IF ~(DDR ad)€326 807 0 ad←cad 0 :END a ad may be an unenclosed name
[11] (arg default)+2+ad,€0 a Supply default if not given
[12] :IF v/s+(1"args)€carg a Name is in args
[13] R+(2>s/args) ((~s)/args) a Return (value) (remaining elements of args)
[14] :ELSE a Name is not in args
[15] R←default args a Return (default value) (all elements of args)
[16] :ENDIF
v

```

Using this function, we can rewrite Example4 as:

```

v Example5 args;title;per;ampl;vmin;vmax;voff;hmin;hmax;hoff;axes;ticks;color
[1]  a Argument names:
[2]  a 'title': Title (required, use '' for blank)
[3]  a 'per': Period (default 2*pi)
[4]  a 'ampl': Amplitude (default 1)
[5]  a 'vmin': Vertical minimum (default -arg[3])
[6]  a 'vmax': Vertical maximum (default arg[3])
[7]  a 'voff': Vertical offset (default 0)
[8]  a 'hmin': Horizontal minimum (default 0)
[9]  a 'hmax': Horizontal maximum (default arg[2])
[10] a 'hoff': Horizontal offset (default 0)
[11] a 'axes': Displayed axes: 'x'; 'y'; 'xy' or 'yx' for both; '' for neither
[12] a (default 'xy')
[13] a 'ticks': Tick interval: x,y; 0 for do not display (default 0.2*arg[3]),
[14] a 0.1*arg[2])
[15] a 'color': Colors: bg rgb trpl, fg rgb trpl (default (0 0 0)(255 255 255))

```

```

[16]  A
[17]  :IF 82=[DR +args o args+,cargs o :END
[18]  (title args)+'title' ' ' ArgValue args
[19]  (per args)+'per' (o2) ArgValue args
[20]  (ampl args)+'ampl' 1 ArgValue args
[21]  A Processing...
[22]  (vmin args)+'vmin' (-ampl) ArgValue args
[23]  (vmax args)+'vmax' ampl ArgValue args
[24]  (voff args)+'voff' 0 ArgValue args
[25]  A Processing...
[26]  (hmin args)+'hmin' 0 ArgValue args
[27]  (hmax args)+'hmax' per ArgValue args
[28]  (hoff args)+'hoff' 0 ArgValue args
[29]  A Processing...
[30]  (axes args)+'axes' 'xy' ArgValue args
[31]  (ticks args)+'ticks' ((.2*ampl),.1*per) ArgValue args
[32]  color+'color' ((0 0 0) (255 255 255)) ArgValue args
[33]  A Processing...
v

```

Now the user who only cares about the title, amplitude, vertical minimum and vertical maximum, as in Examples 2 through 4, can type:

```
Example5 ('title' 'Curve #1') ('ampl' 4) ('vmin' 5) ('vmax' 5)
```

or:

```
Example5 ('vmin' 5) ('vmax' 5) ('ampl' 4) ('title' 'Curve #1')
```

or any other permutation of the arguments; position in the strand has become meaningless. He need not know what the arguments of no interest to him are, how many there are, or even that they exist.

Note the slight bit of additional user-friendliness provided by Example5[17], which allows the user who only cares about one argument (and presumably has no idea what a "strand" is) to type, say:

```
Example5 ('title' 'Curve #1')
```

which is consistent with what he types when supplying more than one argument, rather than:

```
Example5 ,c'title' 'Curve #1'
```

which is noticeably different from the more general case, and looks rather odd.

The next time you write a function that needs to take multiple arguments, some of which users are likely to want to allow to take on default values, consider using named arguments. You'll be making your users' lives easier.

What, in Computing, is Worth Learning?

by Anthony Camacho

A Statement

I started programming in 1958 and was immediately irritated by the programmer habit of calling an instruction a "statement". I have done my best to avoid following this unseemly trend. I think it may be (at least indirectly) responsible for some of the many difficulties we labour under today.

The Imperative Voice

When I learned grammar the imperative was the voice used for commands. All communications to a computer are in the imperative voice. Every press of a key or movement of a mouse or click of a mouse button (or movement of a mouse wheel) is a command and, if the computer is working, all it can do is to obey. If we were all always clear that what we are doing when we operate a computer is commanding it to do things then we might take the trouble to find out what the commands actually are and might be less willing to accept the computer man's starting point (which is "I know better than you what you are trying to do and will make the computer do it my way").

The Will

People have wishes; computers do not. Computers may be programmed to pursue an assigned goal and thus mislead the naïve into thinking computers want things. If you can only communicate with another being in the imperative voice, and the only responses are obedient or random, I don't know whether it would be possible to discern a wish, even allowing for the possibility that a computer can have one (which I emphatically deny).

Thinking

Doing logic may be thinking, but if all you can do is logic and if you can only do it when told and with specified premises, then that is probably *not* thinking. If Babbage's Analytic Engine had ever been made, its processing could hardly have been called thinking, any more than the Turing machine's processes. Perhaps it is of the essence of thinking that the train of thought can be interrupted. A computer's train of thought cannot be interrupted in the sense that an unforeseen

idea arising during the logic can divert the train of thought to change its goal. Let me hasten to deny that I am saying that thinking lies in an unpredictable outcome. What I am saying is that if the outcome is completely determined then I doubt if the process is, beyond doubt, "thinking".

'Helping' You

I now have four grandchildren and am well qualified to appreciate the consequences of accepting an offer of help from any or all of them. The help offered by Microsoft is worse than what my grandchildren can do. At least *they* make no pretence of trying to further my wishes. Microsoft staff (and not only them) try to help by knowing better than I do what it is I want to do. I cannot begin a paragraph with a number or a bullet without getting all subsequent paragraphs beginning with a number in sequence or a bullet. This is the default behaviour! The time and trouble it takes to stop the program doing this is more than I would have spent in ten years numbering paragraphs by hand.

Concealment

In pursuing the aim of making the machine 'friendly' to the user, Microsoft have largely hidden from the user, the logical control of the computer. For example on this new computer I have five icons on the desktop that I know offer to get updates, make contacts etc. with the internet. Two of them are the machine manufacturer's (set up and connect icons) and one is the virus software update but the rest are Microsoft. I don't have a clear idea what will happen if I click on one of these. I have not entered any telephone numbers, so I don't know what will be dialled. For all I know the call will be made to a call centre in Hong Kong. I can check my telephone bill on calls I dial or when I have entered the number into the computer, but not when I use these icons. Furthermore there are programs that run when my machine first boots up that I do not want to run and that I cannot find out how to stop.

When Things Go Wrong

In particular I haven't any way of telling how to correct things if something goes wrong. When I clicked the option on the virus software window to create a rescue disk it told me that Rescue 32 had executed an invalid instruction in module KRNL386.EXE at 0001:00002d51 and when I clicked the box to continue I was shown a blue screen with "A fatal exception OE has occurred at 0187:BFF9DFFF. The current application will be terminated". There is no way of telling what went wrong. When I learned about planning, I was taught that a good plan was one

which allowed for all eventualities; it foresaw that there would be difficulties and provided some way of overcoming them, even if there was no easy way of knowing precisely what kind of difficulties they would be. The current machine philosophy is to plan for everything going right first time and, if things don't, tough!

No Manuals

The replacement of manuals (which required that someone made an intellectual effort to organise the information logically) by help files (which don't) has been a disaster. The typical help file begins by assuming that you need to be told what options there are in the menu on the screen and what each of them does. If what you need is not allowed for then there is nothing more you can do with it. It is possible that I have forgotten it if I have ever found something in a help file that was helpful but I certainly can remember many occasions when I didn't. It is not even possible to find out what files provide which facilities. On my other machine the scroll bar sometimes fails: the symptom is that scrolling begins and continues whenever the mouse is anywhere on the bar. Furthermore the direction of scrolling varies. I suspect that if I knew which .DLL or .VXD file provided the scroll bars I could reload it from a Windows Cabinet and my troubles would be over - but there is no practical way of finding out. There is a horrible tendency for the style to creep out of the machine onto paper. Some of the books I have recently bought follow the help file pattern: they propose a sequence of steps (all expressed in the imperative mood) which the author thinks will work (and maybe does on his machine) but without any allowance for the steps that don't work as expected. Imagine a motor manual which tried to tell how to diagnose and repair an internal combustion engine entirely by reference to the driver's controls.

Divide and Rule

I buy a first class machine from one of the most reputable makers. They send it with the wrong network card installed. After three weeks of ringing them and emailing them and even writing letters of complaint to the sales director, I get a network card with a plug to suit my network wiring. Then, when I set it up according to the instructions, it doesn't work as it should. To begin with it can only see one other machine on the network. Then after trying so many things that I cannot report them coherently to the help line, it gets to see three machines. The fourth (which the others can all see and share files with) is still invisible. The help line politely declines to help saying that the manufacturer cannot solve network problems as too much is outside their control. Microsoft too decline to help on the grounds that their arrangement with the manufacturer requires the manufacturer

to provide operating system support. The network card manufacturer's email help service sends me a standard general purpose debugging guide which tells me that I have to specify a machine name and the correct network name (I had, of course, told them that the rest of the network was working and that the problem machine could see one of the other machines: if they had bothered to read what I wrote they would have known I had done all that.) and ends "NOTE: for more information on peer to peer networking in Microsoft Windows 95/98 go to Microsoft's website or contact Microsoft's Technical support".

Parrot Learning

Help files and help lines are, as described above, completely hooked on giving sequences of instructions. They almost never explain what the instructions command the machine to do. When they do give a purpose for an action it is usually an anthropomorphic one: "This tells the machine where to find the XX file." On one occasion, when I was asking a help line why my instructions were not adequate to make my machine do what I wanted, the guy on the other end was telling me, one step at a time what to do and I recognised 'DELTREE' and asked him what he was doing. After a little gentle verbal fencing he admitted that he was intending to re-install my Windows system from scratch! He hadn't asked me. He hadn't explained to me the consequences of such an action. Such people don't. This approach conceals from people the nature of their actions. I don't know where to find the other approach; one that assumes the user knows what he wants and explains the principles underlying the steps needed to get it and maybe even the main things that could go wrong and how to overcome them.

Subordinate Learning

It is the consequences of all these things that make me deeply suspicious of the value of teaching IT (information technology – it doesn't deserve its capital letters) in schools. Mainly, what people learn is how to operate pieces of software. Doing this is mainly a process of learning, parrot fashion, the sequences of mouse movements and clicks to do some simple task such as mailing a letter to all the people on an address list. Then the pupil goes away and discovers that, on the next machine he tries, the software is entirely different and the painfully acquired parrot learning is largely useless. (Actually the varieties of word processors and spreadsheets are so similar in their aims and in the ways the computer can be programmed to provide the commands that a good deal of the range of commands is re-usable.) Nevertheless it is a grave defect of computer learning that what is taught is the parrot-fashion sequences needed to operate a piece of software that will almost certainly be superseded before the pupil has even left

school. I once worked out that I am now on my ninth word processor, counting the four versions of Word for Windows as one, and most of the effort of learning them was wasted.

A Profession?

The mark of a profession is that its respected practitioners know what they are doing and can tell how much work they have to do to complete a task or a stage in the work. In some professions, such as building, there are even people (quantity surveyors) whose job is to measure how much is done and how much there is to do. There is no equivalent of a quantity surveyor in computing. Very few programmers can keep to their estimates. Even fewer are secure enough to dare to offer fixed prices (or schedules) for the work. It is the norm that computer projects are delivered late and cost more than estimated. It is even quite common for a project to get abandoned after large expenditure because the budget holder belatedly realises that he is unable to control the work he wants done. The mark of a professional programmer is that he will do a fixed price job once the specification is agreed. There are very few such people. Of course it is partly the problems described above which put people off. If I knew that I would be able to get Windows to do what I wanted I would be able to give a fixed price for the whole job. Since I may have to spend a week (no exaggeration) getting a new machine to communicate over the network, I have to include rather large safety margins if the estimate is to include getting a program to work under Windows. So does everyone else.

Worth Learning?

It would be worth learning how to produce a reliable estimate. It would be worth learning how to regain control of the machine from the operating system. What will continue to be relevant in ten years is more worth learning than anything that will be superseded. It is not easy to distinguish between these.

Hardware Changes

When I started programming the IBM 650 (the first commercial computer of which 1000 were sold) I refused to believe that the program I wrote had run correctly because there was no pause between reading the punched cards containing my first program and the punching out of the result. I was wrong; the result was correct. (Incidentally that may have been the last program I wrote that ran first time, error free.) The 650 had a word time of 0.96 milliseconds; each location on the drum became available once every 48 milliseconds and if the

locations of addends and result on the drum were optimised an addition could be done in about 7 milliseconds. The drum stored 2000 ten-digit numbers. The machine I am typing this on has a word time of a nanosecond and 128 million bytes of memory available at about three nanoseconds plus forty thousand million bytes on a disk available 4096 bytes at a time with a delay of under ten milliseconds. Its operations are a million times faster, its access to data about ten thousand times faster and it can store at least two million times as much, so overall it is about sixteen orders of magnitude more capable. The IBM 650 cost £75,000 in 1958 money – say the equivalent of £3,000,000 to £4,000,000 today.

Software Changes

Much as the hardware has changed, I think the software has changed more, but the measures to judge it by are not so easy to see. The hardware does the same things, but faster and with more data; the software now handles pictures and music and video and the interface with the user is completely different. The instruction codes, the utilities, the calls to the operating system, all have been changed and the operating system is now huge. Gone are the days when one could blow one 4K ROM chip and have a reasonable debugger available on command. Debug, NasDis (disassembler) and Zeap (Z80 assembler) all fit into 8 chips. Nowadays even Visual BASIC takes two 1.44 Mbyte disks. The Windows operating system nowadays uses 90% or more of the machine resources!

For some years one could rely on DOS doing what it used to do the year before. The calls to read and write a file remained unchanged for many years. Now DOS features that Windows used to support are being phased out, even this reliance is shaky. I am told that I can no longer (in Windows ME or 2000) do the things I used to do in CONFIG.SYS and AUTOEXEC.BAT. Not only that: there is no longer a way to do all of those things!

What Can't Change?

The problem is to discern the stable features of the landscape. What is there that one can rely on staying the same?

Computers will always be slaves

A computer is an algorithmic machine: all communications to it are in the imperative voice. This will still be true and there will still be programs in ten years. The programming languages then in use will be those that have proved themselves suitable. Perhaps, by then, people will have ceased to treat Edsger Dijkstra's view that learning BASIC causes brain damage as a clever-clever joke

and begun to realise that it is true. Some of today's programs will still be running then, of course, and so some of the programming languages you can learn today will still be saleable skills in ten and maybe even twenty years.

All computers will ever hold is a binary representation

A computer is a binary machine: everything it stores must be converted to and from a binary representation. The representations of symbols, values and pictures will still have to be translated to and from binary. The established representations will doubtless be joined by others, but are themselves unlikely to be superseded. It is interesting to consider how long IBM's double precision 64-bit floating point numbers will be the primary scientific numerical representation and whether what supersedes them will just have greater precision or have a variable precision (maybe perfect – without any tolerance arising from the representation at all). One programming language already provides this. It is called J (not Java).

Computer data will always be held in files

Information will continue to be held in files and file handling will continue to be a major part of data processing. It may be that some file types become established as the most suitable for the kind of data to be held. Typefaces, for example, which were once held as bit maps will plainly never be held that way again; they are held as the curves of their outlines so that there are no limits on enlargement and reduction. It would be a surprise if the multiplicity of formats for pictures doesn't shake out a bit in ten years. Word 2000, when asked, offers to import a picture from any of about 35 different file formats.

Non-computer people will always be responsible for the work

The jobs computers do (including personal computers) will still be the responsibility of the people that bear the responsibility for seeing that the work gets done and not of the programmers and analysts that design the computer systems. Although programmers and analysts are usually paid more, an application's success depends on how well it meets the needs of the users. It is a common fault that the system only does what the analyst *thinks* the user wants. Success sometimes depends on the user feeling a commitment to make a system work and this is unusual unless the user has fully understood and agreed to the design. This ought to be obvious (but see HMW's sustaining news in Vector 17.4). Companies ought to put users in charge of the computer people to ensure that the system design can do the job and that the change-over procedure is practical and offers fall-back at every stage. Nowadays the fact that computer people are more highly paid often means that *their* convenience overrides the convenience of the

people responsible for the work. This is just one of the many reasons why projects fail.

The user will always have second thoughts

It will still be true that one has to walk before one can run. New systems should be introduced partially or on samples of the work and proved to be satisfactory before gradually taking over the whole of the work. Pilot running is better than parallel running and if the system, at the pilot stage, can be changed to incorporate every improvement that the responsible staff can see, so much the better.

So we need ways to run pilot systems with prototype programs that can easily be amended to incorporate the improvements that are only thought of when the staff have some experience with the pilot.

The big problem will always be to discern the real need

Users and computer people can easily be misled (mislead themselves – mislead each other) and agree to designs that don't work when installed. Far too little effort is made by managers to ensure that their staff can use the system they agree to. When the staff try the prototype, listen to what they say: if changes are not needed then the system is exceptional and the designer deserves a bonus.

. . . . and to agree how the system will meet it – an agreement that will stick

Even if the staff have agreed and signed the design in blood, the system may need to be changed. There is no escaping the need for a prototype. To meet these last two needs calls for the skill to convert between expressions of the users' needs and expressions of the systems design. The field specifications and the range of values each can take and the screen designs and how the user will interact with each of them have to be agreed with the user and the prototype changed in the light of experience in use. The structure of files, of processes, of validations, of file amendments, of back-up arrangements, of security and of response times are all things the designer should handle without confusing the user.

The way these aspects are expressed in the design may make a significant difference in the difficulty of translating the constraints and benefits of a particular design into expressions the user can appreciate.

Conventional wisdom has it that the benefit of a high level language is that it relieves the programmer of much of the drudgery in converting the program specification into machine code and of course it does do this. Systems don't

usually get into difficulties because the program doesn't meet the program specification. The usual problem is that the delivered system doesn't meet the real-world need.

Actually the main benefit of a high level language is the same benefit A. N. Whitehead claimed for a good notation:

By relieving the brain of all unnecessary work, a good notation sets it free to concentrate on more advanced problems, and in effect increases the mental power of the race.¹

¹Introduction to mathematics: A N Whitehead: Chapter 5

Vector Back Numbers

Back numbers of Vector are available from:

**British APL Association,
c/o Gill Smith,
Brook House, Gilling East,
YORK YO62 4JJ**

Price in UK: £10 per complete volume (4 issues);
£12 (overseas); £16 (airmail) including postage.

Please note that Vol.1 No.2 is now out of stock.

The Image2 Addon

by Clifford A. Reiter and Zachary X. Reiter

In an earlier note Cliff described the first release of the image addon for J [5]. Meanwhile, Zach was becoming experienced with Linux and C. He took on the project of doing direct file to memory reading of jpeg files as a warm-up exercise. Since the approach in [5] involved intermediate conversion to a temporary bitmap file and then reading the result into J, Zach expected his version would be more efficient. Moreover, because of the use of a third party dll in the first release, the image addon was not available for Linux. Zach hoped to change that too. His project was successful and there was a flurry of follow-up activity: *.png support was also redone and support for 24 bit portable bitmaps was added. Meanwhile, the html gallery script that Cliff wrote had undergone changes and some other utilities had been developed, so it seemed worthwhile to offer a new release of the image addon. The addon may be downloaded from [3].

Like the first release, a lab introduces users to the image2 addon. Even though the Windows lab uses wd commands, the availability of tcl/tk under linux meant that only minor changes were made to allow the lab to run under Linux. In this note we will list some of the major changes from the first release, give performance comparisons, take a look at a rotation utility, and we will also give an illustration of nontrivial gallery maintenance. Readers experienced with the first release will probably want to also read the minor changes in the release notes.

The choice of open source code for reading and writing jpeg and png files changed for release 2 of the image addon. The jpeg library is now based upon the independent jpeg group's library version 6b [2]. The png library is based upon version 1.0.8 of libpng, [7]. This means that many of the core image addon functions have changed, but most of the high level functions use the same syntax as the first release. Here is a summary of the major changes offered by the second release of the addon.

- Availability on both Windows and Linux platforms
- Library (*.dll, *.so) calls are direct, so temporary files are no longer used for jpeg/png reads/writes
- Jpeg read/writes are faster
- Png read/writes are dramatically faster

- Library file is smaller (under Windows, release 1 used three dll's totaling 656k; this is replaced by one dll of size 288k)
- 24 bit Portable aNyMap support is added (*.pnm, *.pgm, *.ppm)
- Limited support for grayscale images as h by w by 1 arrays
- Lossless jpeg transforms (rotations/flips) available
- A script, `rot_form.ijs`, gives a GUI form for rotating images interactively (the windows version is wd based, linux is tcl/tk based)
- Sample scripts included illustrate simple and complex usage

Comparisons

Table 1 shows timing and space comparisons for image reads and writes of various sizes and a combined gallery creation benchmark. All the tests were run on the same machine, a 850 MHz Pentium III PC with 128 MB of memory. The Linux version of J used for the test was 4.05a and the Windows version was 4.05d. Benchmarks for both the first and second release of the Windows version of the addon are given. All the images are 24 bit images. Here the jpeg files being "small", "medium" and "huge" correspond to images with size 423 by 174, 768 by 576, and 2048 by 1536. For the png images, "small", "medium", "large", and "huge" are 100 by 50, 525 by 195, 880 by 880 and 1720 by 1720. The html gallery creation test used 47 PNG and JPEG Images.

Notice that the results are fairly consistent: space usage by release 2 is better than for release 1, but there is very little difference between the Linux and Windows space usage for release 2. Jpeg operations are noticeably faster for release 2, with Linux being marginally better. Png operations are significantly better for release 2, with Linux again being marginally better than Windows again. We note that Dazeley offers J functions for direct JPEG creation although we were unable to run timing tests comparing our library call functions with his direct functions. Still, readers with certain applications in mind may want to use his functions [1].

Host system	Linux Image2		Windows Image2		Windows Image1	
	time (secs)	space (bytes)	time (secs)	space (bytes)	time (secs)	space (bytes)
Benchmarks						
Small JPEG Read	0.0305495	1.31456e6	0.0320786	1.31251e6	0.0727334	2.11104e6
Medium JPEG Read	0.173818	1.04896e7	0.185458	1.04876e7	0.642457	1.67911e7
Huge JPEG Read	1.28645	8.38899e7	1.36022	8.38879e7	4.45657	1.34232e8

Small JPEG Write	0.0547015	1.32966e6	0.0606069	1.32966e6	0.095579	1.59853e6
Medium JPEG Write	0.311928	1.05047e7	0.353221	1.05047e7	0.467396	1.26086e7
Huge JPEG Write	2.2544	8.3905e7	2.51351	8.3905e7	3.24597	1.00689e8
Small PNG Read	0.002567	85760	0.0042236	83712	0.0231914	144960
Medium PNG Read	0.0416605	2.62528e6	0.0436879	2.62323e6	0.352292	4.20819e6
Large PNG Read	0.364219	2.09754e7	0.387315	2.09733e7	2.74285	3.35683e7
Huge PNG Read	1.10173	8.38898e7	1.15532	8.38877e7	10.4585	1.34231e8
Small PNG Write	0.0087595	99584	0.0108659	99584	0.0486519	123840
Medium PNG Write	0.136627	2.6391e6	0.160027	2.6391e6	0.927157	3.17126e6
Large PNG Write	0.985568	2.09892e7	1.13583	2.09892e7	6.36973	2.51914e7
Huge PNG Write	3.18563	8.39037e7	3.75061	8.39037e7	26.5668	1.00689e8
HTML Image Gallery Write	50.0724	9.86659e7	57.9819	9.86666e7	168.131	1.34258e8

Table 1. Comparison of Performance of the Image addon releases

Rotation Form

One of the unpleasant but necessary steps described in [5] was the need to rotate some small proportion of the images. While rotation can routinely be accomplished, the nuisance was in identifying and listing of the names of the images which needed to be rotated. In this release, there is a script, `rot_form.ijs`, that defines an interactive form which displays a preview of an image which may be rotated via GUI buttons, menus or hot keys. By default, trimmed, "lossless" rotations are used. If the image needs to be trimmed (the size is not a multiple of the internal JPEG blocksize), this results in loss of some edge pixels. An option allows the simultaneous creation of thumbnails. If that option is used, the html galleries of the images with that thumbnail size may be very rapidly produced. Figure 1 shows an example of the form in use where the image currently previewed should be rotated clockwise 90 degrees for proper viewing.

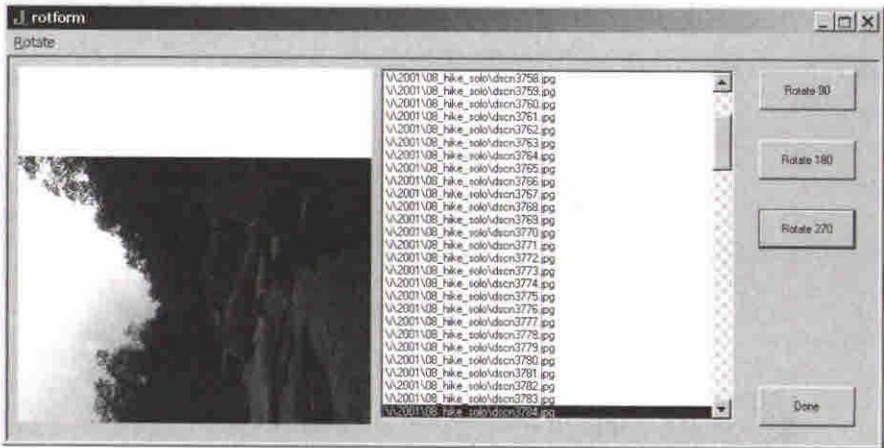


Figure 1. The image rotation form in use.

Preserving Gallery Descriptions

We next describe an example where the html gallery tools are used to maintain a fairly complex set of image galleries. In particular, this is used to maintain the galleries of photos of symmetry, chaos and fractals at [6]. The script we use for that maintenance, *sample2.ijs* is included with the addon. Images are added to any one of several directories and the script, with one line, builds a linked set of image galleries. Figure 2 shows an example of one of the pages. Notice that the gallery has a title and a head image, there is a table of images with associated descriptive text, and at the bottom of the page there is a row thumbnails providing links to all the galleries. As images are added to the gallery, descriptive text is added for each. This is accomplished by editing the html file with a convenient editor (Notepad, Word, Netscape). The preservation of that added descriptive text as new images are added becomes an important requirement. This requires the ability to retrieve information from the existing html galleries and to coordinate it with the updated gallery. We describe how that capability is implemented below. This is meant as an illustration of the flexibility and power of the gallery building tools. While we have successfully retrieved information from html files after editing with the above noted editors, html editors may make extensive changes to files, and hence it is prudent to maintain an archive copy of the gallery.

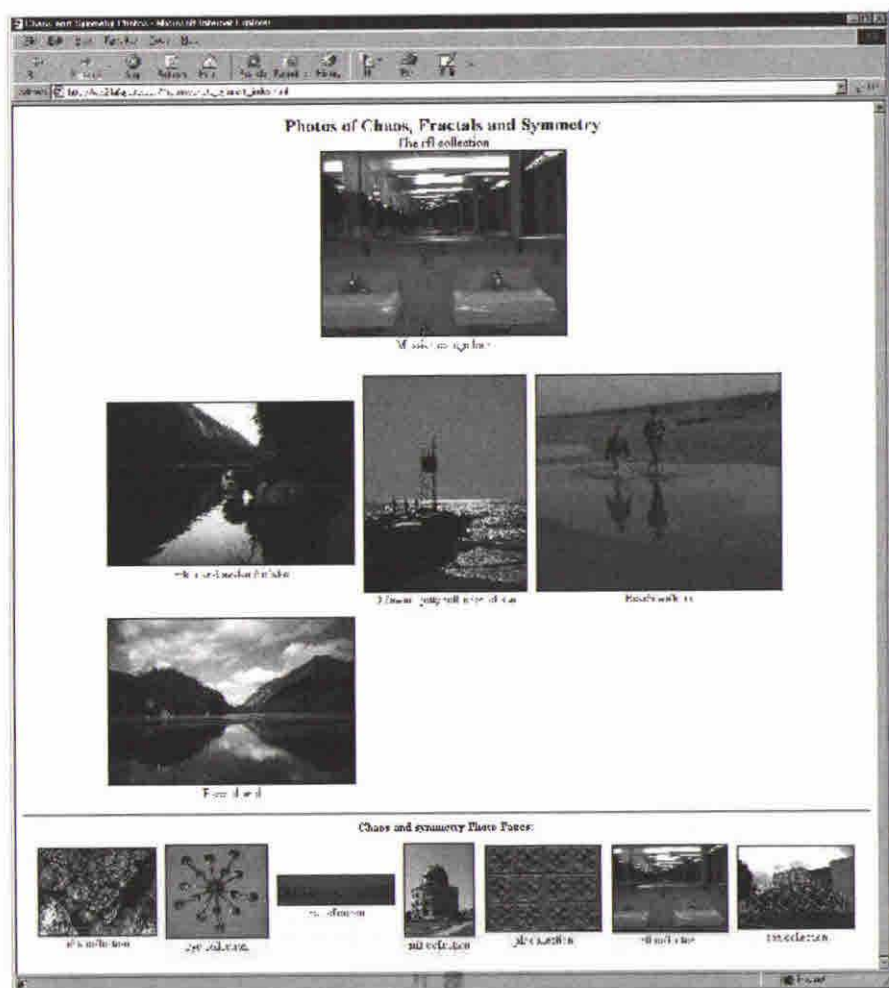


Figure 2. A gallery of reflection pictures under development.

The default descriptive text associated with each image in a image gallery created with `mk_html_gallery` is the name of the image. In the `image2` addon, functions determine the html text to prepend and append to the thumbnail image that appears in each table data entry. The default functions are shown below.

```

td_prolog
+---+-----+
|3|:| '<center>' |
+---+-----+

td_postlog
+---+-----+
|3|:| '<br>', CRLF, (short_fn >{.y.}), '</center>' |
+---+-----+

```

The script sample2.ijs defines a function `get_gallery_itd` that results in a boxed array giving the image names, thumbnail names, and descriptive text that it locates within the html file which is its argument. Here is a truncated example from the image galleries [6].

```

$ get_gallery_itd '\i\www_photo_symm\cyc_index.html'
15 3

4{. get_gallery_itd '\i\www_photo_symm\cyc_index.html'
+-----+-----+-----+
|cyc_index.jpg      |th_cyc_index.jpg      |A chandelier      ...
+-----+-----+-----+
|cyc_dcwmdome.jpg   |th_cyc_dcwmdome.jpg   |Washington DC war ...
+-----+-----+-----+
|cyc_del_circle.jpg|th_cyc_del_circle.jpg|Delaware beach cro...
+-----+-----+-----+
|cyc_j_disc.jpg     |th_cyc_j_disc.jpg     |Cyclic patterns an...
+-----+-----+-----+

```

The function `mk_desc_td_postlog` makes a replacement `td_postlog`, using the descriptive text from `get_gallery_itd` when possible and using the filename otherwise. This is illustrated below where we see that the image "cyc_index.jpg" already existed, and hence the existing description "A chandelier" resulted while the image "testing.jpg" did not exist, so its name was used as a description. Notice that the argument to the function `td_postlog` is a boxed list consisting of the filename and the thumbnail name.

```

mk_desc_td_postlog '\i\www_photo_symm\cyc_index.html'
td_postlog 'cyc_index.jpg'; 'th_index.jpg'
<br>
A chandelier</center>

td_postlog 'testing.jpg'; 'th_testing'
<br>
testing.jpg</center>

```

The function that retrieves the descriptive text strips off leading and trailing white space and html markers. The assumption being that the html text will be added by the prelog and postlog functions. One can "hide" html code simply by inserting nonprintable characters that are not recognized as white space.

Wish List

While the improvements and enhancements in the image2 addon have been quite useful to the authors, several facilities could be useful future extensions. Integration of 8-bit and 24 bit raster graphics would be handy. While that is straightforward conceptually, many details would need to be attended to and accomplishing it a manner that is syntactically simple, convenient, compatible with previous releases and without introducing performance degradation might be challenging.

While the file formats supported include widely used types, other file formats are from time-to-time convenient. Graphics interchange format (*.gif) would be useful, but legal entanglements make offering support for the format unlikely; see [4] for both a discussion of the format and the entanglements. Access to the animation formats used by digital cameras would also be useful. While extracting and writing frames would be of interest in general, even being able to extract a first frame would allow thumbnail generation and hence the inclusion of animations in image galleries could be made routine.

The ability to conveniently select subsets of existing galleries for creating another gallery would be handy. Such a facility could easily grow into a major image management application, but still, often one wants to pick and choose the "best 10 pictures from the hike" and it would be convenient to do that without needing to keyboard image names or numbers.

Lastly, the sample2 gallery maintenance described in the previous section utilized files from several directories and resulted in files in a single directory. We could aim for more automatic construction of linked galleries where the link structure arises from boxing or a directory structure. Far more complex webs of galleries could be maintained with such a facility.

References

- [1] R. Dazeley, *J is for JPEG: Windows Bitmap to JPEG Baseline Compression*, *Vector* 18 1 (2001) 103-124.
 - [2] Independent JPEG Group, source code jpegsrc6b, <http://www.ijg.org/>
 - [3] Jsoftware, Downloads, Addons, Image2 Addon, at <http://www.jsoftware.com>.
 - [4] John M. Miano, *Compressed Image File Formats*, Addison-Wesley-Longman, 1999.
 - [5] C. Reiter, *Web Image Galleries made Routine*, *Vector* 18 1 (2001) 69-82, <http://www.vector.org.uk/v181/cliff181.htm>.
 - [6] C. Reiter, *Photos of Chaos, Fractals and Symmetry*, http://www.lafayette.edu/~reiterc/photo_symm/index.html.
 - [7] G. E. Schlnat, A. Dilger, G. Randers-Pehrson, et al., libpng 1.0.8 reference library, <http://www.libpng.org/pub/png/libpng.html>.
-

SHARP APL

SHARP APL 6.0 for UNIX & Linux

- Full implementation of SHARP APL
- Based on Iverson's APL Dictionary
- Rich in functionality
- Comprehensive documentation
- Linux version free for personal use

SHARP APL Java Interface

- Available for SHARP APL 6.0
- Platform independent Graphical User Interfaces
- Access from APL to Java, including Java services such as JDBC
- Access from Java to APL
- Free with personal use version of SHARP APL for Linux

For more
information contact:

Europe,
Benoit Paquin,
benoit_paquin@soliton.com

North America,
Nancy Lamb,
nancy@soliton.com

Download
SHARP APL for Linux
www.soliton.com/Linux



**SOLITON
ASSOCIATES**
Enterprise Software Solutions

Havsgaardsvej 4
DK 2900 Hellerup
Denmark
Tel: +45.2621.6099
Cell: +45.3940.1641

44 Victoria Street, Suite 2100
Toronto, Ontario,
Canada M5C 1Y2
Tel: 416.364.9355
Fax: 416.364.6159

1100 University Avenue,
Suite 111
Rochester, NY, USA 14607
Tel: 716.256.6466
Fax: 716.256.6469

TECHNICAL SECTION

This section of VECTOR is aimed principally at those of our readers who already know APL. It will contain items to interest people with differing degrees of fluency in APL.

Contents

Hacker's Corner: Ghost Variables		90
At Play with J: The Counterfeit Coin Problem	Eugene McDonnell	93
Bach's Endlessly Rising Canon	Ray Cannon	104
Extending the Domains	Phil Chastney	115

Erratum – Function Models

In Vector 18.2 (page 134 is typical) a change to the mapping in Phil's Unicode font caught out Production in a big way. The symbols are still used consistently, but where you see 1/7 you should really see Ø and 8/9 and 1/9 are U and I. Finally for 5/9 read S throughout. Hopefully, this one will get it right! The 'January 2001' version of the font on the Vector website is the correct one. Sorry, Phil.

Hacker's Corner: Ghost Variables

by Ray Cannon

*As I was going up the stairs,
I met a man, who was not there,
He was not there again today,
I do wish he would go away.*

How to Emulate "Persistent Local Variables" in Dyalog APL/W.

Stephen Taylor's article on "Persistent Local Variables", and recent exchanges in the "DFNS discussion group" have prompted me into writing these notes.

In APL, every time a function is called, any LOCAL variable defined within the function needs to be assigned before it can be used. That is on subsequent calls, the function retains no memory of its local variables' previous values. Outside the function, these local variables have no presence.

However, if the function refers to a GLOBAL variable, once initially assigned, no further assignments are required, and values are maintained between calls. Outside the function, these global variables do have a presence, and can be "molested".

It would be very useful if we could define a third type of variable, which although local to the function (*i.e.* have no presence outside the function) could retain their value between calls. This is what is meant by the term "Persistent Local Variable".

As way of an example consider the dynamic function *Next*:

```
Next←{
  w≠'':α.next←w      ⍝ Assign a new starting value
  α{α.next}α.next++1 ⍝ Return incremented value
}
```

When called, its left argument is a reference to a namespace, in which it stores its GLOBAL variable "next". Its right hand argument may be either a numeric which it uses to initialise itself, or the empty text vector '', in which case, it increments the value of the global variable, and returns that value as its result.

```

      □SE.Next 0
      □SE.Next ''
1
      □SE.Next ''
2

```

Note that the global variable “next” is located in the quadSE namespace:

```

      □SE.□nl 2
next
      □SE.next
2

```

Now Dyalog APL allows one to assign functions and derived functions. So I can “glue” the quadSE namespace onto the *Next* function, and call it *NextOne*:

```

      NextOne←(□se)•Next
      NextOne 10
      NextOne ''
11
      NextOne ''
12

```

But this is still using the same global variable

```

      □SE.next
12

```

However, if instead of using an existing NAMED namespace, I create an UN-NAMED namespace

```

      NextTo←(□NS'')•Next
      NextTo 50
      NextTo ''
51
      NextTo ''
52

```

The global variable *next* exists in an UN-NAMED namespace to which I have NO access except via the derived function *NextTo*.

Note that if I create another Un-Named namespace "It" does not effect the first UN-NAMED namespace:

```

      NextFree+([NS''])*Next
      NextFree ~50
      NextFree ''
-49
      NextFree ''
-48
      NextTo ''
53
      NextOne ''
3

```

So there we have it, outside the function, a variable that has no presence, but whose value persists from one call to the next.

NextTo and *NextFree* both have their own separate copy of a "GLOBAL" (but invisible) variable called *#.[Namespace].next*. One could describe them as functions "HAUNTED" by their "GHOST" variables.

Since these GHOST variables are really GLOBAL, one should be able to share them between two functions. (In FORTRAN I think this is described as COMMON data.)

Well, since "Real programmers can write FORTRAN in any language"....

Operator "attach" should do the trick:

```

a(aa attach ww)w
  s(1>a),'+w*aa'
  s(2>a),'+w*ww'

'PUT' 'GET'({a.data+w} attach {a.data}) [NS'']

```

This creates functions *Put* and *a* which share the same COMMON but private variable *#.[Namespace].data*

```

      Put 1 2 3
      Get ''
1 2 3
      Put 'Hello Sprite World'
      Get ''
'Hello Sprite World'

```

At Play With J: The Counterfeit Coin Problem

by Eugene McDonnell

The Counterfeit Coin Problem, unlike many mathematical puzzles, is not a creation of ancient times nor of the 19th century, and does not appear in the classical works of Loyd, Dudeney, Ball, or Kraitchik. It springs from a problem posed by E. D. Schell in the January 1945 issue of the American Mathematical Monthly:

You have eight similar coins and a beam balance. At most one coin is counterfeit and hence underweight. How can you detect whether there is an underweight coin, and if so, which one, using the balance only twice?

Try solving this. I give my solution at the end of this paper.

Most of the solutions I have seen for this kind of problem give an initial allocation of the coins to the balance's pans, and on the basis of the weighing give another allocation, and so on. In 1978 J. G. Mauldon published an IBM Research Report RC 7476, in which he gave a solution in which the weighings were predetermined, not a result of a previous weighing, entitled "Strong Solutions for the Counterfeit Coin Problem". His statement of the problem is generalized from Schell's. I'll call this the first problem:

Given C coins, of which it is suspected that one (at most) is counterfeit (either underweight or overweight), it is required, in at most W weighings on an ordinary beam balance, to identify the counterfeit (if present) and to determine whether it is heavy or light.

He defines a strong solution to which "The choice and distribution of coins for each weighing is to be independent of the other weighings." In support of his method, he proves the theorem that "if the pair (W,C) admits a solution at all, then it admits a strong solution." Thus, no other solution can be better than his strong technique. His method is array oriented, and he gives a suite of APL direct definition functions which give a solution to the problem, acknowledging "his indebtedness to the encouragement and valuable advice of Dr. Kenneth Iverson." He also gives solutions for a second problem, where we are given that exactly one coin is counterfeit, and we are not required to specify whether it is heavier or lighter, but merely to identify it, and a third problem in which in addition to the given set of coins, we are allowed to incorporate into the weighings an arbitrary

number of coins known to be of standard weight. I'll only discuss the first problem. His solutions to all three problems are similar.

He defines a solution as a table of weighings showing the allocation of coins at each weighing. In table (A) are his solutions for all nine cases for which W is 3. These are for values of C from 4 through 12, inclusive.

C = 4, 7, 10	C = 5, 8, 11	C = 6, 9, 12
0 0 1 2	1 1 0 2 2	0 1 2 0 1 2
0 2 1 0	1 2 2 1 0	0 1 2 1 2 0
1 0 0 2	2 1 2 1 0	1 2 0 0 1 2
0 1 2 0 0 1 2	2 0 1 1 1 0 2 2	0 1 2 0 1 2 0 1 2
1 2 0 0 2 1 0	2 1 0 1 2 2 1 0	0 1 2 1 2 0 1 2 0
2 0 1 1 0 0 2	2 0 1 2 1 2 1 0	1 2 0 0 1 2 1 2 0
0 1 2 0 1 2 0 0 1 2	0 1 2 2 0 1 1 1 0 2 2	0 1 2 0 1 2 0 1 2 0 1 2
1 2 0 1 2 0 0 2 1 0	1 2 0 2 1 0 1 2 2 1 0	0 1 2 1 2 0 1 2 0 1 2 0
1 2 0 2 0 1 1 0 0 2	2 0 1 2 0 1 2 1 2 1 0	1 2 0 0 1 2 1 2 0 2 0 1

(A)

(A)

Mauldon calls the solution for twelve coins *maximal*. A maximal solution is one in which C is the largest number of coins admitting a solution for a given W . Solutions for less than the maximal number of coins he calls *submaximal*. He tackles maximal solutions first, for these are used in forming submaximal solutions as well.

Each solution has three rows, one for each of the three weighings, and four through 12 columns, one for each of the coins. Each row gives an allocation of the coins as being either set aside, or put in the left pan, or put in the right pan, represented by 0, 1, or 2, respectively. Notice that in each weighing the number of coins placed on the left pan is the same as the number placed on the right pan, that is, the solutions are *balanced*. For example, in the four-coin case the first weighing sets aside coins 0 and 1, puts coin 2 in the left pan, and coin 3 in the right pan. When $3|C$ is one, that is, for C of 4, 7, and 10, the number of coins set aside is one more than the number on each pan. When $3|C$ is 2, for C of 5, 8, and 11, the number of coins set aside is one less than the number on each pan. When $3|C$ is 0, as in the right column, for 6, 9, and 12, the number of coins set aside is the same as the number on each pan. At each weighing, the possible results are: the pans are level, or the left pan is lower, or the right pan is lower, represented by 0, 1, and 2, respectively. The result of all three weighings is thus a list V of three items, chosen from 0 1 2.

For example, if there are four coins, and the counterfeit is coin 0, and is heavier than a good coin, the result of the first weighing is 0, since coin 0 is set aside; for

the same reason, the result of the second weighing is also 0; in the third weighing coin 0 is in the left pan, so the result is 1. The overall result V is thus 0 0 1, and this corresponds to column 0 of the 4-coin solution. Additional complications come about if the counterfeit is lighter, for which table (B) is appropriate:

10 0 2 1	2 2 0 1 1	0 2 1 0 2 1	
10 1 2 0	2 1 1 2 0	0 2 1 2 1 0	
12 0 0 1	1 2 1 2 0	2 1 0 0 2 1	
10 2 1 0 0 2 1	1 0 2 2 2 0 1 1	0 2 1 0 2 1 0 2 1	
12 1 0 0 1 2 0	1 2 0 2 1 1 2 0	0 2 1 2 1 0 2 1 0	
11 0 2 2 0 0 1	1 0 2 1 2 1 2 0	2 1 0 0 2 1 2 1 0	(B)
10 2 1 0 2 1 0 0 2 1	0 2 1 1 0 2 2 2 0 1 1	0 2 1 0 2 1 0 2 1 0 2 1	
12 1 0 2 1 0 0 1 2 0	2 1 0 1 2 0 2 1 1 2 0	0 2 1 2 1 0 2 1 0 2 1 0	
12 1 0 1 0 2 2 0 0 1	1 0 2 1 0 2 1 2 1 2 0	2 1 0 0 2 1 2 1 0 1 0 2	

The entries in (B) are not used for allocating the coins, but rather to determine the false coin when it is lighter than the good coins. For example, if there are four coins, and the false coin is coin 0, and is *lighter* than the others, the result would be 0 0 2, corresponding to column 0 of the 4-coin table in (B). The tables in (B) are the 3s complement of those in (A).

The table below gives some of the vital statistics of the problem:

W	N	L	G
3	9	4	12
4	27	13	39
5	81	40	120
6	243	121	363
7	729	364	1092
8	2187	1093	3279

Column W gives the number of weighings required, column N gives the number of different cases that W weighings can solve, column L gives the least number of coins for W , and column G gives the greatest number of coins for W . For example, if W is four, 27 cases are solvable, with 13 coins the smallest case, and 39 coins the largest. I don't show the case for W of 2, since three is the only meaningful case. A one-coin case admits of no comparisons, and a 2-coin case can't discriminate between heavier and lighter coins.

Given any number of coins K between L and G inclusive, W may be found from K by taking the ceiling of the base-3 log of 3 plus twice K :

```
WK =: >.@(3&^.)@(3&+ )@(2&* )
WK 4+i.9
3 3 3 3 3 3 3 3 3
WK 13+i.27
4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4
```

N is a power of three, namely 3^{W-1} , and is included between L and G , and is the only power of 3 included. Column L is the sum-scan of powers of 3; the first item is $1+3$, the second is $1+3+9$, and so forth. Column G is thrice column L , and is also given by

((3^W)-3)%2
12 39 120 363 1092 3279

Given any positive integer q greater than one, its representation in base q has 1 as its most significant digit (msd), and is in fact 10. For any q greater than 2, its double is represented in base q by 20. Its square is 100 and the square's double is 200. Its cube is 1000, and so forth. All the numbers from q up to but not including its double have msd of 1. All the numbers from p any power of q up to but not including its double have msd of 1. For example, the ternary representations 1 and 2 are 1 2; of 3 4 5 6 are 10 11 12 20; of 9 through 18 are:

100 101 102 110 111 112 120 121 122 200

There are L integers less than N that have msd of 1. For example, if W is 3, then N is 9 and there are 4 ternary integers with msd 1 less than 9; these are 1, 3, 4, and 5, which have ternary representations of 1, 10, 11, and 12. When represented in a radix large enough to represent N , the msd numbers have leading zeros. For example, 9 is represented by 100, and those for 1, 3, 4, and 5 are then 001, 010, 011, and 012.

Constructing a maximal solution

We'll use the case where W is 3 to exemplify the general case.

Start with the ternary representations of the msd numbers less than 3^W-1 . In our case these are 1 3 4 5.

First we produce the powers of 3 less than 3^W-1 :

```
W =: 3
3 ^ i. <: W
1 3
```

Next we use the hook (+i.) with each of these:

```
(+ i.) &.> 3 ^ i. <: W
+---+
|1|3 4 5|
+---+
```

Last, we raze this:

```
; (+ i.) &.> 3 ^ i. <: W
1 3 4 5
```

We convert these to ternary, getting four distinct representations:

```
] za =: (W # 3) #: ; (+ i.) &.> 3 ^ i. <: W
0 0 1
0 1 0
0 1 1
0 1 2
```

Each row of za is used to create two additional rows by adding 1 and 2, mod 3, to it. Adding 1, mod 3, to 0 1 2 gives 1 2 0; adding 2 to 0 1 2 gives 2 0 1. Consequently each three-row subtable has distinct rows. Notice that each column is also balanced, having one each of 1 and 2.

```
] zb =: 3|0 1 2+/"1 za
0 0 1
1 1 2
2 2 0
0 1 0
1 2 1
2 0 2
0 1 1
1 2 2
2 0 0
0 1 2
1 2 0
2 0 1
```

This is turned into a single table by applying append insert (.,/) to it. Since the individual columns of the subtables were balanced the whole column is also balanced.

In this case each weighing places four coins in each pan.

```

      ,/ zb
0 0 1
1 1 2
2 2 0
0 1 0
1 2 1
2 0 2
0 1 1
1 2 2
2 0 0
0 1 2
1 2 0
2 0 1

```

I'll transpose this to allow you to more easily to compare with the lower right-hand corner of table (A):

```

] zc =: |: , / 3 | 0 1 2 +/ "1 zb
0 1 2 0 1 2 0 1 2 0 1 2
0 1 2 1 2 0 1 2 0 1 2 0
1 2 0 0 1 2 1 2 0 2 0 1

```

The entire maximal solution process can be encapsulated in monad *SX*, which takes the number of weighings as argument.

```

SX =: 13 : ' ,/ 3 | 0 1 2 +/"1 (y. # 3) #: ; (+ i.) &. > 3 ^ i. <: y.'
|: SX 3
0 1 2 0 1 2 0 1 2 0 1 2
0 1 2 1 2 0 1 2 0 1 2 0
1 2 0 0 1 2 1 2 0 2 0 1

```

Constructing a general solution

Mauldon's method of obtaining a general solution involves a fairly complicated way of choosing one of two tables to be appended, in the cases where the number of coins has a 3-residue of 1 or 2, or no appended table for the case where the 3-residue is 0. It simplifies things considerably if a third, empty table, is provided for this last case. If we call the appended tables *A0*, *A1* and *A2*, for residues of 0, 1 and 2, respectively, and form them into a list of boxes *AS*. Mauldon doesn't give the principles used in constructing *A1* and *A2*, he merely presents them without apology.

```

] AS =: A0;A1;A2
+---+-----+-----+
|  | 0 0 1|2 2 2|
|  | 0 2 0|0 1 0|
|  | 1 1 0|1 0 1|
|  | 2 0 2|1 1 2|
|  |      |1 2 1|
|  |      |0 2 2|
|  |      |2 1 1|
|  |      |2 0 0|
+---+-----+-----+

```

Each of these is balanced, and the last five rows of A2 are also balanced. If K is the number of coins in C, the proper table to append can be given by:

```

> AS {~ 3 | K
2 2 2
0 1 0
1 0 1
1 1 2
1 2 1
0 2 2
2 1 1
2 0 0

```

This isn't all that is needed. The three columns in the tables are suited to the least number of weighings that may be required. If more weighings than three are needed, the first column is replicated $W-2$ times. For example, if W is 5 there are five weighings and the first column is replicated thrice:

```

((W-2)#0), 1 1#*1 A2
2 2 2 2 2
0 0 0 1 0
1 1 1 0 1
1 1 1 1 2
1 1 1 2 1
0 0 0 2 2
2 2 2 1 1
2 2 2 0 0

```

A dyad SA to produce a table to append having the necessary number of columns can thus be given by:

```

SA =: 13 : '((y.-2)#0),1 2){"1>AS{~3|x.'

```

where x. is the number of coins and y. is the number of weighings.

The two dyads SX and SA can be combined to give a general solution function SG :

```
SG =: 13 : '(-x.){.x.(SX,SA)y.'
```

where x . and y . are as in SA . The phrase $(-x.){}$ forms the solution by taking the last x . rows of the table formed by appending the maximum and the appended tables.

A solution for the case of 8 coins and 3 weighings can be obtained by:

```
8 SG 3
2 2 2
0 1 0
1 0 1
1 1 2
1 2 1
0 2 2
2 1 1
2 0 0
```

I found it convenient to have a monad which takes a list of coins as argument. The monad is:

```
SC =: (SG WK)@#
C =: 0 0 0 0 0 0 1 0 0
SC C
2 2 2
0 1 0
1 0 1
1 1 2
1 2 1
0 2 2
2 1 1
2 0 0
```

Finding a false coin

We can find a solution by writing:

```
S+//."1&. |:C
```

The dual transpose ($\&. |:$) causes the arguments to be transposed before being used. This has no effect on the coin list C , but is effective on S , interchanging columns and rows. The rank one ("1) allows the rows of transposed S to be used individually with C . The sum by key ($+//.$) adds the items of C according to the keys in the row of transposed S .

The result of sum key of the first row with C is:

```

    2 0 1 1 1 0 2 2 +//. 0 0 0 0 0 1 0 0
  0 1 0

```

The result comes from summing all the items of the right argument corresponding to 2s in the left argument (0), then those corresponding to 0s (1) then those corresponding to 1s (0).

The same thing occurs when I use *all* the rows of transposed S with C:

```

    S +/ /. "1 &. |: C
  0 1 1
  1 0 0
  0 0 0

```

This is difficult to interpret because the first column 0 1 0 is in the order 2 0 1, the order in which they occur in the first column of S; the second column 1 0 0 is in the order 2 1 0; the third column is in the order 2 0 1.

In order to avoid this difficulty, I prefix the solution rows (transposed columns) with 0 1 2 and the coin list with 0 0 0. The prefixed zeros on the coin list can't alter the result, but the 0 1 2 prefixed to the columns ensures that the result comes in a way that is easy to interpret.

```

    ] zr =: }. S (0 1 2&,@[ +//. 0 0 0&,@])"1&. |: C
  0 0 0
  0 1 1

```

I drop the first row of the result, which corresponds to the coins set aside, because in the physical experiment these are not seen. The table zr is interpreted thus: the rows correspond to left pan and right pan, and the columns correspond to weighings. In the first weighing the left and right pans were level. In the second and third weighing the right pan was lower.

Now, if I take the difference of the left and right pan rows I get:

```

    -/}. S (0 1 2&,@[ +//. 0 0 0&,@])"1&. |: C
  0 _1 _1

```

And I need the 3s-complement of this:

```

    3|-/}. S (0 1 2&,@[ +//. 0 0 0&,@])"1&. |: C
  0 2 2

```

This tells me that in the first weighing the pans were level, and in the last two the right pan was lower.

The dyad *WR* encapsulates the preceding steps:

```
WR =: 13 : '3|-/ }. x. {0 1 2&,@[ +//. 0 0 0&,@])"1&. |: y.'
] WL =: S WR C
0 2 2
```

The next step is to find the index of *WL* in either *S* or its 3s complement. The 3s complement is formed by taking the 3-residue of *-s*. This is laminated (*,:*) to *S*, forming *SC*:

```
] SC =: (, :3&|@-)S
2 2 2
0 1 0
1 0 1
1 1 2
1 2 1
0 2 2
2 1 1
2 0 0
1 1 1
0 2 0
2 0 2
2 2 1
2 1 2
0 1 1
1 2 2
1 0 0
```

By using rank 2 1 with the index of function, we can obtain the indices in both planes of *SC*:

```
SC i."2 1 WL
5 8
```

Since there are 8 items in each of the tables, the result 5 8 means that the weighing list was found in item 5 of the first table, and not at all in the second. This sequence is encapsulated in dyad *WI*:

```
WI =: 13 : '((, :3&|@-)x.)i."2 1 y.'
S WI WL
5 8
```

This has all the information needed for the answer. The index is clearly the smaller of the two values. The heavier or lighter indication is given by whether

the index is in the first or second table: if in the first, it is heavier; if in the second it is lighter. The final result can thus be given by *RW*:

```
RW =: 13 : '(<./y.),({&_1 1)</y.)'
RW 5 8
5 1
```

This says that coin 5 is false, and it is heavier than a good coin.

To complete the problem, it would be necessary to provide for the case where there isn't a false coin. What happens then?

```
S WI 0 0 0
8 8
RW 8 8
8 _1
```

So the result when there is no false coin is an impossible index and a lighter coin indication.

Solution to Schell's Problem

Assume the eight coins are labeled A B C D E F G H. Then the steps below show how to solve the problem with two weighings. The first column gives the step number, and the next two columns give the allocation of coins to pans. The three columns at the right indicate the result of the weighing. After step 1, only one of either step 2 or step 3 or step 4 is executed. Each of these steps has 3 possible outcomes. For example, if the result of step 1 is "right pan high", go to step 3, which tells us to place coin D in the left pan and coin E in the right pan. If the left pan is now high, this means that D is the false coin, and so forth.

step	left pan	right pan	left pan high	right pan high	pans level
1	A B C	D E F	go to step 2	go to step 3	go to step 4
2	A	B	A	B	C
3	D	E	D	E	F
4	G	H	G	H	none

Bach's Endlessly Rising Canon

by Ray Cannon (ray_cannon@compuserve.com)

Sending MIDI Messages to a Sound-Card with J

This article by Martin Neitzel appeared in Vector Vol.16 No3, fascinated me, so I downloaded the latest version of J and tried it out. It all worked, but being a long-sighted APL bigot, (I do not get on well with J, as I can't tell a ":" from a ";" without a magnifying glass <grin>) I decided to convert Martin's code into Dyalog APL/W. For the remainder of this article, I will assume you have read Martin's article.

Sending MIDI Messages to a Sound-Card with Dyalog APL/W

(All the code is available on-line in a Dyalog APL/W version 9 workspace from the Vector Web site <http://www.vector.org.uk/resource>.)

First the WIN32 API calls used need to be "mapped" into `⎕NA` calls. All the MIDI related calls are in the WINMM.DLL (Windows Multi-Media), which should be located in the Windows System directory.

```
'gnd'⎕NA'I winmm.C32|midiOutGetNumDevs '
'gdc'⎕NA'I winmm.C32|midiOutGetDevCapsA
      U2 >{U2 U2 U4 T['',(*MAXPNAMELEN),']} U2 U2 U2 U2 U4 } U2'
'open'⎕NA'U winmm.C32|midiOutOpen =U U U U U'
'close'⎕NA'U winmm.C32|midiOutClose U'
'sm'⎕NA'U winmm.C32|midiOutShortMsg U U'
```

There, all the hard work is done, now you can write your great OPUS in A flat.

How many MIDI devices are on my PC and what can they do?

To find out the number of Midi devices , use `midiOutGetNumDevs`, this is a niladic function and simply returns the total number of Midi devices installed on your PC.

To find out the capabilities of each device use `midiOutGetDevCapsA`. This takes 3 arguments, the device number (origin zero), a pointer to the MIDIOUTCAPS structure, and the size of this structure. (See MIDIOUTCAPS in appendix 1)

The function *MidiCapabilities*, does this all for you.

```

    v r+MidiCapabilities;nos;[]IO;list;gnd;chnls;chnl;qname;dnos;names;rc;cap1;gdc;no;name;types;MAXPNAMELEN;sz;wMid;wPid;vDriverVersion;szPname;wTechnology;wVoices;wNotes;wChannelMask;dwSupportwMid;wPid;vDriverVersion;szPname;wTechnology;wVoices;wNotes;wChannelMask;dwSupport
[1]      a Describe the capabilities of the Midi devices on this machine.
[2]      a Returns a 3 element nested array of <nv device number> <v of Midi device names> <nv of Midi device types>
[3]      []IO+0      a I hate origin 0 but it make life easier here, just
[4]      MAXPNAMELEN+32 a max length of product name (including NULL)
[5]      a Define []NA calls
[6]      'gnd'[]NA'I winmm.C32|midiOutGetNumDevs '
[7]      'gdc'[]NA'I winmm.C32|midiOutGetDevCapsA U2 >{U2 U2 U4 T[',(wMAXPNAMELEN),']} U2 U2 U2 U2 U4 } U2'
[8]      aFind out how many Midi output devices are connected
[9]      nos+gnd
[10]     'This Machine has ',(wnos),' Midi output devices connected to it. The y are:'
[11]     sz+{/2 2 4 MAXPNAMELEN 2 2 2 2 4 a Calculate the size of the MIDIOUTCAP structure
[12]     names+0p<'
[13]     types+10
[14]     dnos+10
[15]     chnls+0p<10
[16]     a list of synthesizer by type
[17]     list+' 'output port' 'generic internal synth' 'square wave internal synth' 'FM internal synth' 'MIDI mapper'
[18]     list,+hardware wavetable synthesizer (I think)' 'software synthesizer (I think)'
[19]     :For no :In wnos      a Loop for each device found
[20]     ''
[21]     rc cap1-gdc no sz sz      a Read the capabilities
[22]     :If rc=0      a Check that it worked ok
[23]     a Split up MIDIOUTCAP str
[24]     wMid wPid vDriverVersion szPname wTechnology wVoices wNotes wChannelMask dwSupport+cap1
[25]     name+(szPname1=AV)+szPname      a Extract the Product name
[26]     qname+'',name,''      a and put quotes round it
[27]     :Select wTechnology      a Process info by device type
[28]     :Case 1
[29]     'Midi Device:',qname
[30]     'A midi output port which requires a MIDI instrument to be connected to it. (Ignored by this WS).'
[31]     :CaseList 2 3 4 6 7
[32]     'Midi device:',qname
[33]     'Is a ',(wTechnology>list),' (type ',(wTechnology), ' device).'
[34]     'It has ',(wVoices),' voices.'
[35]     'Maximum number of simultaneous notes it can play is:',(wNotes),','
[36]     'It support Midi channel numbers ',(chnl+((16p2)wChannelMask)/16),','
[37]     :Select dwSupport
[38]     :Case 1
[39]     'It supports a mono volume control'
[40]     :Case 3

```

```

[41]          'It supports a Stereo volume control'
[42]          :Case 4
[43]          'It supports patch caching.'
[44]          :Case 5
[45]          'It supports mono volume control and patch caching.'
[46]          :Case 7
[47]          'It supports Stereo volume and patch caching.'
[48]          :End
[49]          * Remember name and type of any synthesizer
[50]          names,+<name
[51]          types,+wTechnology
[52]          dnos,+no
[53]          chnls,+<chnl
[54]          :Case 5
[55]          'Midi device:',qname,' is a "MidiMapper",
            used to remap "non-standard Midi Channels", forget it.'
[56]          :Else
[57]          'Midi device of unknown type found:',qname
[58]          :End
[59]          :End
[60]          :End
[61]          r+dnos names types chnls

```

To use a Midi Device

Midi devices need to be "opened" (midiOutOpen), "sent messages" (midiOutShortMsg) and "closed" (midiOutClose).

Opening is easy, just give midiOutOpen the (origin zero) number of the Midi device you want to use. It will return you a handle to that device. You will need this handle to send messages to it and most importantly CLOSE the device. I emphasise this point, as if you do not close it properly, you may not be able to use it again without closing down Windows. (I will cover sending messages in more detail in the next section.)

The function *Midi* demonstrates all of this.

```

* Midi;gnd;gdc;sm;cap;no;nos;rc;cap1;c;open;hand1;hand;close;bin;err;et
xt;names;types;select;channels;dur;data
[1]  * Demo of Midi [NA] calls
[2]  * Display Midi output devices, and get info about any synthesizers
[3]  nos names types channels+MidiCapabilities
[4]  :If 0=pnos
[5]  'Sorry no synthesizers found on this machine.'
[6]  :Else * Set up some calls to the Midi APIs
[7]  'open'[NA]'U winmm.C32\midiOutOpen =U U U U U'
[8]  'close'[NA]'U winmm.C32\midiOutClose U'
[9]  'sm'[NA]'U winmm.C32\midiOutShortMsg U U'
[10] select->pnos          * Select a synthesizers at random
[11] no-select>nos          * Get the internal number of this device
[12] rc hand1+open 256 no 0 0 0          * Open this device

```

```

[13]      :If rc=0                                a If the open worked
[14]      data←BERC select>channels    a "Create" some Music to pisy
[15]      'Using Midi device:',(select>names),' to play Canon'
[16]      hand1 Play data                a Play the "Music"
[17]      bin←close hand1                a Close this device
[18]      :Else                                a Else the open failed
[19]      'An error occurred trying to use ',select>names
[20]      'err'␣NA'U winmm.C32\midiOutGetErrorTextA U =0T U'
[21]      c+150                            a buffer for error message
[22]      rc etxt←err rc(cp'')c
[23]      etxt
[24]      :End
[25]      :End

```

Sending Messages to a MIDI Device

To play a sound, what do we need to tell the device? I have written a function Play, to play some sound that requires knowing:

When to action request - from 0 to n (where n is the length of a piece of string/music).

What request to action "set Instrument" "Start-note" "End-note" "Duration of note" "Other" and "QUIT"

Which channel (instrument) is it played on (See INSTRUMENTS in appendix 2)

How high a pitch - 1-127 (60=Middle C)

How long a note

How loud a volume (velocity) - 0 to 127

I pass this information to Play as a 6-column matrix, 1 row per action.

```

      v hand1 Play data;show;time;bo;more;now;action;bin;tick;channel;volume;
pitch;length;fix;instr;␣IO
[1]  a Play sound defined in <data> on midi device with handle <hand1>
[2]  a data[:;1]↔ When to action request
[3]  a data[:;2]↔ What request to action 'I' 'S' 'E' 'D' 'Q'
                    Instrument/Start-note/End-note/Duration QUIT
[4]  a data[:;3]↔ Which channel (instrument) does it apply to
[5]  a data[:;4]↔ How hight a pitch
[6]  a data[:;5]↔ How long a note
[7]  a data[:;6]↔ How loud a volume (verlocity)
[8]  ␣IO+1
[9]  a Magic numbers used in midiOutShortMsg (128 144 160 176 192)
[10] a 128-143 a stop note on channel 0-15
[11] a 144-159 a start note on channel 0-15
[12] a 160-175 a ? on channel 0-15
[13] a 176-191 a ? on channel 0-15

```

```

[14]  a 192-207 a Set up an instrument on channel 0-15
[15]  fix+((IO+0 o 2561w) a Encode data into midiOutShortMsg required form
[16]  time+~1          a Initialise starting point
[17]  tick+0.01        a Define a default tick
[18]  more+1           a Control flag. Is there any more data?
[19]  :While more       a Loop for next slice of time
[20]      time++1       a Increment timer
[21]      bo+data[;1]ttime a what to action are due NOW
[22]      :If v/bo       a if anything to be actioned
[23]          now+bo/data a select items to actioned now
[24]          show+10
[25]          now+now['EDISQ'&now[;2];] a Sort into order, end a note
                                   before starting a new one.
[26]      :For action :In +now a Loop for each action required
[27]      :Select action[2]    a Process for this action
[28]      :Case 'E'           a End of a note requested
[29]          channel pitch length volume+action[3 4 5 6]
[30]          bin+sm hand1(fix 0 volume pitch(128+channel))
[31]      :Case 'I'           a Set a new instrument up
[32]          channel instr+action[3 4]
[33]      a'Ch:',(vchannel), 'Instr:',v((>"Instruments);instr>Instruments
[34]          bin+sm hand1(fix 0 0 instr(192+channel))
[35]      :Case 'D'           a Duration of "gap"
[36]          tick+action[3]
[37]      :Case 'O'           a Other action
[38]          channel pitch length volume+action[3 4 5 6]
[39]          bin+sm hand1(fix 0 volume pitch(160+channel))
[40]      :Case 'S'           a Start a new note
[41]          channel pitch length volume+action[3 4 5 6]
[42]          show,+pitch volume
[43]          bin+sm hand1(fix 0 volume pitch(144+channel))
[44]      a Now add an action to data to end this note!
[45]          action[1 2]+(time+length)'E' a End time, END action
[46]          data,[1]+action a Append to list of actions
[47]      :Else               a Any other request...eg Quit
[48]          more+0          a Make sure we stop looping
[49]      :EndSelect
[50]      :EndFor
[51]      :EndIf
[52]      [DL tick a Another slice of time flys by, flap flap.
[53]  :EndWhile

```

What to Send?

Since first hearing Pink Floyd's "Echoes" from Meddle and then reading about Bach's "Endlessly Rising Canon" in Douglas R. Hofstadter's "Gödel, Escher, Bach: an Eternal Golden Braid", I have always wanted to "implement" an endlessly rising sound. So here is my chance.

```

v r+BERC channels;IO;notes;rot;voices;starts;i;vols;last;off;channels;
inst;voice;length;offset;when;note;volume;instruments;dur
[1]  a Returns some data to play on supplied <channels>

```

```

      Bach Endless Rising Canon
[2]  a r[;0]↔ When to action request - from 0 to n
      (where n is the length of a piece of string/music).
[3]  a r[;1]↔ What request to action 'I' 'S' 'E' 'D' 'O' 'Q'
      Instrument/Start-note/End-note/Duration/Other/QUIT
[4]  a r[;2]↔ Which channel (instrument) is it played on
[5]  a r[;3]↔ How high a pitch - 1-127 (60=Middle C)
[6]  a r[;4]↔ How long a note
[7]  a r[;5]↔ How loud a volume (velocity) - 0 to 127
[8]  [IO=0 a I hate origin 0 but it make life easier here, just
[9]  channels+64p(channels+9)/channels a Exclude percussion
[10] dur+0.5 a set a duration for each "tick"
[11] length+12 a number of "tick" units to play note for
[12] a Set up 48 different volumes from very soft to very loud
[13] vols+[0.5+127*10*10*(1+(1+124)+2.4)+11 a Log scale
[14] vols+vols,φvols a soft(>0) loud(=127) soft(>0) over 48 steps
[15] a Set up some pitches - range of notes (Middle C=60),
[16] a one per volume C D E F# G# A# c d e f# g# a# etc
[17] notes+22+*(p vols)p2
[18] a Play a lot of notes on different channels at different volumes
[19] r+0p=6p0 a prototype data
[20] r,+c0 'D'dur a First set up duration
[21] :For when :In 1pnotes
[22] r,+c0,'I',when,14a Set a instrument on each channel
[23] :For rot :In 6*18 a 8 voices 6 notes apart
[24] r,+c+when'S'((rotφchannels)[when])((rotφnotes)[when])1
      ((rotφvols)[when])
[25] :End
[26] :End
[27] last+[/>"r
[28] r,+clast,'I',0,126 a Set a instrument
[29] r,+clast'D' 0.1 a Set up duration
[30] r,+c(clast+2)'S' 0 50 30 127 a Applause
[31] last+40+[/>"r a Make sure to stop it at the end
[32] r,+clast'Q' a Quit
[33] r,+16+"r

```

▽

What Next

That's up to you, but I want to know how to set up my own instrument. I would like to be able to play a dog's bark on my computer. (Listen to "Seamus" on Pink Floyd's Meddle) and of course what I really want to do is produce "Cannon's Endlessly Rising Bark".

Appendix 1

MIDIOUTCAPS

The MIDIOUTCAPS structure describes the capabilities of a MIDI output device.

```
typedef struct {  
    WORD    wMid;  
    WORD    wPid;  
    MMVERSION vDriverVersion;  
    CHAR    szPname[MAXPNAMELEN];  
    WORD    wTechnology;  
    WORD    wVoices;  
    WORD    wNotes;  
    WORD    wChannelMask;  
    DWORD    dwSupport;  
} MIDIOUTCAPS;
```

wMid

Manufacturer identifier of the device driver for the MIDI output device.
Manufacturer identifiers are defined in Manufacturer and Product Identifiers.

wPid

Product identifier of the MIDI output device.
Product identifiers are defined in Manufacturer and Product Identifiers.

vDriverVersion

Version number of the device driver for the MIDI output device.
The high-order byte is the major version number,
and the low-order byte is the minor version number.

szPname

Product name in a null-terminated string.

wTechnology

Flags describing the type of the MIDI output device.
It can be one of the following:
MOD_MIDIPORT = 1 ; output port
The device is a MIDI hardware port.
MOD_SYNTH = 2 ; generic internal synth
The device is a synthesizer.
MOD_SQSYNTH = 3 ; square wave internal synth
The device is a square wave synthesizer.

MOD_FMSYNTH = 4 ; FM internal synth

The device is an FM synthesizer.

MOD_MAPPER = 5 ; MIDI mapper

The device is the Microsoft MIDI mapper.

MOD_WAVETABLE = ??? 6

The device is a hardware Wavetable synthesizer.

MOD_SWSYNTH = ??? 7

The device is a software synthesizer.

wVoices

Number of voices supported by an internal synthesizer device.

If the device is a port, this member is not meaningful and is set to 0.

wNotes

Maximum number of simultaneous notes that can be played by an internal synthesizer device.

If the device is a port, this member is not meaningful and is set to 0.

wChannelMask

Channels that an internal synthesizer device responds to, where the least significant bit refers to channel 0 and the most significant bit to channel 15.

Port devices that transmit on all channels set this member to 0xFFFF.

dwSupport

Optional functionality supported by the device.

It can be one or more of the following:

MIDICAPS_CACHE = 0004h

Supports patch caching.

MIDICAPS_LRVOLUME = 0002h ; separate left-right volume control

Supports separate left and right volume control.

MIDICAPS_STREAM

Provides direct support for the midiStreamOut function.

MIDICAPS_VOLUME = 0001h ; supports volume control

Supports volume control.

If a device supports volume changes,

the MIDICAPS_VOLUME flag will be set for the dwSupport member.

If a device supports separate volume changes on the left and right channels, both the MIDICAPS_VOLUME and the MIDICAPS_LRVOLUME flags will be set for this member.

Appendix 2

INSTRUMENTS

```

v r←Instruments
[1]  r Return the "Standard MIDI Patch Assignments" (eg Instrument list)
[2]  r←Vector of 2 element vectors of(<patch no><instrument name>)
[3]
[4]  a Information from:-
[5]  aPlatform SDK: Windows Multimedia
[6]  aStandard MIDI Patch Assignments
[7]  aThe standard MIDI patch assignments for authoring MIDI files
[8]  are based on the MIDI Manufacturers Association specification.
[9]  aFollowing are the standard MIDI patch assignments.
[10]
[11]  aPiano
[12]  r←c0 'Acoustic grand piano'
[13]  r,←c1 'Bright acoustic piano'
[14]  r,←c2 'Electric grand piano'
[15]  r,←c3 'Honky-tonk piano'
[16]  r,←c4 'Rhodes piano'
[17]  r,←c5 'Chorused piano'
[18]  r,←c6 'Harpsichord '
[19]  r,←c7 'Clavinet'
[20]  aChromatic Percussion
[21]  r,←c8 'Celesta '
[22]  r,←c9 'Glockenspiel '
[23]  r,←c10 'Music box'
[24]  r,←c11 'Vibraphone'
[25]  r,←c12 'Marimba'
[26]  r,←c13 'Xylophone '
[27]  r,←c14 'Tubular bells'
[28]  r,←c15 'Dulcimer'
[29]  aChromatic Percussion
[30]  r,←c16 'Hammond organ '
[31]  r,←c17 'Percussive organ '
[32]  r,←c18 'Rock organ'
[33]  r,←c19 'Church organ'
[34]  r,←c20 'Reed organ'
[35]  r,←c21 'Accordion '
[36]  r,←c22 'Harmonica '
[37]  r,←c23 'Tango accordion '
[38]  aGuitar
[39]  r,←c24 'Acoustic guitar (nylon)'
[40]  r,←c25 'Acoustic guitar (steel)'
[41]  r,←c26 'Electric guitar (jazz)'
[42]  r,←c27 'Electric guitar (clean)'
[43]  r,←c28 'Electric guitar (muted)'
[44]  r,←c29 'Overdriven guitar'
[45]  r,←c30 'Distortion guitar'
[46]  r,←c31 'Guitar harmonics '
[47]  aBass
[48]  r,←c32 'Acoustic bass '
[49]  r,←c33 'Electric bass (finger)'

```

```

[50] r,+c34 'Electric bass (pick)'
[51] r,+c35 'Fretless bass '
[52] r,+c36 'Slap bass 1 '
[53] r,+c37 'Slap bass 2 '
[54] r,+c38 'Synth bass 1'
[55] r,+c39 'Synth bass 2'
[56] aStrings
[57] r,+c40 'Violin'
[58] r,+c41 'Viola '
[59] r,+c42 'Cello (The answer to the ultimate question)'
[60] r,+c43 'Contrabass'
[61] r,+c44 'Tremolo strings '
[62] r,+c45 'Pizzicato strings '
[63] r,+c46 'Orchestral harp '
[64] r,+c47 'Timpani '
[65] aEnsemble
[66] r,+c48 'String ensemble 1'
[67] r,+c49 'String ensemble 2'
[68] r,+c50 'Synth. strings 1 '
[69] r,+c51 'Synth. strings 2 '
[70] r,+c52 'Choir Aahs '
[71] r,+c53 'Voice Oohs'
[72] r,+c54 'Synth voice '
[73] r,+c55 'Orchestra hit'
[74] aBrass '
[75] r,+c56 'Trumpet'
[76] r,+c57 'Trombone'
[77] r,+c58 'Tuba'
[78] r,+c59 'Muted trumpet '
[79] r,+c60 'French horn '
[80] r,+c61 'Brass section '
[81] r,+c62 'Synth. brass 1'
[82] r,+c63 'Synth. brass 2'
[83] aReed '
[84] r,+c64 'Soprano sax '
[85] r,+c65 'Alto sax'
[86] r,+c66 'Tenor sax '
[87] r,+c67 'Baritone sax'
[88] r,+c68 'Oboe'
[89] r,+c69 'English horn'
[90] r,+c70 'Bassoon '
[91] r,+c71 'Clarinet'
[92] aPipe Synth '
[93] r,+c72 'Piccolo '
[94] r,+c73 'Flute'
[95] r,+c74 'Recorder '
[96] r,+c75 'Pan flute'
[97] r,+c76 'Bottle blow'
[98] r,+c77 'Shakuhachi '
[99] r,+c78 'Whistle'
[100] r,+c79 'Ocarina'
[101] aLead Synth'
[102] r,+c80 'Lead 1 (square) '
[103] r,+c81 'Lead 2 (sawtooth)'
[104] r,+c82 'Lead 3 (calliope lead)'

```

[105] r,+c83 'Lead 4 (chiff lead) '
[106] r,+c84 'Lead 5 (charang)'
[107] r,+c85 'Lead 6 (voice)'
[108] r,+c86 'Lead 7 (fifths) '
[109] r,+c87 'Lead 8 (brass + lead) '
[110] aPad '
[111] r,+c88 'Pad 1 (new age)'
[112] r,+c89 'Pad 2 (warm)'
[113] r,+c90 'Pad 3 (polysynth) '
[114] r,+c91 'Pad 4 (choir) '
[115] r,+c92 'Pad 5 (bowed) '
[116] r,+c93 'Pad 6 (metallic)'
[117] r,+c94 'Pad 7 (halo)'
[118] r,+c95 'Pad 8 (sweep) '
[119] aSound Effects '
[120] r,+c120 'Guitar fret noise '
[121] r,+c121 'Breath noise'
[122] r,+c122 'Seashore'
[123] r,+c123 'Bird tweet'
[124] r,+c124 'Telephone ring'
[125] r,+c125 'Helicopter'
[126] r,+c126 'Applause'
[127] r,+c127 'Gunshot '

Extending the Domains

by Phil Chastney (*philip_chastney@yahoo.com*)

0. Introduction

We all understand the meaning of the APL expression

$V \uparrow V$.

The question is: how would you generalise that expression to n dimensions?

Our suggestion is that $V \uparrow V$ should be sufficient for all inputs with rank ≥ 1 .

1. Extending a Function's Domain

A function mapping is always defined for the simplest case. This definition is then extended, or can be extended, to higher-order arrays provided the arguments meet certain rules of conformability.

This is another of those neat devices by which APL achieves its remarkably concise expressive power. The idea here is to generalise it a little.

1.1 A function is defined using formal parameters. When the formal parameter is given an actual value, we say the parameter is **bound**. In conventional logic, a parameter is either bound or free and, not for the first time, we find that concepts devised for static logic are inadequate for computing processes. We shall bind a parameter to an actual value one dimension at a time, so that a parameter may be free, partially bound, or fully bound.

There is a long established convention in APL that the leading, leftmost, earlier, slower cycling, outer dimensions represent a higher level of detail, and have a greater importance, than the later dimensions. Reduction, for instance, will, by default, collapse out the last, and therefore least important, dimension.

When an actual value has precisely the number of dimensions specified by the functional model, we say the parameter is **fully bound**; when it has more, it is **partially bound**.

(These are distinct from the similarly named concepts in conventional logic, where an n -place function is "fully bound" when all n parameters are given actual values. If some, but not all, parameters are known, the function is "partially bound". These are not concepts we need at this stage.)

1.2 When an actual value has more dimensions than are required by the functional model, the value is "split" on the leading dimension, the function is applied to each of the "slices" thus produced, and these partial results are then "joined", to give the actual result. This process of binding one (leading) dimension at a time, is applied recursively.

We do not need to define "splitting" and "joining" precisely, since they are not visible outside the interpreter. Suffice to say that one process is the inverse of the other.

With dyadic functions, the left and right arguments are both split at the same time. When both arguments are fully bound, the function can be applied.

For example:

1 0 2 ϕ 3 4 ρ 12

requires us to split each argument on the first dimension:

1	ϕ	1	2	3	4
0	ϕ	5	6	7	8
2	ϕ	9	10	11	12

Now that each pair is fully bound, with a scalar left argument applied to a vector right argument, in accordance with a functional model of $\langle R^*N \rangle \leftarrow \langle N; R^*N \rangle$, we can apply the function 3 times:

2	3	4	1
5	6	7	8
11	12	9	10

and join up the separate result items, to give us our the result proper:

2 3 4 1
5 6 7 8
11 12 9 10

1.3 Should the left argument be fully bound, while the right argument is only partially bound, the "splitting" process continues, and we test for conformability again, using the fully bound left argument with each slice of the right argument.

For example:

10 20 30 + 3 4 p112

requires us to split each argument on the first dimension:

10 + 1	2	3	4
20 + 5	6	7	8
30 + 9	10	11	12

so that we are fully bound on the left, but we need to split again on the (newly created) first dimension of each item in the right argument:

10 + 1	10 + 2	10 + 3	10 + 4
20 + 5	20 + 6	20 + 7	20 + 8
30 + 9	30 + 10	30 + 11	30 + 12

Now that each pair is fully bound, with a scalar left argument and a scalar right argument, we can apply the function 12 times:

11	12	13	14
25	26	27	20
39	40	41	42

which we join

11	12	13	14
25	26	27	20
39	40	41	42

and join again, to give us our the result proper:

11 12 13 14
25 26 27 20
39 40 41 42

Similarly, should the right argument be fully bound, while the left argument is only partially bound, the "splitting" process continues, and we test for conformability again, using the fully bound right argument with each slice of the left argument.

1.4 With nested arrays, the process may well continue until all the outer dimensions have been bound. Binding the last dimension, which is equivalent to

looping through a vector with a scalar index, effectively discloses each nested item in turn, at which point the conformance testing process is applied all over again.

10 20 30 + (99; 2 3 ρ16; 2 3 5)

requires us to split each argument on the first dimension, incidentally disclosing thereby each item in the right argument:

10 + 99
20 + 2 3 ρ16
30 + 2 3 5

The conformability rules are applied anew to each of these items. The first item is already fully bound, left and right; the second item requires two further splits before achieving this happy state, and the third row requires just one more split. The result is a three element nested vector.

1.5 All this assumes that the intermediate results *can* be "joined" in some way: in particular, we wish to preserve rectangularity, and the structure (depth of nesting) should remain unaltered. No "fill" operation is defined.

If the process fails on any one item, the arguments are not conformable and the whole extension process fails. Compare

A← 3 4 ρ 1 0 1 1, 1 1 0 1, 0 1 1 1

B← 3 4 ρ112

A/B

1 3 4

5 6 8

10 11 12

with the following:

A← 3 4 ρ 1 0 1 1, 1 1 0 1, 0 1 0 1

B← 3 4 ρ112

LENGTH ERROR

A/B

^

1.6 Just briefly, a few standard (re)definitions:

Where the mapping for a monadic function is defined for a k -dimensional object, and it is applied to an n -dimensional object (with $n \geq k$, necessarily), the function

is applied to the last k dimensions.

The leading $(n-k)$ dimensions are known, to use a term credited to Paul Berry, as the **frame**. When $(n-k)=0$, the frame is said to be **empty**.

Where the function is defined for any arbitrarily large array (monadic *rho*, for example, or *transpose*) and it is applied to an n -dimensional object, then we take $k=n$, and the frame is necessarily empty. All other functions are, to greater or lesser degree, **pervasive**.

With a dyadic function, the definitions are similar, except that k , n and $(n-k)$ may be different for the left and right arguments. This gives rise to a **left frame** and a **right frame**, which may differ in length.

If the two frames differ by, say, d minor dimensions, the effect of the splitting and joining rules is to notionally extend the shorter frame by appending thereto a further d dimensions. (Note that it is the end of the *frame* that gets extended, not the whole dimension vector.) Not unnaturally, this is known as **frame extension**.

In the particular case where the function returns a scalar result from a scalar argument (left or right or both), and precisely one of the appropriate argument(s) is scalar before any splitting process begins, this feature is known as **scalar extension**¹. This commonly occurs with the scalar dyadics, dyadic epsilon and dyadic iota.

When the left and right frames have the same rank, and the corresponding dimensions in each frame have the same length, the two arguments are said to **conform**.

More generally, if m is the length of the shorter of the left frame and the right frame, and the first m dimensions in each frame are identical in length, the two arguments are said to be **conformable**.

2. Applying the Extended Definition

2.1 The complete model for a function may be the sum of many simple mapping. These simple mappings must be such that we can unambiguously choose which mapping to apply, knowing only the type and rank of the arguments. Otherwise we have a DEFINITION ERROR.

¹⁾ a slightly unfortunate term, since it is actually the *empty frame* which gets extended

(This is actually quite a difficult condition to enforce, in the general case. For example, a function defined over $(\langle \mathbb{R}^* \mathbb{N} \rangle \leftarrow \langle \mathbb{N}; \mathbb{R}^* \mathbb{N} \rangle) \oplus (\mathbb{R}^* \mathbb{N} \leftarrow \langle \mathbb{N}^* \mathbb{N}; \mathbb{R} \rangle)$ cannot sensibly be applied when both arguments are vector. This could be resolved by introducing some sort of ordering of the separate phrases in a function model. Or, more simply, by rejecting such esoteric structures with a NONCE ERROR.)

2.2 A rough definition of the process for applying a monadic function to n -dimensional arguments is given in the box.

2.3 The process of applying a dyadic function is an elaboration thereof.

We check whether we can bind either the left or the right argument.

If we can bind neither,

we compare the leading dimensions of the left and right rank vectors. The corresponding dimensions match if they are equal in length (or, perhaps, if one of them is unit-length).

In the event of a mis-match, we have a LENGTH ERROR.

In the event of a match, we split on the leading dimension, and the process is repeated recursively.

Once one argument is fully bound, we can treat the rest of process like the application of a monadic function. If that process fails to terminate in good order, and there are still candidate models to among, then we split along the leading dimension of the fully bound argument, and call the dyadic process recursively.

If we exhaust the second set of dimensions without finding an appropriate simple mapping, we have a DOMAIN ERROR.

2.4 This process is sufficiently general to accommodate any ad hoc extensions to higher rank.

In most cases, however, the mapping will be something simple like $\mathbb{R} \leftarrow \langle \mathbb{R}; \mathbb{R} \rangle$. Given an expression like `(scalar+matrix)`, we can immediately identify the left and right frames, and set up an addressing loop without needing to go through all this taradiddle.

While ad hoc extensions are deprecated there is no way we can guarantee that they will not occur in user-defined functions. Also, this very complexity may be useful later, when handling sub-components in arrays of user-defined objects.

3. Some Anomalies

3.1 To quote one of Scott and Strachey's typically apposite phrases:

... any attitude of restricting generality is a bad habit which can be misleading in analogous but more complicated situations

Scott and Strachey do not specifically mention the dangers of inconsistent ad hoc extensions, but they too are an obstacle to generality, and we cannot ignore them.

3.2 In 1979, Paul Berry wrote:

"When a function takes two arguments, there may be a requirement that the shape of one argument in some manner conform to the shape of the other. The arguments may have to have the same rank, or the same shape, or to differ in rank or shape in a particular way.

Two arrays whose rank and shape are acceptable for a particular dyadic function are said to be **conformable**. The conformability requirements of a function are described with the definition of each."

Fifteen years later, this paragraph had disappeared from the Sharp APL Reference Manual, to be replaced by a discussion of the more general concepts of *frame* and *cell*, which are realised through the use of operators specific to that product. The designers will have found this approach necessary because, as implied above, the conformability requirements do not follow a consistent pattern. Some of the exceptions are outlined below.

3.3 The obvious functional model for *catenation* of character arrays would be $(H*N) \leftarrow (H*N; H*N)$, that is to say, two vectors are joined to make a new vector. This works most of the time, but, applying the splitting and joining rules, to the expression

$(2\ 3\ \rho\ 'abcdef'), 'XY'$

we note that the right argument is fully bound, but that we need to split the left:

'abc' ,
'XY'
'def' ,
'XY'

At this point, we can apply the function twice, to derive two vectors:

'abcXY'
'defXY'

which are then joined to give the result proper, just as it does in J. In APL, however, to quote page 6-85 of the current Sharp APL manual:

"If the arrays are not of the same rank, then either one of them must be a scalar, or of rank which differs from the other by one. If they differ by one in rank, the array of lesser rank is treated as if it had length 1 along the axis of joining. If one is a scalar, it is treated as an array of the same shape as the other except along the axis of join, which is treated as length 1."

which is a model of clarity in a difficult situation. However, the fact that an explanation is necessary is indicative of an anomaly.

If catenation had the mapping $(H*N) \leftarrow (H*N; H)$, then with

$(2 \ 3 \ \rho \text{'abcdef'}) , \text{'XY'}$

we note that neither argument is fully bound, and we therefore split both

'abc' , 'X'
'def' , 'Y'

At this point, we can apply the function twice, to derive two vectors:

'abcX'
'defY'

which are then joined to give the result demanded by the APL standard.

The catenation function for character arguments, then, uses one of the models

$(H*N) \leftarrow (H*N; H*N)$
 $(H*N) \leftarrow (H*N; H)$
 $(H*N) \leftarrow (H; H*N)$

according to context — a context, moreover which is data-dependent, and can therefore only be determined at run-time. This is not incompatible with our objective of making all expressions automatically extendable to higher dimensions, but it does conflict with the objective of predictability.

Consistency and simplicity must be the priorities. If we adopt the mapping

$(U*N) \leftarrow (U*N; U*N)$

for all cases, then

- i) the function behaves as in J (no special-casing for ranks differing by 1);
- ii) we can predict the function's behaviour when the arguments differ in type.

Given $F \leftarrow \langle A; B; C \rangle$

then $F \leftarrow F, 99 \ 100 \ 101$

is fully bound on both sides, so the expression will deliver the vector

$\langle A; B; C; 99; 100; 101 \rangle$.

It is worth observing that

$3 \downarrow F, 99 \ 100 \ 101$

will deliver the vector

$\langle 99; 100; 101 \rangle$.

The result has acquired another degree of nesting, but is to all intents and purpose indistinguishable from the original vector of $99 \ 100 \ 101$. This is analogous to the way the result of $0.5 + X - 0.5$ can be used in contexts restricted to integer values, even though it is quite possibly stored as floating-point.

3.4 The mapping for both *take* and *drop* on numeric arrays is

$\langle R^*N \rangle \leftarrow \langle N; R^*N \rangle$.

The standard definition for these two functions on higher-dimensions implies a mapping of

$\langle R^*N^*N \rangle \leftarrow \langle N^*N; R^*N^*N \rangle$.

For instance, according to our split/join rules, the expression

$2 \ ^{-3} \uparrow \ 3 \ 4 \ \rho \ 12$

should fail with LENGTH ERROR when the interpreter tries to split and partner off the two elements from the left argument and the three slices from the right argument, whereas the standard definition will return the result

$2 \ 3 \ 4$
 $6 \ 7 \ 8$.

If the *take* function were always applied with the mapping $\langle R^*N \rangle \leftarrow \langle N; R^*N \rangle$ then we would have to use two separate calls to produce the above result, something along the lines of:

$2 \ \uparrow[1] \ ^{-3} \uparrow \ 3 \ 4 \ \rho \ 12$.

3.5 It may be totally useless, but our extension rules allow the following:

$1 \ 6 \ 6 \ 6$
 $1 \ 2 \ 3 \ 4 \ 5 \ 6$
 $1 \ 2 \ 3 \ 4 \ 5 \ 6$

1 2 3 4 5 6 .

We have a problem. The ever popular idiom, $\uparrow\rho\rho A$, should, according to our rules, return a $(1 \times N)$ matrix, given a mapping for monadic iota of $(N^*N) \leftarrow N$.

In fact, the model has been extended on an ad hoc basis to $(N^*N) \leftarrow N \oplus N^*1$. This conflicts with our objective of "automatic extension". If we restrict monadic iota to scalar arguments, then all arguments must be scalarised before being fed to monadic iota. This is easily done using the monadic *head* and *tail* functions which were suggested in "Reflexive Data Domains", with the following definitions:

$$\begin{aligned}\uparrow V &\leftrightarrow (1 0) \rho V \\ \downarrow V &\leftrightarrow 1 \downarrow V.\end{aligned}$$

The idiom $\uparrow\rho\rho A$ now becomes $\uparrow \uparrow \rho \rho A$, and, in general, use of *head* and *tail* will lead to slightly untidier code for some idioms, but shorter code and greater generality for many utilities.

3.6 The reverse happens with *compress* and *expand*, because here we find a scalar can be interpreted as a single-element vector. Somewhat exceptionally, both left and right arguments are interpreted as unit-length vectors, even when both are scalar. A vector result from, e.g. $1/6$, is the only possible consistent treatment, given that $0/6$ must also be a vector. Both instances are clearly ad hoc extensions of the obvious models: $(R^*N) \leftarrow (N^*N; R^*N)$ in the case of replication, and $(R^*N) \leftarrow (2^*N; R^*N)$ in the case of expansion.

As it happens, ad hoc extensions to lower rank cause us no problems, because the split-and-join rules will first find and apply the definition for vectors.

3.7 A slightly different thing happens with the scalar dyadics, where a number of implementations treat single-element vectors as scalars. In such cases, $1\ 2\ 3\ +, 99$ returns a three element vector.

In effect, the unit-length vector has been extended, and this has lead to proposals that all unit-length dimensions be deemed to "match" the corresponding dimension of the other argument. Except that this extension would not occur if the other argument were a scalar.

This is getting messy, but a consistent set of rules is possible. And it could be useful. For instance, with

$$A \leftarrow 3\ 3\ 4\ \rho\ \text{THIS} \diamond B \leftarrow 3\ 4\ \rho\ \text{THAT}$$

we could specify either

$$A + 1\ 3\ 4\ \rho B$$

or

$$A + 3 \ 1 \ 4 \ \rho B$$

which involve changes in the dope vector only, and avoid the usual replication.

3.8 It is really unfortunate, but we have introduced an anomaly of our own: scalar extension does not apply to the left argument of *membership* (ϵ) when the right argument is a domain. The function model is $2 \leftarrow \langle U; D \rangle$.

When the right argument is drawn from the normal data domain, the present model for dyadic epsilon is

$$(\mathbf{2} \leftarrow \langle H; H^*N \rangle) \oplus \mathbf{2} \leftarrow \langle R; R^*N \rangle.$$

Had the function model been $\mathbf{2} \leftarrow \langle U; U^*N \rangle$ we would have been able to look for a given array within a list of arrays, which might have been useful for table lookup, but with this definition the left argument would not be pervasive.

3.9 It might be possible to extend the definition of *assignment*, if we had a definition of "namelists" which we could use as the left argument.

3.10 Domino, inner and outer products, *encode* and *decode* have their own rules of application. Inner and outer products are our first examples of dyadic operations involving domain expressions, and will have to wait; *encode* and *decode* can then be formalised in terms of inner and outer products. The extension of monadic domino to vector and scalar arguments causes us no problems; dyadic domino remains an exception.

4. Conclusions

4.1 This is not a new problem. When Byron wrote

Our souls at last congenial meet,
Nor can thy lot my rank disgrace;
Our intercourse is not less sweet,
Since worth of rank supplies the place.

he was obviously thinking of frame extension. The couple of hundred years that have elapsed since that was written have seen a number of other approaches to the problems of conformance.

4.2 Such approaches have typically concentrated on the problems of making certain functions pervasive on nested arrays. The less complicated approach taken here is to define rules which apply to all functions equally, be they primitive, derived or user-defined. These rules can then be made a part of the

definition of function application, and would not need defining specially for any of the primitives.

4.3 Such an approach requires consistency in the definition of primitive functions, and so the principal costs associated with this approach are

- the need to redefine *catenation*, *take* and *drop* and monadic *iota*;
- which, in turn, makes it incompatible with all existing implementations.

4.4 The benefit is that a function written for the simplest case can be automatically extended by the interpreter to any number of dimensions and any depth of nesting.

This is a goal worth pursuing.

How we get there is another matter.

The naïve formulation given above can be explained and understood in approximately 5 minutes, but it currently stands in need of further work.

4.5 For the working programmer, there is only one sensible course of action: live with the present restrictions until a better implementation appears.

Designers and implementors of new interpreters, however, recognising that almost all implementations are dialects of the standard, need not feel so constrained.

5. References

- 5.1 [anon.], *Sharp APL Reference Manual*
Soliton Associates, Toronto [1994]
- 5.2 Paul Berry, *Sharp APL Reference Manual*
I P Sharp Associates Ltd, Toronto [1979]
- 5.3 George Gordon, Lord Byron, *To E—*.
(in) *Hours of Idleness* [1807]
- 5.4 Dana Scott and Christopher Strachey
Towards a Mathematical Semantics for Computer Languages
(in) *Proc. of Symposium on Computers and Automata* (ed. J Fox)
Polytechnic Institute of Brooklyn Press, New York [1971]

Index to Advertisers

Dyadic Systems Ltd	6
MicroAPL Ltd	2
Strand Software	30
Soliton	88
Vector Back Numbers	79

All queries regarding advertising in VECTOR should be made to Gill Smith, at 01439-788385, Email: apl385@compuserve.com.

Submitting Material to Vector

The Vector working group meets towards the end of the month in which Vector appears; we review material for issue n+1 and discuss themes for issues n+2 onwards. Please send the text of submitted articles (hardcopy with diskette as appropriate) to the Vector Working Group via:

Vector Administration, c/o Gill Smith
Brook House
Gilling East
YORK, YO62 4JJ
Tel: +44 (0) 1439-788385
Email: apl385@compuserve.com

Authors wishing to use Word for Windows should contact Vector Production for a copy of the APL2741 TrueType font, and a suitable Winword template. These may also be downloaded from the Vector web site at www.vector.org.uk

Camera-ready artwork (e.g. advertisements) and diskettes of 'standard' material (e.g. sustaining members' news) should be sent to Vector Production, Brook House, Gilling East, YORK YO62 4JJ. Please also copy us with all electronically submitted material so that we have early warning of possible problems.

The British APL Association

The British APL Association is a Specialist Group of the British Computer Society. It is administered by a Committee of officers who are elected by a postal ballot of Association members prior to the Annual General Meeting. Working groups are also established in areas such as activity planning and journal production. Offers of assistance and involvement with any Association matters are welcomed and should be addressed in the first instance to the Secretary.

2001/2002 Committee

Chairman	Adrian Smith 01439-788385 apl385@compuserve.com	Brook House Gilling East YORK YO62 4JJ
Secretary	Anthony Camacho 0117-973-0036 acam@tesco.net	11 Auburn Road Redland BRISTOL BS6 6LS
Treasurer	Nicholas Small 01223-570850 treas.apl@bcs.org.uk	12 Cambridge Road Waterbeach Cambridge CB5 9NJ
Journal Editor	Stefano Lanzavecchia stf@apl.it	c/o APL Italiana Corso Vercelli 54 20145 - Milano Italy
Projects and Publicity	Dr Alan Mayer 01792-205678x4274 a.d.mayer@swansea.ac.uk	European Business Management School Swansea University Singleton Park, SWANSEA SA2 8PP
Webmaster	Ray Cannon 01252-874697 ray_cannon@compuserve.com	21 Woodbridge Road Blackwater, Camberley Surrey GU17 0BS
Activities	Jon Sandles 01904-612882 jon_sandles@csi.com	138 Burton Stone Lane YORK YO30 6DF
Education	Vacant post	
Administration	Rowena Small 01223-570850 treas.apl@bcs.org.uk	12 Cambridge Road Waterbeach Cambridge CB5 9NJ

Journal Working Group

Editor:	Stefano Lanzavecchia	see above
Production:	Adrian & Gill Smith	Brook House, Gilling East, YORK (01439-788385)
Advertising:	Gill Smith	Brook House, Gilling East, YORK (01439-788385)
Support Team:	Jonathan Barman (01488-648575), Anthony & Sylvia Camacho (0117-973-0036), Ray Cannon (01252-874697), Stephen Taylor (077-1340-0852) Bob Hockstra (01483-771028), Jon Sandles (01904-612882), Marc Griffiths	

Typeset by APL-385 with MS Word for Windows
Printed in England by Short-Run Press Ltd, Exeter

VECTOR

VECTOR is the quarterly Journal of the British APL Association and is distributed to Association members in the UK and overseas. The British APL Association is a Specialist Group of the British Computer Society. APL stands for "A Programming Language" – an interactive computer language noted for its elegance, conciseness and fast development speed. It is supported on most mainframes, workstations and personal computers.

SUSTAINING MEMBERS

The Committee of the British APL Association wish to acknowledge the generous financial support of the following Association Sustaining Members. In many cases these organisations also provide manpower and administrative assistance to the Association at their own cost.

Causeway Graphical Systems Ltd
The Maltings, Castlegate,
MALTON, North Yorks YO17 7DP
Tel: 01653-696760
Fax: 01653-697719
Email: sales@causeway.co.uk
Web: www.causeway.co.uk

Compass Ltd
Compass House
60 Priestley Road
GUILDFORD, Surrey GU2 5YU
Tel: 01483-514500

Dyadic Systems Ltd
Riverside View, Basing Road,
Old Basing, BASINGSTOKE,
Hants, RG24 0AL
Tel: 01256-811125
Fax: 01256-811130
Email: sales@dyadic.com
Web: www.dyadic.com

Dutch APL Association
Postbus 1341
3430BH Nieuwegein
Netherlands
Tel: +31 347 342 337

Insight Systems ApS
Nordre Strandvej 119G
DK-3150 Hellebæk
Denmark
Tel: +45 70 26 13 26
Fax: +45 70 26 13 25
Email: info@insight.dk
Web: www.insight.dk

HMW Computing
Hamilton House,
1 Temple Avenue,
LONDON EC4Y 0HA
Tel: 0870-1010-469
Email: HMW@4xtra.com

MicroAPL Ltd
The Roller Mill, Mill Lane
Uckfield
E. Sussex TN22 5AA
Tel: 01825-768050
Fax: 01825-749472
Email: MicroAPL@microapl.demon.co.uk
Web: www.microapl.co.uk/apl

Soliton Associates Ltd
Havsgårdsvej 4
2900 Hellerup
Denmark
Tel: +45 2621 6099
Email: sales@soliton.com

APL2000
Rapid Application Development
6610 Rockledge Drive, Suite 502
Bethesda MD 20817, USA
Email: sales@apl2000.com
Web: www.apl2000.com